Crypto Guide

Cryptographic primitives from scratch

m@rek.onl

Abstract

Building on the mathematics of the *Math Guide*, this volume of *The Zcash Arboretum* develops from first principles the cryptographic primitives used in Zcash's Orchard protocol: the provable-security model and the hardness assumptions; hash functions and the random oracle model; pseudorandom functions, block ciphers, and format-preserving encryption; symmetric and authenticated encryption and key derivation; key agreement; commitment schemes; digital signatures; interactive proofs, zero knowledge, and SNARKs; and Merkle trees with their vector-commitment abstraction.

Contents

1 The provable-security model

The preceding volume supplied the algebraic objects: groups in which discrete logarithms appear hard, finite fields, and elliptic curves. This volume turns them into cryptography. An *object* is not yet a *guarantee*. To assert that a scheme "is secure" is to make a precise mathematical claim, and to prove it demands a precise mathematical framework in which the claim lives. That framework—the apparatus of modern provable security—is the subject of this section.

The central methodological idea is the following. One almost never knows how to prove unconditionally that a cryptographic scheme is secure; such a proof would typically imply $\mathrm{P} \neq \mathrm{NP}$ and a great deal more. Instead we adopt a *relativised* standard of proof. One isolates a small number of computational problems—factoring, discrete logarithm, decisional Diffie–Hellman, and the like—that have resisted decades of attack, and *reduces* the security of the schemes to the conjectured hardness of those problems. A proof of security is then a proof of an implication: *if* the underlying problem is hard, *then* the scheme is secure. Equivalently, in its contrapositive and more operationally useful form: *any* adversary that breaks the scheme can be mechanically transformed into an algorithm that solves the underlying hard problem. Since no such algorithm is believed to exist, one concludes that no such adversary exists.

To render any of this rigorous, we must pin down four things: what counts as a "scheme" parametrised by a notion of input size; what counts as an "adversary"; what it means for an adversary to "break" a scheme, measured by a quantity called its *advantage*; and what it means for one problem to reduce to another. We develop these in turn, followed by the two idealisations under which proofs proceed (the standard model and the random oracle model), and finally the translation between the clean asymptotic language and the concrete "λ-bit security" numbers that practitioners actually quote.

1.1 The security parameter

Every cryptographic scheme in this monograph is not a single object but an infinite *family* of objects, indexed by a positive integer called the security parameter.

> **Definition 1.1** (Security parameter)**.** The *security parameter*, denoted $\lambda \in \mathbb{N}$, is a positive integer that controls the sizes of all objects in a cryptographic scheme: key lengths, group orders, output lengths, and so on. The system conventionally provides it to every algorithm in unary, written 1^λ (the string of λ ones), so that an algorithm running in time polynomial in the length of its input automatically gets time polynomial in λ.

Two questions immediately arise: why a family, and why unary.

The reason for a *family* is that security is inherently asymptotic. There is no such thing as an absolutely unbreakable scheme with fixed finite key length: an adversary with enough time can always enumerate a finite key space. What we can hope for is that as λ grows, the resources required to break the scheme grow far faster than the resources required to use it. The honest parties' costs (key generation, encryption, signing) should grow *slowly* in λ—polynomially—while the best attack's cost should grow *quickly*—super-polynomially, ideally exponentially. The security parameter is the dial that separates these two growth rates, and security is a statement about the limit $\lambda \to \infty$.

The reason for *unary* encoding, 1^λ rather than the $\lceil \log_2 \lambda \rceil$-bit binary representation of λ, is a bookkeeping convenience that aligns two notions of "polynomial time." Complexity theory measures running time as a function of input *length*. If λ came in binary it would have length $\Theta(\log \lambda)$, and an algorithm "polynomial in its input length" would get only $\mathrm{poly}(\log \lambda)$ steps—not even enough to write down a λ-bit key. Padding the parameter to length λ ensures that "polynomial in the input length" coincides with the cryptographically meaningful "polynomial in λ." This is purely a normalisation; nothing of substance depends on it.

Remark 1.2. Because everything is indexed by λ, every quantity we discuss is really a *function* of λ: the adversary's success probability, its running time, the key length. A single number—"the advantage is at most 2^{-128}"—either fixes a concrete λ or describes the function's behaviour for the λ of interest. Keeping the dependence on λ explicit, at least mentally, prevents nearly every confusion in this subject.

1.2 Probabilistic polynomial-time adversaries

Having fixed how scheme sizes scale, we fix what an attacker may do. We model the honest algorithms and the adversary alike as probabilistic polynomial-time machines.

Definition 1.3 (Probabilistic polynomial-time)**.** An algorithm $\mathcal{A}$ is *probabilistic polynomial-time* (PPT) if it is a probabilistic Turing machine—one equipped with a read-only tape of independent uniformly random bits, its *random tape*—and there is a polynomial p such that on every input x, and for every setting of the random tape, $\mathcal{A}$ halts within $p(|x|)$ steps. Write $y \leftarrow \mathcal{A}(x)$ for the random variable obtained by running $\mathcal{A}$ on x with freshly sampled randomness, and $y := \mathcal{A}(x; r)$ for the deterministic output when the random tape is fixed to the string r.

Several modelling choices warrant comment.

Why polynomial time? Polynomial time is the standard mathematical abstraction of "feasible computation." It is robust—closed under composition, and invariant across all reasonable deterministic computational models (the strong Church–Turing thesis)—which makes it a clean class to quantify over. Crucially, we require the *honest* parties to run in polynomial time too; a scheme is useful only if it can be used efficiently. The security claim is then an asymmetry between two polynomial-time worlds (honest use) and a quantified statement that *no* polynomial-time world (attack) succeeds.

Why probabilistic? Randomness is not a luxury but a necessity in cryptography. Deterministic encryption leaks whether two ciphertexts encrypt the same message; key generation must sample secrets unpredictably. We therefore grant the adversary randomness too, both for fairness and because a deterministic adversary is the special case where the random tape is ignored.

Definition 1.4 (Non-uniform PPT)**.** A *non-uniform* PPT adversary is a family $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$ of probabilistic circuits (or, equivalently, Turing machines each supplied with an *advice string* z_λ of length $\mathrm{poly}(\lambda)$) such that a single polynomial in λ bounds the size of $\mathcal{A}_\lambda$. The advice may depend arbitrarily on λ but not on the particular inputs drawn at run time.

Remark 1.5. The distinction between uniform adversaries (a single machine that reads 1^λ and works for all λ) and non-uniform ones (a separate circuit per λ, or a uniform machine with per-λ advice) is a genuine one. Non-uniformity grants the adversary free precomputed advice tailored to each parameter size—for example, a short hint about the group of the day. Security against non-uniform adversaries is the stronger and now-standard requirement, partly because non-uniform models compose more cleanly under reduction (one may "hard-wire" a fixed good choice of randomness or auxiliary input). All definitions below can be read in either flavour; we state hardness assumptions against non-uniform adversaries when it matters and otherwise leave the choice implicit.

Remark 1.6 (Expected versus strict polynomial time, and quantum adversaries). Two refinements recur. First, some definitions allow *expected* polynomial-time adversaries, bounding the average rather than worst-case running time; this matters for certain extraction arguments in zero-knowledge, and we flag it where it arises. Second, for post-quantum security (taken up in §2.8) one replaces PPT by *quantum* polynomial-time (QPT), a uniform family of polynomial-size quantum circuits. The structural definitions of this section (advantage, indistinguishability, reduction) are agnostic to which class of adversaries one quantifies over; only the underlying hardness assumptions change.

1.3 Distribution ensembles and indistinguishability

The most basic security goal is that two situations *look the same* to every efficient observer—a real protocol transcript versus a simulated one, an encryption of m_0 versus an encryption of m_1, a pseudorandom string versus a truly random one. To formalise "look the same" we first formalise the two situations as *ensembles* of probability distributions, after which we define three grades of sameness.

Definition 1.7 (Probability ensemble)**.** A *probability ensemble* is a sequence $X = \{X_\lambda\}_{\lambda\in\mathbb{N}}$ of probability distributions (equivalently, random variables), one for each value of the security parameter, where X_λ is supported on binary strings whose length is bounded by a polynomial in λ. There is often an additional index—an input a—written $\{X_\lambda(a)\}$; for clarity we suppress it unless needed.

1.3.1 The statistical distance and its variational characterisation

The natural metric on a single pair of distributions is the statistical (total-variation) distance.

Definition 1.8 (Statistical distance)**.** Let X, Y be discrete random variables over a common finite or countable set S. Their *statistical distance* (or total variation distance) is

$$\Delta(X,Y) \;:=\; \frac{1}{2}\sum_{s\in S}\bigl|\Pr[X=s]-\Pr[Y=s]\bigr|.$$

Proposition 1.9 (Variational characterisation)**.** *For X, Y over S,*

$$\Delta(X,Y) \;=\; \max_{T\subseteq S}\bigl(\Pr[X\in T]-\Pr[Y\in T]\bigr) \;=\; \max_{D}\bigl(\Pr[D(X)=1]-\Pr[D(Y)=1]\bigr),$$

where the last maximum ranges over all (even computationally unbounded, deterministic) decision functions $D : S \to \{0,1\}$. In particular, no procedure whatsoever—efficient or not—can distinguish X from Y with advantage exceeding $\Delta(X,Y)$.

Proof. The set form is the variational characterisation in the statistical-distance theorem of the *Math Guide* (§"Statistical distance"), whose argument is unchanged over a countable S. Only the reformulation over decision functions is new: a deterministic D is the indicator of the set $T = D^{-1}(1)$, so the maximum over D equals the maximum over T; a randomised D is a convex combination of deterministic ones and cannot do better. □

Proposition 1.10 (Properties of statistical distance)**.** *The statistical distance is a metric on probability distributions: $0 \le \Delta(X,Y) \le 1$, it is symmetric, $\Delta(X,Y) = 0$ iff X and Y are identically distributed, and it satisfies the triangle inequality $\Delta(X,Z) \le \Delta(X,Y)+\Delta(Y,Z)$. Moreover it never increases under (possibly randomised) post-processing: for any function—indeed any randomised algorithm—f,*

$$\Delta\bigl(f(X),f(Y)\bigr) \;\le\; \Delta(X,Y).$$

Proof. These are the metric and data-processing parts of the statistical-distance theorem of the *Math Guide* (§"Statistical distance"), again with the argument unchanged over a countable S. □

1.3.2 The three grades of indistinguishability

We now lift the comparison of single distributions to ensembles, in three strengths, from strongest to weakest. The weakest grade requires the asymptotic notion of a vanishing quantity, which we fix

once and for all.

Definition 1.11 (Negligible function)**.** A function $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible*, written $f = \mathrm{negl}(\lambda)$, if for every $c \in \mathbb{N}$ there exists λ_0 such that $f(\lambda) < \lambda^{-c}$ for all $\lambda \geq \lambda_0$; equivalently, f decays faster than the reciprocal of every polynomial. A function is *non-negligible* if it is not negligible; it is *noticeable* if there exists c with $f(\lambda) \geq \lambda^{-c}$ for all sufficiently large λ (noticeable functions are non-negligible, but not conversely). A function is *overwhelming* if $1 - f$ is negligible. The class is closed under addition and under multiplication by any polynomial—the closure property on which the hybrid argument below depends.

Definition 1.12 (Perfect, statistical, and computational indistinguishability)**.** Let $X = \{X_\lambda\}$ and $Y = \{Y_\lambda\}$ be probability ensembles.

- X and Y are *perfectly indistinguishable*, written $X \equiv Y$, if X_λ and Y_λ are *identically distributed* for every λ; equivalently $\Delta(X_\lambda, Y_\lambda) = 0$ for all λ.
- X and Y are *statistically indistinguishable*, written $X \approx_s Y$, if their statistical distance is negligible:
$$\Delta(X_\lambda, Y_\lambda) \;=\; \mathrm{negl}(\lambda).$$
- X and Y are *computationally indistinguishable*, written $X \approx_c Y$, if for every PPT algorithm D (the *distinguisher*), the function
$$\mathrm{Adv}^{\mathrm{dist}}_{X,Y}(D,\lambda) \;:=\; \Big|\Pr\big[D(1^\lambda, X_\lambda) = 1\big] - \Pr\big[D(1^\lambda, Y_\lambda) = 1\big]\Big|$$
is negligible in λ. The probabilities are over the draw from the ensemble and over D's internal randomness.

These three notions sit in a strict hierarchy. The implications are immediate from the definitions and the variational characterisation; standard examples witness the strictness.

Proposition 1.13 (Hierarchy)**.** *For all ensembles X, Y:*

$$X \equiv Y \implies X \approx_s Y \implies X \approx_c Y.$$

Both implications are strict: there exist ensembles that are statistically but not perfectly indistinguishable, and ensembles that are computationally but not statistically indistinguishable.

Proof. If $X \equiv Y$ then $\Delta(X_\lambda, Y_\lambda) = 0$, which is negligible, so $X \approx_s Y$. If $X \approx_s Y$, then for any—in particular any PPT—distinguisher D, Proposition 1.9 gives $\mathrm{Adv}^{\mathrm{dist}}_{X,Y}(D,\lambda) \leq \Delta(X_\lambda, Y_\lambda) = \mathrm{negl}(\lambda)$, so $X \approx_c Y$.

For strictness of the first implication, let X_λ be uniform on $\{0,1\}^\lambda$ and let Y_λ be uniform on $\{0,1\}^\lambda \setminus \{0^\lambda\}$. They are not identically distributed (the point 0^λ has probabilities $2^{-\lambda}$ and 0), so not perfectly indistinguishable, yet $\Delta(X_\lambda, Y_\lambda) = 2^{-\lambda} = \mathrm{negl}(\lambda)$, so they are statistically indistinguishable.

For strictness of the second, the existence of a pseudorandom generator (whose existence follows from that of one-way functions — the Håstad–Impagliazzo–Levin–Luby theorem, which we use without proof) furnishes an ensemble: let $G : \{0,1\}^\lambda \to \{0,1\}^{2\lambda}$ be a PRG, let $X_\lambda := G(U_\lambda)$ for U_λ uniform on $\{0,1\}^\lambda$, and let $Y_\lambda := U_{2\lambda}$ be uniform on $\{0,1\}^{2\lambda}$. By definition of a PRG, $X \approx_c Y$. But X_λ is supported on at most 2^λ strings out of $2^{2\lambda}$, so a set T equal to the image of G has $\Pr[X_\lambda \in T] = 1$ while $\Pr[Y_\lambda \in T] \leq 2^\lambda / 2^{2\lambda} = 2^{-\lambda}$; hence $\Delta(X_\lambda, Y_\lambda) \geq 1 - 2^{-\lambda}$, which is not negligible. Thus $X \approx_c Y$ but $X \not\approx_s Y$. □

Remark 1.14 (The price and the prize of computational indistinguishability). The strictness example is instructive. Statistically the PRG output and a random string are almost as far apart as two distributions can be—distance nearly 1—because the PRG image is a vanishing fraction of the codomain. An unbounded distinguisher simply checks membership in the image and wins. The entire content of computational indistinguishability is that *finding* that image membership test is infeasible. This is the bargain at the heart of cryptography: we trade the impossible (information-theoretic security with short keys) for the merely conjectured-hard (computational security), and in exchange we obtain the ability to encrypt long messages under short keys, build public-key primitives, and so on. Statistical and perfect security, when attainable, are unconditional and need no hardness assumption; computational security buys vastly more functionality at the cost of resting on an assumption.

A property that makes $\approx_c$ usable in proofs is that efficient operations preserve it: an adversary cannot manufacture a distinguishing edge out of indistinguishable inputs by post-processing.

Lemma 1.15 (Closure under efficient post-processing). *If $X \approx_c Y$ and M is any PPT algorithm, then the ensembles $\{M(1^\lambda, X_\lambda)\}$ and $\{M(1^\lambda, Y_\lambda)\}$ satisfy $\{M(X_\lambda)\} \approx_c \{M(Y_\lambda)\}$.*

Proof. Suppose some PPT distinguisher D separates $\{M(X_\lambda)\}$ from $\{M(Y_\lambda)\}$ with advantage $\delta(\lambda)$. Define $D' := D \circ M$, i.e. $D'(1^\lambda, z) := D(1^\lambda, M(1^\lambda, z))$. Since M and D are both PPT, so is D'. Then

$$\mathrm{Adv}^{\mathrm{dist}}_{X,Y}(D', \lambda) = \left|\Pr[D(M(X_\lambda)) = 1] - \Pr[D(M(Y_\lambda)) = 1]\right| = \delta(\lambda).$$

By $X \approx_c Y$ the left side is negligible, so δ is negligible. □

1.3.3 The hybrid argument

Many proofs require the comparison of two ensembles that are far apart but connected by a chain of pairwise-close intermediate ensembles. The hybrid argument is the workhorse that turns such a chain into a single conclusion. It is essentially the triangle inequality, but with a refinement that explains why the number of hybrids must be polynomial.

Lemma 1.16 (Hybrid lemma). *Let $t = t(\lambda)$ be polynomially bounded, and let $H^0_\lambda, H^1_\lambda, \ldots, H^t_\lambda$ be a sequence of distributions (the* hybrids*). Suppose that for every PPT distinguisher D the neighbouring advantages*

$$\alpha_i(D, \lambda) := \left|\Pr[D(H^i_\lambda) = 1] - \Pr[D(H^{i+1}_\lambda) = 1]\right|$$

are uniformly *negligible: a single negligible function ν (depending on D) satisfies $\alpha_i(D, \lambda) \le \nu(\lambda)$ for all $i \in \{0, \ldots, t(\lambda) - 1\}$ simultaneously. Equivalently, $\max_{0 \le i < t(\lambda)} \alpha_i(D, \lambda) = \mathrm{negl}(\lambda)$, i.e. $\alpha_{i(\lambda)}(D, \lambda)$ is negligible along* every *index function $i(\lambda)$—arbitrary, not merely efficiently computable, since the maximising index need not be efficient. Then $\{H^0_\lambda\}$ and $\{H^t_\lambda\}$ are computationally indistinguishable, i.e. for every PPT D, $\left|\Pr[D(H^0_\lambda) = 1] - \Pr[D(H^t_\lambda) = 1]\right| = \mathrm{negl}(\lambda)$.*

Proof. Fix a PPT distinguisher D and let ν be the uniform bound the hypothesis supplies for D. By the triangle inequality applied to the telescoping sum,

$$\left|\Pr[D(H^0_\lambda) = 1] - \Pr[D(H^t_\lambda) = 1]\right| = \left|\sum_{i=0}^{t-1} \left(\Pr[D(H^i_\lambda) = 1] - \Pr[D(H^{i+1}_\lambda) = 1]\right)\right| \le \sum_{i=0}^{t-1} \alpha_i(D, \lambda),$$

and the hypothesis bounds the sum by $t(\lambda) \cdot \nu(\lambda)$, which is negligible since t is a polynomial. Hence $\{H^0\} \approx_c \{H^t\}$. □

Uniformity in i is essential, and no argument derives it from the weaker hypothesis that each α_i is negligible for every *fixed* i: there the threshold beyond which the i-th term is small can depend on i,

and the dependence cannot be removed. A countermodel makes this concrete: take $t(\lambda) = \lambda + 1$ and let H^i_λ be the point mass at 0 if $i \le \lambda$ and the point mass at 1 otherwise. For every fixed i the advantage $\alpha_i(D, \lambda)$ is nonzero only at $\lambda = i$, hence negligible; yet H^0_λ and $H^{t(\lambda)}_\lambda$ are the point masses at 0 and 1, perfectly distinguishable. Applications discharge the uniform hypothesis by reduction. When the hybrids are efficiently samplable given $(1^\lambda, i)$, the *random-index* distinguisher D' samples i uniformly from $\{0, \dots, t-1\}$, embeds its challenge at position i, and runs D; writing $p_i := \Pr[D(H^i_\lambda) = 1]$, its distinguishing advantage equals $\left|\frac{1}{t}\sum_{i=0}^{t-1}(p_i - p_{i+1})\right| = |p_0 - p_t|/t$ (which is at most $\frac{1}{t}\sum_i \alpha_i(D, \lambda)$), so the per-step assumption applied to the single machine D' bounds D's total advantage by $t(\lambda) \cdot \mathrm{Adv}(D') = \mathrm{negl}(\lambda)$.

Remark 1.17. The hybrid argument is precisely where the "polynomially many" constraint earns its keep. A negligible per-step advantage multiplied by a polynomial number of steps remains negligible; a negligible per-step advantage multiplied by, say, 2^λ steps need not. This is why constructions that would require exponentially many hybrids do not yield security proofs, and why the closure of negligibility under polynomial factors is not a technicality but the load-bearing wall of the whole edifice.

1.4 Security as a game

We can now state what it means to "break" a scheme. The modern idiom expresses every security definition as a *game* (or experiment) played between a *challenger* and an *adversary* $\mathcal{A}$. The challenger, following a fixed protocol, sets up the scheme, answers whatever queries the adversary may pose, and at the end declares the adversary a winner or loser by outputting a single bit. Security says that no PPT adversary wins by more than it could by trivial guessing.

Definition 1.18 (Security game and advantage)**.** A *security game* (or *experiment*) G is a PPT interactive algorithm, the challenger, that takes 1^λ, interacts with an adversary $\mathcal{A}$, and finally outputs a bit $\mathsf{G}^{\mathcal{A}}(\lambda) \in \{0, 1\}$, where 1 denotes "adversary wins." The *advantage* of $\mathcal{A}$ is a measure of how much better than trivial its winning probability is; its precise form depends on the type of game:

- *Distinguishing (indistinguishability) games*, where the challenger secretly chooses a uniform bit $b \in \{0, 1\}$ and $\mathcal{A}$ outputs a guess b'; the adversary wins iff $b' = b$. Here

$$\mathrm{Adv}_{\mathsf{G}}(\mathcal{A}, \lambda) := \left|\Pr\left[\mathsf{G}^{\mathcal{A}}(\lambda) = 1\right] - \tfrac{1}{2}\right| = \tfrac{1}{2}\left|\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]\right|.$$

  Trivial guessing achieves winning probability $1/2$, hence advantage 0.

- *Search (unforgeability/inversion) games*, where the adversary must produce a specific kind of object—a forgery, a preimage, a discrete logarithm—and wins iff it succeeds. Here the baseline winning probability is (essentially) 0, and one simply sets

$$\mathrm{Adv}_{\mathsf{G}}(\mathcal{A}, \lambda) := \Pr\left[\mathsf{G}^{\mathcal{A}}(\lambda) = 1\right].$$

Definition 1.19 (Security relative to a game)**.** A scheme is *secure* with respect to the game G if for every PPT adversary $\mathcal{A}$,

$$\mathrm{Adv}_{\mathsf{G}}(\mathcal{A}, \lambda) = \mathrm{negl}(\lambda).$$

Two remarks on the shape of this definition are in order. First, the quantifier order is essential: *for every* PPT adversary, the advantage is negligible. No bound constrains the advantage of one fixed adversary; the claim is universal over the entire class. Second, the use of negl on the right is what makes the definition asymptotic and clean: it is robust to the polynomial slop that reductions introduce, and it captures "the adversary does no better than the trivial strategy, up to an amount that vanishes faster than any inverse polynomial."

Example 1.20 (Indistinguishability of encryptions, as a game)**.** To anchor the abstraction, consider the canonical encryption-security game, *indistinguishability under chosen-plaintext attack* (IND-CPA). For a public-key encryption scheme with algorithms (Gen, Enc, Dec):

1. The challenger runs $(pk, sk) \leftarrow \mathrm{Gen}(1^\lambda)$ and sends pk to $\mathcal{A}$.
2. $\mathcal{A}$ chooses two equal-length messages m_0, m_1 and sends them to the challenger.
3. The challenger samples $b \leftarrow \{0,1\}$ uniformly and returns the *challenge ciphertext* $c^\star \leftarrow \mathrm{Enc}(pk, m_b)$.
4. $\mathcal{A}$ outputs a guess b'; the game outputs 1 iff $b' = b$.

The scheme is IND-CPA secure iff $\mathrm{Adv}(\mathcal{A}, \lambda) = \left|\Pr[b' = b] - \frac{1}{2}\right|$ is negligible for every PPT $\mathcal{A}$. Unwinding the definitions, IND-CPA security is exactly the statement that the two ensembles "encryption of m_0" and "encryption of m_1" (for adversarially chosen m_0, m_1) are computationally indistinguishable—the game packaging and the ensemble packaging of Definition 1.12 describe the same thing.

Remark 1.21 (Game-hopping)*.* Once we phrase security as a game, proofs become *sequences of games*. One starts from the real security game and rewrites it through a series of games $\mathsf{G}_0, \mathsf{G}_1, \ldots, \mathsf{G}_k$, each differing from the last by a small, justifiable change, until reaching a final game in which the adversary provably has zero (or trivially negligible) advantage. Each hop is justified either because the two games are identical unless some negligible- probability "bad" event occurs, or because distinguishing the two games would break a hardness assumption (a reduction, below), or because the change is purely syntactic. Bounding the total advantage is then a telescoping sum across hops—a structured cousin of the hybrid argument (Lemma 1.16). This discipline keeps otherwise sprawling proofs honest and auditable.

1.5 Reductions

The engine of provable security is the reduction. A *reduction* is an efficient transformation that converts a successful adversary against a scheme into a successful algorithm against a problem believed to be hard. Its existence proves the implication "Y is hard $\Rightarrow$ X is secure" by establishing the contrapositive "X is broken $\Rightarrow$ Y is solved."

Definition 1.22 (Computational problem and hardness)**.** A *computational problem* Π consists of a PPT instance generator $\mathrm{Inst}(1^\lambda)$ producing a problem instance together with any side information, and a relation deciding when a candidate output *solves* the instance. The *success probability* of an algorithm $\mathcal{B}$ is

$$\mathrm{Succ}_\Pi(\mathcal{B}, \lambda) := \Pr\big[\mathcal{B}(1^\lambda, \mathrm{Inst}(1^\lambda)) \text{ solves the instance}\big].$$

The problem Π is *hard* if $\mathrm{Succ}_\Pi(\mathcal{B}, \lambda) = \mathrm{negl}(\lambda)$ for every PPT $\mathcal{B}$ (for a search problem), or if every PPT $\mathcal{B}$ decides the instance with at most negligible advantage, $\mathrm{Adv}(\mathcal{B}, \lambda) = \mathrm{negl}(\lambda)$ in the sense of Definition 1.18 (for a decision problem).

Definition 1.23 (Black-box reduction)**.** Let X be a scheme with security game G_X and Π a computational problem. A *(black-box) reduction* from breaking X to solving Π is a PPT oracle algorithm $\mathcal{R}$ with the following property. There exist functions $\varepsilon_\Pi(\lambda)$ and $\varepsilon_X(\lambda)$ and a polynomial overhead such that for *every* adversary $\mathcal{A}$ achieving advantage $\varepsilon_X(\lambda) := \mathrm{Adv}_{\mathsf{G}_X}(\mathcal{A}, \lambda)$, the algorithm $\mathcal{R}^{\mathcal{A}}$—which runs $\mathcal{A}$ as a subroutine, simulating $\mathcal{A}$'s game internally—solves Π with success probability

$$\mathrm{Succ}_\Pi\big(\mathcal{R}^{\mathcal{A}}, \lambda\big) \geq \varepsilon_\Pi(\lambda),$$

where the reduction's *loss*, discussed below, relates ε_Π to ε_X. The reduction is *black-box* because $\mathcal{R}$ uses $\mathcal{A}$ only through its input/output interface, making no use of $\mathcal{A}$'s code.

The structure of a reduction is always the same and is worth stating as a recipe. To prove "if Π is hard then X is secure":

1. Assume toward the contrapositive a PPT adversary $\mathcal{A}$ that breaks X with non-negligible advantage ε_X.
2. Construct a PPT algorithm $\mathcal{R}$ that, given a random instance of Π, *embeds* that instance into a simulated security game for $\mathcal{A}$. The simulation must be faithful: the view $\mathcal{R}$ presents to $\mathcal{A}$ must follow (perfectly, statistically, or computationally) the distribution of the real game, so that $\mathcal{A}$'s advantage carries over.
3. Run $\mathcal{A}$. From whatever $\mathcal{A}$ does to win its game, $\mathcal{R}$ *extracts* a solution to the Π-instance.
4. Conclude that $\mathcal{R}$ solves Π with non-negligible probability, contradicting the hardness of Π. Therefore no such $\mathcal{A}$ exists, and X is secure.

A complete, genuine reduction illustrates the point.

Theorem 1.24 (Hard-core security of the Diffie–Hellman-style key, schematically)**.** *Let* $\Pi = \mathrm{CDH}$ *be the computational Diffie–Hellman problem in a cyclic group* $\mathbb{G} = \langle g \rangle$ *of prime order* q*: given* (g, g^a, g^b) *for uniform* $a, b \in \mathbb{Z}/q\mathbb{Z}$*, compute* g^{ab}*. Consider the "key-recovery" game* G_X *for a key-exchange scheme in which the adversary, given the public transcript* (g^a, g^b)*, wins iff it outputs the shared key* g^{ab}*. If* CDH *is hard, then no PPT adversary wins* G_X *with noticeable probability.*

Proof. Suppose $\mathcal{A}$ is a PPT adversary that, on input a transcript (g^a, g^b), outputs g^{ab} with probability $\varepsilon_X(\lambda) = \mathrm{Adv}_{\mathsf{G}_X}(\mathcal{A}, \lambda)$. Construct $\mathcal{R}$ solving CDH. On input a CDH instance $(g, A, B) = (g, g^a, g^b)$:

1. $\mathcal{R}$ forms the simulated transcript (A, B) and feeds it to $\mathcal{A}$ as the public transcript of G_X.
2. Because (a, b) are uniform in the CDH instance *and* in the real game, the view of $\mathcal{A}$ follows *identically* the distribution of its view in G_X (the simulation is perfect; no loss in faithfulness).
3. When $\mathcal{A}$ outputs a candidate key K, $\mathcal{R}$ outputs K as its CDH answer.

Since the simulated and real views are identically distributed, $\mathcal{A}$ outputs $K = g^{ab}$ with exactly probability $\varepsilon_X(\lambda)$; whenever it does, $\mathcal{R}$ returns the correct CDH solution. Hence

$$\mathrm{Succ}_{\mathrm{CDH}}(\mathcal{R}, \lambda) \;=\; \varepsilon_X(\lambda).$$

The running time of $\mathcal{R}$ is that of $\mathcal{A}$ plus a constant amount of group bookkeeping, hence polynomial. If ε_X were noticeable, $\mathcal{R}$ would solve CDH with noticeable probability, contradicting its hardness. Therefore $\varepsilon_X(\lambda)$ is not noticeable, as claimed. In fact, since $\mathcal{R}$ is PPT and CDH is hard (Definition 1.22, search branch), $\varepsilon_X(\lambda) = \mathrm{Succ}_{\mathrm{CDH}}(\mathcal{R}, \lambda) = \mathrm{negl}(\lambda)$. □

This example is deliberately *tight*: the reduction loses nothing, calls the adversary once, and converts advantage ε_X into success probability exactly ε_X. Most reductions are not so fortunate, which leads to the quantitative anatomy of a reduction.

1.5.1 Tightness and security loss

A reduction is not merely a yes/no certificate; it carries quantitative information about *how much* security one scheme inherits from another problem. Two parameters matter: the running-time overhead and the advantage relationship.

Definition 1.25 (Reduction loss and tightness)**.** Consider a reduction $\mathcal{R}$ that transforms an adversary running in time t with advantage ε into a Π-solver running in time $t' = t + t_{\mathcal{R}}$ with success probability ε'. The *security loss* is the factor

$$L(\lambda) \;:=\; \frac{\varepsilon/t}{\varepsilon'/t'} \;=\; \frac{\varepsilon}{\varepsilon'} \cdot \frac{t'}{t},$$

the ratio of the solver's work-per-success to the adversary's work-per-success. A reduction is *tight* if $L(\lambda) = O(1)$ (a small constant, ideally 1), and *loose* if $L(\lambda)$ grows with λ (e.g. $L = q_H$, the number of random-oracle queries, or $L = q_s$, the number of signature queries, or $L = \lambda$). Two common sources of loss are: an *advantage gap* $\varepsilon' \ll \varepsilon$ (the solver succeeds far less often than the adversary, e.g. because the reduction must *guess* where to embed its instance among q queries, succeeding only with probability $1/q$); and a *time blow-up* $t' \gg t$ (the reduction runs the adversary many times, as in rewinding/forking arguments).

Remark 1.26 (Why loss matters concretely). Asymptotically, loss is invisible: a polynomial loss factor turns a negligible advantage into a negligible advantage and a hard problem into security, without qualification. Concretely, however, loss carries a cost. Suppose a scheme reduces to a problem believed to offer 128 bits of security, but the reduction loses a factor $L = 2^{40}$ (not unusual: $q_H \approx 2^{60}$ random-oracle queries times a forking-lemma square-root loss can be much worse). Then the *proof* guarantees only $128 - 40 = 88$ bits of security for the scheme. To restore a 128-bit guarantee one must enlarge the group so that the *underlying* problem offers 168 bits—larger keys, slower operations. A tight reduction lets the scheme inherit the full strength of the assumption at the smallest parameters; a loose one imposes a permanent cost on every user. This is why tightness is a first-class design goal in modern cryptography, not a theoretical nicety.

Remark 1.27 (Rewinding and the forking lemma, in brief). A recurrent source of large loss is *rewinding*: running the adversary, recording its behaviour, then restarting it from a saved state with fresh randomness on some branch to obtain a second, correlated transcript. From two transcripts that share a prefix but diverge afterward one can often extract a secret (e.g. two valid signatures on the same commitment yield the signing key, or two accepting proof transcripts yield a witness). The *forking lemma* quantifies the success probability of obtaining such a pair: if the adversary succeeds with probability ε using one of q possible random-oracle responses, the forked run succeeds with probability roughly $\varepsilon(\varepsilon/q - 1/|\text{range}|)$, i.e. on the order of ε^2/q. The square in ε is a quadratic security loss: to keep ε^2/q noticeable one needs ε noticeably large, roughly halving the bit-security extracted. Such arguments—central to the analysis of Schnorr-type and Fiat–Shamir signatures that recur in the proof systems this monograph builds toward—are the archetype of loose but still valid reductions.

1.6 The standard model and the random oracle model

Reductions reduce security to assumptions. *Which* assumptions one is willing to make defines the *model* in which a proof resides. Two models predominate.

Definition 1.28 (Standard model)**.** A proof resides in the *standard model* if it assumes only the computational hardness of explicitly stated problems (factoring, discrete logarithm, decisional Diffie–Hellman, learning-with-errors, and so on) and grants the adversary no idealised access to any component of the scheme. Every primitive, including every hash function, is a concrete algorithm whose code the adversary may inspect and exploit.

Definition 1.29 (Random oracle model)**.** A proof resides in the *random oracle model* (ROM) if it idealises one or more hash functions as a *random oracle*: a publicly accessible function $\mathcal{O}$ that, on each distinct input, returns an *independent, uniformly random* output of the appropriate length, consistently answering repeated queries with the same value. The oracle is available to the honest parties, to the adversary, and—crucially—to the reduction. No party holds the "code" of $\mathcal{O}$; it can only be queried.

The random oracle model is an idealisation: real hash functions are fixed, public algorithms, not truly random functions. The motivation for adopting it is that the reduction gains two capabilities that the standard model withholds and that render many otherwise-intractable proofs tractable.

Remark 1.30 (The powers of the random oracle: observability and programmability)*.* In the ROM the reduction *simulates* the oracle for the adversary, and this yields two additional capabilities. *Observability*: the reduction sees every query the adversary makes to $\mathcal{O}$. Since the adversary, to use a hash value meaningfully, must query it, the reduction learns the adversary's intermediate computations—for instance the preimage the adversary is about to hash. *Programmability*: the reduction may choose each oracle answer on the fly, subject only to the answers appearing uniform and consistent. It can therefore *plant* its hard-problem instance inside an oracle response—e.g. set $\mathcal{O}(x) := B$ where B is part of a CDH challenge—so that the adversary's success necessarily reveals the solution. Neither capability exists for a concrete standard-model hash function, which fixes its outputs in advance and hides its internal queries.

Example 1.31 (The benefit of the ROM: full-domain-hash, sketched)**.** In the "full domain hash" signature, the signer signs a message m as $\sigma = H(m)^d \bmod N$ where (N, d) is an RSA private key—a modulus $N = pq$ with secret prime factors and a secret exponent d paired with a public e so that $x^{ed} \equiv x \pmod{N}$ for x coprime to N, by Euler's theorem (*Math Guide*, §"Fermat's little theorem and Euler's theorem"); thus $x \mapsto x^e$ is a permutation that only the holder of d can invert—and H maps into $\mathbb{Z}/N\mathbb{Z}$. To prove unforgeability one reduces to RSA inversion: given a challenge y (find y^d), the reduction programs the oracle so that for a random subset of messages $H(m) := y \cdot r_m^e$ (embedding the challenge) and for the rest $H(m) := r_m^e$ (where it knows the "signature" r_m). When the forger outputs a forgery on a message where the reduction embedded the challenge, the reduction extracts y^d. This proof has *no known standard-model analogue* for the plain scheme: it relies essentially on programming H. The cost is a security loss (the reduction must guess which message the forger attacks, incurring a factor q_H unless cleverly optimised)—connecting back to Definition 1.25.

Remark 1.32 (The status of the ROM: heuristic, not theorem)*.* A proof in the ROM is a proof about an *idealised* world: deploying the scheme instantiates $\mathcal{O}$ with a concrete hash function, and the proof does *not* formally transfer—a ROM proof rules out all attacks that treat the hash as a black box, while offering no guarantee against attacks on the specific structure of the chosen hash. Section 3.5 treats the uninstantiability results that make this gap a theorem, and the working stance they justify.

Remark 1.33 (Relatives: CRS, generic-group, and algebraic-group models)*.* Between and beyond these poles lie further idealisations relevant to the proof systems this volume targets. The *common reference string* (CRS) model posits a trusted public string sampled from a prescribed distribution, used by many succinct proof systems. The *generic group model* (GGM) idealises the group dual to the manner in which the ROM idealises a hash: the adversary may only perform group operations through an oracle on opaque "handles," never exploiting the bit representation of group elements; it is the natural setting for lower bounds on discrete-log-type problems. The *algebraic group model* (AGM) is an intermediate, milder idealisation requiring the adversary to "explain" every group element it outputs as a known combination of its inputs. These models trade realism for provability along the same axis as the ROM, and the constructions ahead invoke them by name where they require them.

1.7 Concrete versus asymptotic security and λ-bit security

The asymptotic language—"negligible," "polynomial," "for all sufficiently large λ"—is mathematically clean but operationally silent. It establishes that a scheme is eventually secure; it does not establish whether *this* deployment with *these* parameters resists an adversary with *that* budget. Concrete (or exact) security supplies the missing numbers.

Definition 1.34 (Concrete security: (t, ε)-security)**.** Fix the security parameter to a concrete value (so all sizes are fixed). A scheme is (t, ε)-*secure* for a game G if every adversary running in time at most t has advantage $\mathrm{Adv}_{\mathsf{G}}(\mathcal{A}) \leq \varepsilon$. More refined statements bound the advantage as an explicit function of the adversary's resources, e.g. time t, number of queries q, and memory s: $\mathrm{Adv}_{\mathsf{G}}(\mathcal{A}) \leq f(t, q, s)$.

Definition 1.35 (λ-bit security)**.** A scheme (or problem) has *n bits of security*, or is *n-bit secure*, if every adversary running in time t achieves advantage at most about $t/2^n$; equivalently, the best known attack achieving constant (say $\geq 1/2$) success probability requires on the order of 2^n elementary operations. Thus n-bit security means that breaking the scheme costs about 2^n work. One commonly summarises a scheme by the single number n—"128-bit security"—meaning no feasible adversary exists, where the horizon of feasibility lies conventionally around 2^{128} operations.

The relationship to the asymptotic picture is the following: one chooses the security parameter λ *precisely so that* the resulting concrete scheme delivers the desired number of bits of security. In the cleanest case the two coincide, $n = \lambda$, but in general one must account for the actual cost of the best attack on the underlying problem.

Example 1.36 (Why $\lambda \neq n$ for discrete logarithm)**.** For a generic-group or hash-based primitive, doubling the output length doubles the bit security: a hash with $2n$-bit output offers n-bit collision resistance (birthday bound) and $2n$-bit preimage resistance, so the parameter and the security level track simply. For discrete-logarithm-based primitives the accounting differs and instructs. In a group of prime order $q \approx 2^\ell$, generic attacks (baby-step/giant-step, Pollard's rho) solve discrete logarithm in time $\approx \sqrt{q} = 2^{\ell/2}$. Hence a 256-bit elliptic-curve group offers only about 128 bits of security—one *halves* the field size to obtain the security level. Moreover, for discrete logarithm in the multiplicative group of a finite field, sub-exponential index-calculus attacks force ℓ into the thousands of bits for 128-bit security. This is precisely why elliptic-curve groups (no known sub-exponential attack) are preferred: they permit $\ell \approx 2n$ rather than $\ell \approx$ thousands. The single number n conceals a problem-specific translation from parameter size to security level.

Remark 1.37 (Reading a bit-security budget end to end)*.* Combining the preceding components yields the practitioner's parameter-selection recipe. Decide the target security level n (commonly 128). Identify the underlying hard problem and the cost of its best attack as a function of the parameter (e.g. $2^{\ell/2}$ for elliptic-curve discrete log), and set the parameter so the problem itself offers n bits (here $\ell = 2n = 256$). Then subtract the reduction's security loss (Definition 1.25): if the proof loses $L = 2^k$ bits, the problem must offer $n + k$ bits for the *scheme* to offer n, so one enlarges the parameter accordingly—or, with a tight reduction, one need not. Finally, sanity-check against the ROM caveat (Remark 1.32): a ROM proof bounds only black-box attacks on the hash, so the concrete hash must additionally exhibit no exploitable structure. The clean asymptotic theorem "the scheme is secure if the problem is hard" thus unfolds, through the quantitative machinery of this section, into a concrete choice of bytes.

Remark 1.38 (A note on what security does and does not promise)*.* A caution that frames everything above is in order. A proof of security is a proof *relative to a model*: a definition of the goal (the game), a

class of adversaries (PPT, possibly non-uniform or quantum), and a set of assumptions (the hard problems, and any idealisation such as the ROM). It guarantees nothing outside that model. Side-channel leakage (timing, power), faulty randomness, implementation bugs, broken assumptions, and goals the game did not capture all lie beyond its reach. The value of the framework is not that it renders schemes unconditionally safe—nothing can—but that it renders the *conditions* of safety explicit, falsifiable, and modular, so that progress on assumptions and attacks translates directly into recalibrated, auditable guarantees. Everything this monograph builds rests on this discipline.

2 Hardness assumptions

Every public-key primitive in this monograph ultimately rests on the belief that some computational problem is intractable: that no realistic adversary, given a randomly drawn instance, can solve it except with negligible probability. This section assembles the family of hardness assumptions on which the discrete-logarithm world rests. It begins with the discrete logarithm problem itself, ascends the ladder of Diffie–Hellman variants, measures exactly how hard these problems can be against *generic* attacks, examines how composite group orders undermine that hardness, instantiates everything on the elliptic curves that the remainder of the book uses, and finally confronts the quantum caveat. Throughout, the lodestar is a single number: the *security parameter* λ, treated as the bit-length of security demanded. The recurring conclusion is that to obtain λ bits of security against the best generic attacks the group must have prime order q with $\log_2 q \geq 2\lambda$.

2.1 The security parameter and negligibility

Section 1 supplies the vocabulary we use throughout: a quantity is *negligible* when it decays faster than the reciprocal of every polynomial (Definition 1.11), adversaries are probabilistic polynomial-time (PPT) machines (Definition 1.3), and for *search* problems the figure of merit is the probability of outputting the exact answer, while for *decision* problems it is the distinguishing advantage of Definition 1.18. A problem is *hard* when every PPT algorithm solves a random instance with at most negligible probability (or distinguishes with at most negligible advantage); alongside this asymptotic register we use the concrete register of §1.7, in which the fixed Pasta curves of §2.7 offer "about 126 bits" of security against the attacks catalogued below.

2.2 The discrete logarithm problem

The setting is a finite cyclic group. Recall that a group G is *cyclic* if there is an element $g \in G$, called a *generator*, with $G = \langle g \rangle = \{ g^k : k \in \mathbb{Z} \}$; writing the group additively, as we shall do for elliptic curves, this reads $G = \{ kg : k \in \mathbb{Z} \}$. If $|G| = q$ then $\mathbb{Z}/q\mathbb{Z} \cong G$ via $k \mapsto g^k$, an isomorphism that is trivial to evaluate in the forward direction (by repeated squaring, in $O(\log k)$ group operations) but conjecturally hard to invert. That asymmetry is the discrete logarithm problem.

Definition 2.1 (Discrete logarithm)**.** Let $G = \langle g \rangle$ be a cyclic group of order q, written multiplicatively. For $h \in G$ the *discrete logarithm* of h to the base g, denoted $\log_g h$, is the unique residue $x \in \mathbb{Z}/q\mathbb{Z}$ with $g^x = h$. Existence and uniqueness modulo q follow from $G = \langle g \rangle$ and $\operatorname{ord}(g) = q$: if $g^x = g^{x'}$ then $g^{x-x'} = 1$, so $q \mid x - x'$.

Definition 2.2 (Discrete logarithm problem, DLP)**.** Fix a family $\{G_\lambda\}$ of cyclic groups, where $G_\lambda = \langle g_\lambda \rangle$ has prime order q_λ with $\log_2 q_\lambda = \Theta(\lambda)$, and a sampler that on input 1^λ outputs a description of $(G_\lambda, g_\lambda, q_\lambda)$. The *discrete logarithm problem* is: given such a description and $h = g_\lambda^x$ for x drawn uniformly from $\mathbb{Z}/q_\lambda\mathbb{Z}$, output x. The *DLP assumption* states that for every PPT algorithm $\mathcal{A}$,

$$\Pr_{x \leftarrow \mathbb{Z}/q_\lambda\mathbb{Z}} \left[\mathcal{A}(G_\lambda, g_\lambda, q_\lambda, g_\lambda^x) = x \right] \in \operatorname{negl}(\lambda).$$

We restrict the order q to be *prime* for reasons that will become emphatic in §2.6: in prime order every non-identity element is a generator, there are no proper nontrivial subgroups through which to leak information, and one cannot split the problem across factors. For now we record the convenience that random self-reducibility affords.

Proposition 2.3 (Random self-reducibility of DLP)**.** *In a cyclic group $G = \langle g \rangle$ of prime order q, discrete logarithm is* random self-reducible*: an algorithm that solves a fraction ε of instances (over uniformly random h) converts into one that solves* any *fixed instance with probability ε per trial. Consequently the hardness of the worst case equals the hardness of the average case.*

Proof. Let $\mathcal{A}$ succeed on a uniformly random target with probability ε. Given a fixed challenge $h = g^x$ whose logarithm we seek, draw r uniformly from $\mathbb{Z}/q\mathbb{Z}$ and form $h' = h \cdot g^r = g^{x+r}$. Because r is uniform and addition modulo q is a bijection, h' is uniformly distributed in G, independently of h; hence $\mathcal{A}(h')$ returns $x + r \bmod q$ with probability ε. Subtract r to recover x. Each trial uses fresh r, so the trials are independent and t of them succeed with probability $1 - (1-\varepsilon)^t$, which is overwhelming once $t = \omega(\varepsilon^{-1} \log \lambda)$. One checks correctness of each candidate by testing $g^x \stackrel{?}{=} h$. □

Random self-reducibility is the formal reason cryptographers may speak of "the" hardness of DLP in a group without concern for weak instances: there are none, every instance is as hard as a random one.

2.3 Diffie–Hellman: computational and decisional

DLP asks to invert exponentiation. Many protocols (the Diffie–Hellman key exchange and ElGamal encryption being the historical motivations) need only a weaker-looking guarantee about the *product of exponents*. Suppose Alice publishes g^a and Bob publishes g^b; their shared secret is g^{ab}, which each computes from the other's public value and their own exponent. An eavesdropper sees g, g^a, g^b and wants g^{ab}.

Definition 2.4 (Computational Diffie–Hellman, CDH)**.** With notation as in Definition 2.2, the *computational Diffie–Hellman problem* is: given (g, g^a, g^b) with a, b uniform in $\mathbb{Z}/q\mathbb{Z}$, output g^{ab}. The *CDH assumption* states that for every PPT $\mathcal{A}$,

$$\Pr_{a,b}\left[\, \mathcal{A}(g, g^a, g^b) = g^{ab} \,\right] \in \mathrm{negl}(\lambda).$$

CDH delivers the shared key but not its secrecy: an adversary may be unable to *produce* g^{ab} yet still able to distinguish it from a random group element, which would already break, say, the indistinguishability of an ElGamal ciphertext. The decisional variant excludes this.

Definition 2.5 (Decisional Diffie–Hellman, DDH)**.** The *decisional Diffie–Hellman problem* is to distinguish the two distributions over G^3

$$\mathcal{D}_{\mathrm{dh}} = \left(g^a, g^b, g^{ab}\right), \qquad \mathcal{D}_{\mathrm{rand}} = \left(g^a, g^b, g^c\right),$$

with a, b, c independent and uniform in $\mathbb{Z}/q\mathbb{Z}$. The *DDH assumption* states that every PPT distinguisher D has negligible advantage (Definition 1.18):

$$\mathrm{Adv}_{\mathrm{ddh}}(D) = \left| \Pr_{a,b}[D(g^a, g^b, g^{ab}) = 1] - \Pr_{a,b,c}[D(g^a, g^b, g^c) = 1] \right| \in \mathrm{negl}(\lambda).$$

A triple (g^a, g^b, g^c) with $c \equiv ab \pmod q$ is a *Diffie–Hellman triple*; DDH states that no distinguisher can recognise such triples.

Remark 2.6. DDH, like DLP, is random self-reducible: given a target triple $(u, v, w) = (g^a, g^b, g^c)$, choose r, s, t uniformly and form

$$(u', v', w') = \left(u^r g^t,\ v^s,\ w^{rs} v^{ts}\right) = \left(g^{(ar+t)},\ g^{bs},\ g^{(cr+bt)s}\right).$$

If $c = ab$ then $cr + bt = b(ar + t)$, so the new exponents satisfy the DH relation and (u', v', w') is a fresh uniform DH triple; if $c \neq ab$ the third exponent is uniform and independent. Thus the rerandomisation maps DH triples to DH triples and non-DH triples to uniform triples, so any noticeable advantage on random instances transfers to every instance.

2.4 The hierarchy DDH $\leq$ CDH $\leq$ DLP

The three problems are not independent; they form a chain. Write $A \leq B$ to mean "A reduces to B", i.e. an oracle solving B yields an efficient solver for A; equivalently, B is *at least as hard* as A, so if B is easy then A is easy; equivalently, the assumption "A is hard" is the *stronger* one (it implies "B is hard"). The reductions go

$$\text{DDH} \;\leq\; \text{CDH} \;\leq\; \text{DLP},$$

so the chain orders the problems by increasing hardness and the corresponding assumptions by decreasing strength: assuming DDH hard is the strongest commitment, assuming DLP hard the weakest.

Theorem 2.7 (DLP is at least as hard as CDH)**.** *If DLP is solvable in PPT, then CDH is solvable in PPT. Equivalently, the CDH assumption implies the DLP assumption.*

Proof. Suppose $\mathcal{A}$ solves DLP. Given a CDH instance (g, g^a, g^b), run $\mathcal{A}$ on (g, g^a) to recover $a = \log_g(g^a)$. Then compute $(g^b)^a = g^{ab}$ by fast exponentiation. The total cost is one DLP call and $O(\log q)$ group operations, all PPT, and the answer is exactly the CDH solution. (One need invert only one of the two exponents.) □

Theorem 2.8 (CDH is at least as hard as DDH)**.** *If CDH is solvable in PPT, then DDH is solvable in PPT (indeed with advantage overwhelmingly close to* 1*). Equivalently, the DDH assumption implies the CDH assumption.*

Proof. Suppose $\mathcal{A}$ solves CDH. Construct a distinguisher D for DDH as follows. On input a triple (g^a, g^b, z), call $\mathcal{A}(g, g^a, g^b)$ to obtain a candidate w for g^{ab}, and output 1 iff $w = z$. If the triple is a DH triple then $z = g^{ab}$, and whenever $\mathcal{A}$ succeeds (probability ε, noticeable by hypothesis) one has $w = z$ and D outputs 1. If the triple is random then $z = g^c$ with c uniform and independent of a, b; since w depends only on (g^a, g^b), the event $w = z$ has probability exactly $1/q$. Hence

$$\text{Adv}_{\text{ddh}}(D) \geq \varepsilon - \frac{1}{q},$$

which is noticeable. Amplifying ε to $1 - \text{negl}$ by the random self-reducibility of CDH (rerandomise (g^a, g^b) as in Remark 2.6 and take a majority/consistency vote) drives the advantage to $1 - \text{negl}$. □

The chain is strict in the sense that, in many natural groups, the harder assumptions fail while one believes the easier ones to hold. The principal example is the role of pairings.

Example 2.9 (DDH can be easy while CDH and DLP are hard)**.** Some elliptic-curve groups carry an efficiently computable, non-degenerate *bilinear pairing* $e : G \times G \to G_T$ with $e(g^a, g^b) = e(g, g)^{ab}$. On such a group DDH is *easy*: to test whether (g^a, g^b, z) is a DH triple, check

$$e(g^a, g^b) \stackrel{?}{=} e(g, z),$$

which holds iff $\log_g z \equiv ab$. Yet CDH (and a fortiori DLP) may remain hard, because the pairing reveals g^{ab} only inside the *target* group G_T, not as an element of G. Such groups, called *gap groups*, separate DDH from CDH and form the foundation of pairing-based cryptography. The implication is that DDH is a genuinely stronger assumption: choosing a group where DDH is believed hard (a "DDH group") rules out pairings of this convenient kind on G itself.

Remark 2.10. The converse reductions are not known in general. Whether CDH implies DLP (i.e. whether one can extract a from the ability to compute g^{ab}) is the longstanding *Diffie–Hellman vs. discrete log* question; partial results of den Boer and Maurer show equivalence when a suitable auxiliary group of smooth order is available, but no unconditional equivalence is known. Likewise DDH may be strictly easier than CDH, as Example 2.9 shows. In practice one assumes exactly the strength a protocol requires: signatures and key exchange often need only CDH, whereas ElGamal-style encryption needs DDH for its ciphertexts to hide the plaintext.

2.5 Generic algorithms and the square-root barrier

We now quantify the difficulty of DLP. In a group with no exploitable structure beyond the group law, the best known algorithms are *generic*: they manipulate group elements only through the operations multiply, invert, equality-test, never inspecting the bit-representation of an element. Shoup's idealisation makes the generic model precise: a random injective *labelling* $\sigma : \mathbb{Z}/q\mathbb{Z} \to S$ encodes the element g^k as the opaque string $\sigma(k)$, and the algorithm may only ask an oracle for $\sigma(i + j)$ given $\sigma(i), \sigma(j)$, and compare labels for equality. We exhibit first two generic algorithms achieving $O(\sqrt{q})$ operations, then give a matching lower bound.

Baby-step giant-step

Proposition 2.11 (Baby-step giant-step, BSGS)**.** *In a cyclic group of order q one can compute any discrete logarithm using $O(\sqrt{q})$ group operations and $O(\sqrt{q})$ stored group elements.*

Proof. Let $m = \lceil \sqrt{q} \rceil$. We can write any $x \in \{0, 1, \dots, q-1\}$ as

$$x = i \cdot m + j, \qquad 0 \le i < m, \quad 0 \le j < m,$$

since $m^2 \ge q$. The target relation $g^x = h$ becomes $g^{im+j} = h$, i.e.

$$g^j = h \cdot \left(g^{-m}\right)^i.$$

Baby steps: compute and store the table $\{(j, g^j) : 0 \le j < m\}$, indexed by the group element g^j in a hash map. This costs $m - 1$ multiplications. *Giant steps:* set $u = g^{-m}$ (one inversion and an exponentiation, $O(\log q)$ operations) and for $i = 0, 1, 2, \dots$ compute $h \cdot u^i$ by multiplying the previous value by u; after each, look it up in the table. By the decomposition some pair (i, j) matches, giving $x = im + j$. The search needs at most m giant steps. Total: $O(\sqrt{q})$ operations and $O(\sqrt{q})$ memory. □

Remark 2.12. BSGS is deterministic and unconditionally $O(\sqrt{q})$-time, but its $O(\sqrt{q})$ *memory* is the practical bottleneck: at the 128-bit security level one cannot feasibly store $\sqrt{q} \approx 2^{128}$ table entries. Pollard's rho method matches the time bound with negligible memory, which is why it, rather than BSGS, is the realistic generic attack.

Pollard's rho

Proposition 2.13 (Pollard's rho, sketch)**.** *In a cyclic group of order q (prime) there is a randomised algorithm computing discrete logarithms in expected $O(\sqrt{q})$ group operations and $O(1)$ storage.*

Proof. Sketch. Define a pseudo-random walk on G whose state is a group element together with its known representation $g^\alpha h^\beta$. Partition G into three sets S_1, S_2, S_3 of roughly equal size by some easily computed function of the element's label, and from a point $P = g^\alpha h^\beta$ step to

$$f(P) = \begin{cases} h \cdot P = g^\alpha h^{\beta+1}, & P \in S_1, \\ P^2 = g^{2\alpha} h^{2\beta}, & P \in S_2, \\ g \cdot P = g^{\alpha+1} h^\beta, & P \in S_3. \end{cases}$$

Each step updates the exponent pair $(\alpha, \beta) \in (\mathbb{Z}/q\mathbb{Z})^2$ by an explicit rule and costs $O(1)$ operations. The sequence $P_0, P_1, \ldots$ eventually cycles (the state space is finite), tracing the shape of the Greek letter ρ: a tail leading into a loop. Modelling f as a random function over q elements, the birthday bound makes a collision $P_i = P_j$ (with $i \neq j$) appear after $O(\sqrt{q})$ steps in expectation. A collision yields

$$g^{\alpha_i} h^{\beta_i} = g^{\alpha_j} h^{\beta_j} \implies g^{\alpha_i - \alpha_j} = h^{\beta_j - \beta_i}.$$

Writing $h = g^x$ and taking logarithms modulo the *prime* q:

$$\alpha_i - \alpha_j \equiv x(\beta_j - \beta_i) \pmod{q}.$$

If $\beta_j - \beta_i \not\equiv 0$ it is invertible (primality of q), and $x \equiv (\alpha_i - \alpha_j)(\beta_j - \beta_i)^{-1} \pmod{q}$. Floyd's two-pointer ("tortoise and hare") method, or Brent's improvement, detects the collision with $O(1)$ memory, without storing the trajectory. The exceptional case $\beta_j \equiv \beta_i$ has probability $O(1/q)$, and we re-randomise the start to handle it. □

Remark 2.14. Pollard rho parallelises almost perfectly via van Oorschot–Wiener *distinguished points*: P processors share collisions and achieve a $\sqrt{q}/P$ wall-clock with modest communication, so the relevant security measure is the *cost* of the attack, not the time on a single machine. For this reason concrete security estimates use the operation count $\sqrt{q}$ as the attacker's budget. A further constant-factor saving (a factor $\sqrt{2}$ in groups admitting the $\pm$ automorphism $P \mapsto -P$, as on elliptic curves) lowers the count to $\sqrt{\pi q/4}$, but leaves the $\Theta(\sqrt{q})$ scaling untouched.

The generic lower bound

These attacks are not merely the best known; up to constants they are the best *possible* for any generic algorithm. This is Shoup's theorem, the formal statement of the "square-root barrier".

> **Theorem 2.15** (Shoup's generic lower bound)**.** *Let q be prime. Any generic algorithm that, after making at most n queries to the group oracle, outputs the discrete logarithm of a uniformly random challenge in a group of order q succeeds with probability at most*
>
> $$\frac{\binom{n+2}{2} + 1}{q} = O\left(\frac{n^2}{q}\right).$$
>
> *In particular, to succeed with constant probability one needs $n = \Omega(\sqrt{q})$ queries.*

Proof. Sketch. Use Shoup's idealisation: a random injective label $\sigma : \mathbb{Z}/q\mathbb{Z} \to S$ encodes elements, and the algorithm learns labels only through the oracle. The challenge logarithm is an indeterminate X drawn uniformly from $\mathbb{Z}/q\mathbb{Z}$. Every group element the algorithm can form is, by induction on the queries, the image under σ of a known *linear* polynomial $a_i + b_i X \bmod q$ with coefficients the algorithm chose. Treat X as a formal variable: as long as all the polynomials $a_i + b_i X$ the algorithm has produced are pairwise distinct *as functions of* X, the labels it sees are consistent with X being any value, so the algorithm has gained no information and can do no better than guess, succeeding with probability $1/q$. It can gain information only through an "accidental" collision: two distinct polynomials $a_i + b_i X$ and $a_j + b_j X$ that happen to coincide at the actual secret value x, i.e. $(a_i - a_j) + (b_i - b_j)x \equiv 0 \pmod{q}$. For a fixed pair with $(a_i, b_i) \neq (a_j, b_j)$, this linear equation in x has at most one root modulo the prime q, so over a uniform x it occurs with probability $\leq 1/q$. With n queries (plus the inputs g and h) there are at most $\binom{n+2}{2}$ such pairs; a union bound gives total collision probability $\leq \binom{n+2}{2}/q$. Absent any collision the algorithm wins with probability $1/q$; thus its overall success is at most $\binom{n+2}{2}/q + 1/q = O(n^2/q)$. Solving $n^2/q = \Omega(1)$ gives $n = \Omega(\sqrt{q})$. □

Corollary 2.16 (Why $\log_2 q \geq 2\lambda$)**.** *To force every generic attacker to spend at least 2^λ group operations, hence to obtain λ bits of security against generic attacks, the prime group order q must satisfy*

$$\sqrt{q} \gtrsim 2^\lambda \qquad \Longleftrightarrow \qquad \log_2 q \geq 2\lambda.$$

Proof. By Propositions 2.11 and 2.13 a generic attacker needs only $\Theta(\sqrt{q})$ operations, and by Theorem 2.15 no generic attacker can do asymptotically better. Equating the attacker's cost $\sqrt{q}$ with the target work factor 2^λ gives $q \approx 2^{2\lambda}$, i.e. $\log_2 q \approx 2\lambda$; demanding at least this much yields $\log_2 q \geq 2\lambda$. □

Example 2.17 (The 128-bit target)**.** For $\lambda = 128$ one needs a prime subgroup of order q with $\log_2 q \geq 256$. This accounts for the standardised 256-bit elliptic curves (such as the NIST P-256 / secp256r1 family) targeting 128-bit security with 256-bit group orders: the field and the group order are both about 2^{256}, so $\sqrt{q} \approx 2^{128}$. The Pasta curves of §2.7, with 255-bit orders just above 2^{254} and $\approx$ 126-bit security, sit just below this class. Contrast finite-field DLP in $\mathbb{F}_p^\times$, where the sub-exponential index calculus (Remark 2.18) forces $\log_2 p$ up into the thousands of bits for the same λ; elliptic curves are valued precisely because no such sub-exponential generic-beating algorithm is known.

Remark 2.18 (Why this bound is special to elliptic curves)*.* The square-root barrier governs *generic* groups. In some concrete groups the representation of elements leaks enough structure to beat it. The multiplicative group $\mathbb{F}_p^\times$ is the cautionary case: *index calculus* exploits the fact that integers factor into small primes ("smooth" numbers) to solve DLP in sub-exponential time $\exp\big(O((\log p)^{1/3}(\log\log p)^{2/3})\big)$, far below $\sqrt{p}$. No analogue is known for the group of points on a well-chosen elliptic curve over $\mathbb{F}_p$, because there is no useful notion of a "small" point to factor over. This non-generic gap is precisely why elliptic-curve groups can be small (256 bits) where $\mathbb{F}_p^\times$ must be large (3072 bits) for the same security; it also means the "$\log_2 q \geq 2\lambda$" rule guarantees protection against *generic* attacks only, and one must additionally screen a curve against the special-purpose attacks of §2.7.

2.6 Pohlig–Hellman and the necessity of large prime order

The square-root barrier assumed q prime. If instead the group order factors, discrete logarithm splits into independent pieces of the size of the factors, and the difficulty reduces to that of the *largest prime* factor. This is the Pohlig–Hellman reduction; it is the reason Definition 2.2 insisted on prime order.

Theorem 2.19 (Pohlig–Hellman)**.** *Let $G = \langle g \rangle$ have order $N = \prod_{i=1}^{r} p_i^{e_i}$ with the p_i distinct primes. Then one can compute a discrete logarithm in G using*

$$O\left(\sum_{i=1}^{r} e_i\big(\log N + \sqrt{p_i}\,\big)\right)$$

group operations. In particular $\sqrt{p_{\max}}$ dominates the cost, where $p_{\max}$ is the largest prime dividing N.

Proof. We compute the logarithm $x = \log_g h \bmod N$ modulo each prime power $p_i^{e_i}$ and recombine it with the Chinese Remainder Theorem, valid because the $p_i^{e_i}$ are pairwise coprime and $\prod p_i^{e_i} = N$.

Reduction to prime-power order. Fix a prime power $p = p_i, e = e_i$. Put $g_i = g^{N/p^e}$ and $h_i = h^{N/p^e}$; then g_i has order exactly p^e and $\log_{g_i} h_i \equiv x \pmod{p^e}$, since raising to N/p^e kills the part of x coprime to p's power and leaves $x \bmod p^e$. Thus it suffices to solve DLP in the cyclic group $\langle g_i \rangle$ of order p^e.

Reduction to prime order, digit by digit. Write the unknown residue in base p:

$$x \equiv x_0 + x_1 p + \cdots + x_{e-1} p^{e-1} \pmod{p^e}, \qquad 0 \leq x_k < p.$$

Let $\gamma = g_i^{p^{e-1}}$, an element of order exactly p. To find x_0, raise h_i to the power p^{e-1}:

$$h_i^{p^{e-1}} = g_i^{xp^{e-1}} = \gamma^x = \gamma^{x_0},$$

the last step because all higher digits carry a factor p^e that annihilates γ. So $x_0 = \log_\gamma\left(h_i^{p^{e-1}}\right)$ is one discrete logarithm in a group of prime order p, solvable in $O(\sqrt{p})$ by BSGS or rho (Propositions 2.11, 2.13). Having x_0, set $h_i^{(1)} = h_i \cdot g_i^{-x_0}$; then $\log_{g_i} h_i^{(1)} \equiv x - x_0 \equiv x_1 p + \ldots \pmod{p^e}$ is divisible by p, and the same procedure at exponent p^{e-2} yields x_1. Iterating recovers $x_0, \ldots, x_{e-1}$ with e prime-order DLPs of cost $O(\sqrt{p})$ each, plus $O(e \log N)$ operations for the exponentiations.

Summing over i and applying CRT gives the stated bound. □

Example 2.20 (A composite-order trap)**.** Suppose, contrary to good practice, a group has order

$$N = 2^{256} - 1 = 3 \cdot 5 \cdot 17 \cdot 257 \cdot 65537 \cdot 641 \cdot 6700417 \cdot \ldots,$$

a product of small primes (it is *smooth*). Despite N having 256 bits, the largest prime factor of $2^{256} - 1$ is far below 2^{128}; Pohlig–Hellman solves DLP in roughly $\sqrt{p_{\max}}$ operations, perhaps only about 2^{36} (the largest prime factor here is $\approx 2^{72.3}$, so $\sqrt{p_{\max}} \approx 2^{36}$), eliminating the apparent 128-bit security. A 256-bit order is necessary but *not sufficient*: it must be (almost) prime.

Corollary 2.21 (Design rule for the group order)**.** *For DLP-based security at level λ the group must have order divisible by a prime q with $\log_2 q \geq 2\lambda$, and the cryptographic group must be (or be restricted to) the subgroup of that prime order q. When the natural group has order $N = h \cdot q$ with a small* cofactor *h and large prime q, one works in the order-q subgroup; clearing the cofactor (multiplying inputs by h or checking subgroup membership) prevents small-subgroup attacks that would otherwise leak x mod h via Pohlig–Hellman on the order-h part.*

Remark 2.22. This is the deeper justification for "prime order" in Definition 2.2. Prime order q makes Pohlig–Hellman vacuous (the only factor is q itself), makes every non-identity element a generator (no element lands in a weak proper subgroup), and removes the cofactor bookkeeping entirely. The Pasta curves below are engineered to have *prime* order, so the entire group is its own prime-order subgroup and the cofactor is 1.

2.7 Instantiation on elliptic curves; the Pasta curves

We now fix the concrete groups used in the remainder of the monograph. Recall from the companion volume that an elliptic curve over a finite field $\mathbb{F}_p$ (with $p > 3$ prime) in short Weierstrass form is

$$E : y^2 = x^3 + ax + b, \qquad a, b \in \mathbb{F}_p, \;\; 4a^3 + 27b^2 \neq 0,$$

and that its set of $\mathbb{F}_p$-points,

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 \,:\, y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

together with the chord-and-tangent addition and the point at infinity $\mathcal{O}$ as identity, forms a finite abelian group. The discrete logarithm in this group, "given P and $Q = xP$, find x", is the *elliptic-curve discrete logarithm problem (ECDLP)*, and CDH/DDH transcribe verbatim with multiplicative notation replaced by the additive scalar multiplication $x \mapsto xP$. Hasse's theorem pins down the group size.

Theorem 2.23 (Hasse bound, recalled)**.** *For E over $\mathbb{F}_p$, the number of points satisfies*

$$\left|\, \#E(\mathbb{F}_p) - (p+1) \,\right| \leq 2\sqrt{p}, \qquad \textit{i.e.} \qquad \#E(\mathbb{F}_p) = p + 1 - t, \quad |t| \leq 2\sqrt{p},$$

> *where t is the* trace of Frobenius. *Thus the group order is $p+1$ up to a deviation of order $\sqrt{p}$.*

For the group to give λ-bit security one chooses p of about 2λ bits (so $\#E(\mathbb{F}_p) \approx p \approx 2^{2\lambda}$) and insists, by Corollary 2.21, that $\#E(\mathbb{F}_p)$ be prime (cofactor 1). One then assumes ECDLP, ECCDH, and ECDDH hold in $E(\mathbb{F}_p)$ at the level Corollary 2.16 predicts, *provided* the curve avoids the known structural attacks that beat the generic $\sqrt{q}$ bound. The screening criteria are the following.

Definition 2.24 (Secure-curve criteria)**.** A curve $E/\mathbb{F}_p$ with prime order $q = \#E(\mathbb{F}_p)$ is admissible for λ-bit DLP security if:

1. (*Size / generic.*) $\log_2 q \geq 2\lambda$, so Pollard rho costs $\geq 2^\lambda$ (Cor. 2.16).
2. (*Prime order.*) q is prime, so Pohlig–Hellman is vacuous and the cofactor is 1 (Cor. 2.21).
3. (*Anti-MOV / large embedding degree.*) The multiplicative order k of p modulo q (the *embedding degree*) is large. The MOV/Frey–Rück attack uses a pairing to embed $E(\mathbb{F}_p)$ into $\mathbb{F}_{p^k}^\times$ and then runs sub-exponential index calculus there (Rem. 2.18); this is a threat only when k is small (e.g. supersingular curves, where $k \leq 6$). Demanding k huge defeats it.
4. (*Anti-Smart / not anomalous.*) $q \neq p$, i.e. the curve is not *anomalous* ($\#E(\mathbb{F}_p) = p$). On an anomalous curve the Smart–Satoh–Araki–Semaev attack uses a p-adic ("lift to the formal group / elliptic logarithm") map to solve ECDLP in *linear* time. The trace $t = 1$ case is fatal, and one must exclude it.
5. (*Twist security.*) The *quadratic twist* E' of E (the curve $dy^2 = x^3 + ax + b$ for a fixed nonsquare $d \in \mathbb{F}_p$; every $x \in \mathbb{F}_p$ is then the x-coordinate of a point of E or of E') should also have a large prime order, so that an attacker cannot push computation onto a weak twist via a fault or an invalid-point injection.

Definition 2.25 (The Pasta curves)**.** The *Pasta* curves are a pair of elliptic curves, *Pallas* and *Vesta*, in short Weierstrass form

$$y^2 = x^3 + 5,$$

(so $a = 0$, $b = 5$, a j-invariant-0 curve) defined over two primes p and q of 255 bits each, arranged into a *cycle*:

$$\#\text{Pallas} = q \text{ (Pallas is defined over } \mathbb{F}_p), \qquad \#\text{Vesta} = p \text{ (Vesta is defined over } \mathbb{F}_q).$$

That is, the order of each curve equals the field of definition of the other. Both p and q are prime, of bit-length 255, and are congruent to 1 mod 2^{32}, giving large 2-adic valuation in their multiplicative groups for fast FFTs. Each curve has cofactor 1: the whole group is of prime order.

Remark 2.26 (Why a cycle, and what it buys). The defining feature $\#\text{Pallas} = q$, the base field of Vesta, and $\#\text{Vesta} = p$, the base field of Pallas, is what makes (Pallas, Vesta) an *amicable / cyclic* pair. It lets one curve's scalar field be the other curve's coordinate field, so that a proof system can use Pallas-arithmetic to verify statements about Vesta-arithmetic and vice versa without expensive non-native field emulation. This is the mechanism behind recursive proof composition in Halo 2: the recursion alternates between the two curves, each verifying a proof expressed over the other's scalar field. The cycle structure does not affect the hardness assumptions employed, ECDLP/ECCDH/ECDDH on each curve; the cycle is an *efficiency* property, not a security one, and one screens the two curves independently against Definition 2.24.

Proposition 2.27 (The Pasta curves meet the criteria)**.** *Pallas and Vesta satisfy criteria (1)–(4) of Definition 2.24 at $\lambda \approx 126$; criterion (5) fails for both curves — neither twist has prime order, and Pallas's twist additionally falls below the SafeCurves 2^{100} twist-attack cost threshold (about 2^{88}, versus about 2^{108} for Vesta) — but the failure is rendered moot by mandatory point validation.*

Proof. Sketch, criterion by criterion. (1) Each order is a 255-bit prime just above 2^{254} (so $\log_2 q \approx 254.0$), giving $\log_2 q \geq 2\lambda$ with $\lambda = 127$ and generic $\sqrt{q} \approx 2^{127}$. Plain Pollard rho costs about $\sqrt{\pi q/2} \approx 2^{127.3}$ operations; the elliptic $\pm$-automorphism speedup (Remark 2.14) lowers this to $\sqrt{\pi q/4} \approx 2^{126.8}$; and the order-6 automorphism group of these j-invariant-0 curves — the factor $\sqrt{m}$ for an order-m automorphism group of Remark 2.28, here an extra $\sqrt{3}$ — lowers it to $\sqrt{\pi q/12} \approx 2^{126.0}$. Hence "about 126 bits" of security (the same range as the 128-bit curves of Example 2.17). (2) Both orders are prime by construction, so cofactor 1 and Pohlig–Hellman vacuous. (3) Both curves have very large embedding degree (the embedding degree of each is essentially $q-1$-sized, astronomically larger than any feasible index-calculus target), so they are *not* supersingular and MOV/Frey–Rück does not apply; indeed $y^2 = x^3 + 5$ over these primes is ordinary. (4) Neither curve is anomalous: #Pallas $= q \neq p$ and #Vesta $= p \neq q$, so the trace $t = p + 1 - q \neq 1$ and the Smart attack does not apply. (5) is the one criterion the pair does not meet: the Pasta search deliberately ignored twist security (the curves were generated with the search flag `--ignoretwist`), and neither twist has prime order. The Pallas twist has order $3^2 \cdot 7 \cdot 8191 \cdot 85021 \cdot 11540602760389 \cdot P_{176}$ for a 176-bit prime P_{176}, so the best attack on the twist costs about 2^{88}, below the SafeCurves 100-bit twist-security threshold; the Vesta twist, of order $3^3 \cdot 7 \cdot 13 \cdot 2851 \cdot 30097 \cdot P_{217}$ for a 217-bit prime P_{217}, happens to clear that threshold at about 2^{108}. This is harmless in our setting: twist attacks require an implementation that accepts attacker-supplied x-coordinates without validation (x-only ladders) or skips on-curve checks. Orchard implementations never use x-only arithmetic, and every deserialised point is checked to satisfy $y^2 = x^3 + 5$ (decompression of an x on the twist fails because $x^3 + 5$ is then a non-square); with cofactor 1 there are no small subgroups on the curve itself. Invalid-curve and twist-point injection are therefore excluded by validation, not by twist structure. Therefore the standard generic analysis governs, and the security level is ≈ 126 bits. □

Remark 2.28 (j-invariant 0 and endomorphisms). The choice $y^2 = x^3 + b$ gives j-invariant 0, a curve with complex multiplication by $\mathbb{Z}[\omega]$ where ω is a primitive cube root of unity. This furnishes a cheap endomorphism $\phi(x, y) = (\beta x, y)$ acting as multiplication by a fixed scalar ζ on the group, enabling the GLV speedup for honest scalar multiplication. The same endomorphism gives an attacker a slightly larger automorphism group and hence the constant-factor Pollard-rho speedup mentioned in Remark 2.14 (a factor $\sqrt{m}$ for an order-m automorphism group), but this is a small constant and does not change the $\Theta(\sqrt{q})$ scaling. CM by a small discriminant is, by itself, not known to weaken ECDLP; the danger lies only in *small embedding degree* and *anomalousness*, both excluded above.

2.8 The quantum caveat: Shor's algorithm

Every hardness assumption above is stated against classical PPT adversaries. A large fault-tolerant quantum computer changes the picture qualitatively, and we must state precisely what it does and does not break.

Theorem 2.29 (Shor, informally)**.** *There is a quantum algorithm that, given a cyclic group $G = \langle g \rangle$ of order q (with efficient group operations) and a target $h = g^x$, outputs x in time polynomial in* $\log q$*, succeeding with overwhelming probability. Consequently DLP, and hence CDH and DDH, are* not *hard against quantum adversaries: all three fall in quantum polynomial time, in* any *group, including elliptic-curve groups.*

Proof. Sketch. Discrete logarithm is an instance of the *hidden subgroup problem* over the abelian group $\mathbb{Z}/q\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}$. Define $f(\alpha, \beta) = g^{\alpha} h^{-\beta} = g^{\alpha - x\beta}$. Then $f(\alpha, \beta) = f(\alpha', \beta')$ iff $\alpha - x\beta \equiv \alpha' - x\beta' \pmod{q}$, so f is constant exactly on the cosets of the subgroup

$$H = \{(\alpha, \beta) : \alpha \equiv x\beta \pmod{q}\} = \langle (x, 1) \rangle \subseteq (\mathbb{Z}/q\mathbb{Z})^2,$$

a "line of slope x". The quantum algorithm prepares a *uniform superposition* over $(\mathbb{Z}/q\mathbb{Z})^2$—a single register holding every pair at once, each with equal weight—evaluates f into a second register, and applies the *quantum Fourier transform*, the reversible change of basis that re-expresses the register in the characters of $(\mathbb{Z}/q\mathbb{Z})^2$; measuring the result samples characters that vanish on H, and a handful of such samples reveal the slope x by classical linear algebra modulo q. The entire procedure uses $O(\mathrm{poly}(\log q))$ qubits and gates. Because it queries the group only through the black-box operation $(\alpha, \beta) \mapsto g^{\alpha} h^{-\beta}$, it is agnostic to the group's representation: it breaks elliptic-curve DLP exactly as it breaks $\mathbb{F}_p^{\times}$ DLP and integer factoring (the latter via the order-finding variant). □

Remark 2.30 (Quantum does not contradict the square-root barrier). There is no contradiction with Shoup's bound (Theorem 2.15): the proof of that lower bound assumes the *generic* model for *classical* algorithms making sequential, individual oracle queries. Shor's algorithm is non-generic in the relevant sense, it queries the group function in superposition and exploits the periodicity revealed by the Fourier transform, an ability the classical generic model does not grant. Hence the square-root barrier and Shor's polynomial-time attack coexist: the former bounds classical generic attackers, the latter is a quantum non-generic attacker.

Remark 2.31 (What Shor breaks and what survives). Shor's algorithm decisively breaks every assumption in this section, the entire discrete-log family and integer factoring, in polynomial time, so all of RSA, finite-field Diffie–Hellman, and elliptic-curve cryptography are *quantum-broken*. What it does *not* break:

- *Symmetric primitives and hash functions.* The best generic quantum attack on a search problem is Grover's algorithm, a quadratic, not exponential, speedup. A symmetric cipher with a 2λ-bit key, or a hash with a 2λ-bit output, retains λ bits of *quantum* security. Doubling key/output lengths suffices.
- *Post-quantum assumptions.* Hardness based on lattices (LWE, SIS), codes, multivariate systems, and isogenies is not known to fall to Shor and underlies post-quantum cryptography.

The discrete-log assumptions adopted here are therefore secure only under the standing premise that no cryptographically relevant quantum computer exists. Within that premise, and against all classical attacks, the Pasta curves deliver the ≈ 126-bit security analysed in §2.7, which is the ground on which the Halo 2 constructions to come are built.

Remark 2.32 (Standing assumptions for the sequel). We summarise here the premises on which the rest of the monograph relies. For each Pasta curve $C \in \{\text{Pallas}, \text{Vesta}\}$ with prime order q_C and generator g_C:

1. ECDLP is hard in $\langle g_C \rangle$: no classical PPT adversary finds x from xg_C with non-negligible probability (concretely, cost $\geq 2^{126}$).
2. ECCDH and, where needed, ECDDH hold in $\langle g_C \rangle$ (Definitions 2.4, 2.5).
3. The adversary is classical and probabilistic-polynomial-time (Remark 2.31).

By the hierarchy of §2.4, assuming ECDDH is the strongest and implies the rest; a given construction invokes the weakest assumption it requires.

3 Hash functions and the random oracle model

A hash function compresses arbitrary data into a short, fixed-length tag. This single, simple-sounding operation is the most ubiquitous tool in cryptography: it underlies digital signatures, commitment schemes, message authentication, key derivation, proof-of-work, Merkle trees, and the Fiat–Shamir transform that turns interactive proofs into the non-interactive arguments at the heart of the Halo 2 system. A hash function can serve so many purposes because, when it behaves "ideally," it acts like a deterministic source of fresh randomness keyed by its input. This section develops the subject from the ground up: we fix the syntax, isolate the three classical security notions and the generic birthday attack that bounds them, formalise the idealisation that the random oracle model provides (together with its discontents), treat the operations needed downstream—domain separation, hashing into a finite field, and hashing onto an elliptic curve—and close with the arithmetisation-friendly hash Poseidon that Orchard evaluates inside its circuit.

3.1 Syntax

We start from the bare data type. Throughout, $\{0,1\}^*$ denotes the set of all finite binary strings, $\{0,1\}^n$ the strings of length exactly n, and $|x|$ the length of a string x. We write concatenation as $x \parallel y$.

Definition 3.1 (Hash function). A *hash function* with output length $n \in \mathbb{N}$ is a function

$$H : \{0,1\}^* \longrightarrow \{0,1\}^n.$$

We call the elements of the codomain *digests*, *hashes*, or *tags*. H is *compressing* on a domain $D \subseteq \{0,1\}^*$ if D contains strings longer than n; when the domain is all of $\{0,1\}^*$ the function is overwhelmingly many-to-one.

Two remarks on the syntax are in order before we attach security to it.

Remark 3.2 (Keyed versus unkeyed; the foundations problem). Definition 3.1 describes a single, fixed, unkeyed function such as the SHA-256 standard. This is what practice deploys, but it is awkward for asymptotic security definitions: for any fixed compressing H there *exist* two strings $x \neq x'$ with $H(x) = H(x')$ (by the pigeonhole principle), and there exists a tiny program—"print these two hardcoded strings"—that outputs such a pair. How to write that program down efficiently is not known, but the asymptotic statement "no efficient adversary finds a collision" is false for a fixed function. The standard theoretical remedy is a *keyed family* (sometimes called a hash function ensemble): a function

$$H : \mathcal{K} \times \{0,1\}^* \longrightarrow \{0,1\}^n,$$

where one samples a key $s \in \mathcal{K}$ publicly at random and fixes it thereafter, written $H_s(\cdot) = H(s, \cdot)$. The key is not secret; it merely indexes which member of the family is in force, defeating the hardcoding objection because the adversary must succeed for a random member. Throughout this section we state security notions for the keyed family, since that is where the definitions are clean, but every concrete construction discussed is unkeyed, and the reader should mentally identify the key with "the choice of standard." We make the distinction explicit when it matters.

Remark 3.3 (Variable output length). Some primitives produce a digest of caller-chosen length; these are *extendable-output functions* (XOFs), with syntax $H : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$, where $H(x, \ell)$ has length ℓ and is a prefix-consistent stream (longer requests extend shorter ones). SHAKE128 and SHAKE256 from the SHA-3 family are the canonical examples. XOFs are the natural tool for hashing to a field or curve, where one needs a controlled number of uniform bits; we revisit them in §3.7.

3.2 Security notions: preimage, second-preimage, and collision resistance

The notion of a "good" hash function admits no single definition; distinct applications require distinct guarantees, and these guarantees form a strict hierarchy. We isolate the three classical notions

below. Each takes the form of a game between a challenger and a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, and the function is secure if every PPT $\mathcal{A}$ wins with probability negligible in a security parameter λ (with the output length $n = n(\lambda)$, the key length, and a polynomially bounded challenge input length $m = m(\lambda)$ all functions of λ; the games below sample the challenge x uniformly from $\{0,1\}^m$). Recall that a function $\varepsilon(\lambda)$ is *negligible*, written $\varepsilon \in \mathrm{negl}(\lambda)$, if it decays faster than the reciprocal of every polynomial.

Definition 3.4 (Preimage resistance / one-wayness)**.** A keyed hash family H is *preimage resistant* if for every PPT adversary $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{pre}}_{H}(\mathcal{A}) \;=\; \Pr[\, s \leftarrow \mathcal{K};\; x \leftarrow \{0,1\}^m;\; y \leftarrow H_s(x);\; x' \leftarrow \mathcal{A}(s,y) \;:\; H_s(x') = y \,] \;\in\; \mathrm{negl}(\lambda).$$

Informally: given a digest y of a random message, no efficient adversary can find *any* message hashing to y. Note that $\mathcal{A}$ need not recover the original x; any preimage wins.

Definition 3.5 (Second-preimage resistance)**.** A keyed hash family H is *second-preimage resistant* if for every PPT adversary $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{sec}}_{H}(\mathcal{A}) \;=\; \Pr[\, s \leftarrow \mathcal{K};\; x \leftarrow \{0,1\}^m;\; x' \leftarrow \mathcal{A}(s,x) \;:\; x' \neq x \;\wedge\; H_s(x') = H_s(x) \,] \;\in\; \mathrm{negl}(\lambda).$$

Here the challenger *gives* the adversary a specific message x, and the adversary must produce a different message colliding with it.

Definition 3.6 (Collision resistance)**.** A keyed hash family H is *collision resistant* if for every PPT adversary $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{col}}_{H}(\mathcal{A}) \;=\; \Pr[\, s \leftarrow \mathcal{K};\; (x,x') \leftarrow \mathcal{A}(s) \;:\; x \neq x' \;\wedge\; H_s(x) = H_s(x') \,] \;\in\; \mathrm{negl}(\lambda).$$

Here the adversary chooses *both* messages and need only make them collide.

The three notions differ in who chooses what, and this dictates their relative strength. The following proposition records the hierarchy; it is elementary but instructive, since the reductions reveal precisely why collision resistance is the strongest demand.

Proposition 3.7 (Hierarchy of notions)**.** *Let H be a keyed hash family that is compressing (input length $m > n$). Then:*

1. *Collision resistance implies second-preimage resistance.*
2. *Second-preimage resistance implies preimage resistance, up to a polynomial loss, when H is sufficiently compressing.*

Neither implication reverses in general.

Proof. (1) Collision ⇒ second-preimage. We give a reduction: from an adversary $\mathcal{A}$ breaking second-preimage resistance we construct $\mathcal{B}$ breaking collision resistance. On input key s, $\mathcal{B}$ samples $x \leftarrow \{0,1\}^m$ itself, runs $x' \leftarrow \mathcal{A}(s,x)$, and outputs the pair (x,x'). Whenever $\mathcal{A}$ succeeds one has $x' \neq x$ and $H_s(x') = H_s(x)$, which is precisely a collision. Hence $\mathsf{Adv}^{\mathrm{col}}_{H}(\mathcal{B}) \geq \mathsf{Adv}^{\mathrm{sec}}_{H}(\mathcal{A})$, and if the left side is negligible so is the right.

(2) Second-preimage ⇒ preimage (with compression). We argue the contrapositive: an adversary $\mathcal{A}$ inverting H yields $\mathcal{B}$ finding second preimages. On input (s, x), $\mathcal{B}$ computes $y = H_s(x)$, runs $x' \leftarrow \mathcal{A}(s, y)$, and outputs x'. When $\mathcal{A}$ succeeds, $H_s(x') = y = H_s(x)$, so x' is a valid digest preimage; it remains to argue $x' \neq x$ with good probability. This is where compression enters. Fix $\mathcal{A}$'s coins; for each digest y the output $x' = \mathcal{A}(s, y)$ is then a single string, so at most one input per fibre satisfies $x = \mathcal{A}(s, H_s(x))$—at most 2^n inputs in all, one per digest—and over the uniform choice of x, $\Pr[x' = x] \leq 2^n/2^m = 2^{n-m}$. Hence $\mathrm{Adv}^{\mathrm{sec}}_H(\mathcal{B}) \geq \Pr[\mathcal{A}\text{ succeeds}] - \Pr[x' = x] \geq \mathrm{Adv}^{\mathrm{pre}}_H(\mathcal{A}) - 2^{n-m}$. The loss is additive, not multiplicative: $\mathcal{A}$'s successes may concentrate on small fibres, so nothing bounds $\Pr[x' \neq x]$ conditioned on success. The hedge "sufficiently compressing" in the statement earns its keep here: for $m - n = \omega(\log \lambda)$ the term 2^{n-m} is negligible, and second-preimage resistance forces $\mathrm{Adv}^{\mathrm{pre}}_H(\mathcal{A})$ to be negligible as well.

Non-reversal. Contrived counterexamples demonstrate the separations. For "preimage-resistant but not collision-resistant," take any one-way H and define $H'(x) = H(\lfloor x \rfloor)$, where $\lfloor x \rfloor$ deletes the last bit; H' inherits one-wayness on a random input, yet $x \,\|\, 0$ and $x \,\|\, 1$ constitute an immediate collision. The other separations are similar; the implications are one-directional. □

Remark 3.8 (Which application needs which). The notions are not interchangeable. A password-hashing scheme storing H(password) requires *preimage* resistance and nothing more—an attacker who learns the digest must not recover an input. A scheme that signs $H(m)$ and permits an adversary to later substitute a forged document requires *second-preimage* resistance for the honestly chosen m. A scheme in which the *signer themselves* might cheat—preparing two contracts with the same hash, signing the benign one, and later presenting the malicious one—requires full *collision* resistance, since here the adversary controls both messages. Commitment schemes, Merkle trees, and the Fiat–Shamir transform all fall in this last, most demanding category, which is why collision resistance is the headline property of a cryptographic hash.

3.3 The birthday bound on generic collision finding

A *generic* attack that applies to *any* hash function, treating it as a black box, bounds collision resistance from above. This attack determines how large n must be: it sets the security ceiling that no construction can exceed. The attack is the birthday attack, named after the birthday paradox.

Lemma 3.9 (Birthday bound). *Let $H : \{0,1\}^* \to \{0,1\}^n$ and write $N = 2^n$. Suppose we evaluate H on q distinct inputs whose digests behave as independent uniform samples from $\{0,1\}^n$ (the idealised model). Let C be the event that two of the q digests coincide. Then*

$$\Pr[C] \;=\; 1 - \prod_{i=0}^{q-1}\Big(1 - \frac{i}{N}\Big),$$

and this satisfies the two-sided estimate

$$\Pr[C] \;\geq\; 1 - e^{-q(q-1)/(2N)}, \qquad \textit{more usefully} \quad \Pr[C] \;\leq\; \frac{q(q-1)}{2N} \;\leq\; \frac{q^2}{2N},$$

and for the lower direction $\Pr[C] \geq 1 - e^{-q(q-1)/(2N)} \geq \frac{1}{2}$ once $q \geq 1.18\sqrt{N}$. In particular a collision appears with constant probability after about $\sqrt{N} = 2^{n/2}$ evaluations.

Proof. This is the birthday-bound proposition of the *Math Guide* (§"The union bound and a birthday calculation"), with $N = 2^n$ and $k = q$; its proof establishes the exact product expression. Only the $1.18\sqrt{N}$ threshold is new: setting the exponent of the lower bound to $-\ln 2$ gives $q(q-1) = 2N \ln 2$, i.e. $q \approx \sqrt{2 \ln 2}\sqrt{N} \approx 1.18\sqrt{N}$, at which point $\Pr[C] \geq \frac{1}{2}$. □

Remark 3.10 (Reading the bound and its memory cost). The consequence is the *square-root* law: an n-bit digest yields only $n/2$ bits of collision security. To attain the conventional 128-bit collision security one must take $n = 256$; this is precisely why SHA-256 (and not SHA-128, which does not exist) is the workhorse, and why Fiat–Shamir transcript hashes must have at least 256-bit output. The naive attack as described stores all $q \approx 2^{n/2}$ digests, costing $2^{n/2}$ memory, but cycle-finding techniques (Floyd's or Brent's algorithm, and Pollard's rho / van Oorschot–Wiener parallel collision search) achieve the same $2^{n/2}$ time in essentially constant memory by iterating $x \mapsto H(x)$ and detecting the cycle, which renders the attack genuinely practical. Note the contrast with preimage and second-preimage resistance, whose best generic attack is brute-force search costing 2^n evaluations: these notions enjoy full n-bit security, twice the collision security, since there the adversary cannot exploit the birthday coincidence among self-chosen pairs.

3.4 The random oracle model

The security notions of §3.2 capture specific, isolated properties. In practice, however, we use hash functions as if they were much stronger: as if $H(x)$ were a uniformly random value, freshly and independently sampled for each new input x, yet consistently returned on repeats. This idealisation is the *random oracle*.

Definition 3.11 (Random oracle)**.** A *random oracle* with output length n is a function $\mathcal{O} : \{0,1\}^* \to \{0,1\}^n$ drawn uniformly at random from the set of all such functions. Equivalently, $\mathcal{O}$ runs *lazily*: it maintains a table; on a query x it returns the stored value if it has already seen x, and otherwise samples $\mathcal{O}(x) \leftarrow \{0,1\}^n$ uniformly, records it, and returns it. The defining features are: outputs on *distinct* inputs are *independent* and *uniform*; outputs on *repeated* inputs are *consistent*; and one can access the table *only* by querying—there is no shorter description of $\mathcal{O}$ than its (exponential) truth table.

Definition 3.12 (Random oracle model)**.** A scheme lives in the *random oracle model* (ROM) if every party, honest or adversarial, has oracle access to a single shared random oracle $\mathcal{O}$ in place of a concrete hash function, and one proves security relative to the random choice of $\mathcal{O}$. On deployment, a concrete hash such as SHA-256 or BLAKE2b *instantiates* $\mathcal{O}$.

The benefit of this idealisation to a proof is that it grants the analyst two properties that no concrete hash can offer but that a random oracle does, almost by definition.

Proposition 3.13 (Powers granted by the ROM)**.** *In a security proof in the ROM, the reduction (which simulates $\mathcal{O}$ to the adversary) gains two capabilities unavailable for a concrete hash:*

1. Extractability (observability): *the reduction sees every input on which the adversary queries $\mathcal{O}$. Since the only way to learn anything about $\mathcal{O}(x)$ is to query x, if the adversary's output depends on $\mathcal{O}(x)$ then the reduction holds x.*
2. Programmability: *the reduction may choose the response $\mathcal{O}(x)$ on a fresh query x to be* any *value of its liking, provided that value is distributed uniformly (so the simulation is perfect). It can thus plant a challenge—e.g. set $\mathcal{O}(x)$ equal to a value it must invert—inside an oracle answer.*

Proof. Both follow immediately from the lazy-sampling realisation (Definition 3.11). For observability, the reduction *is* the table-keeper: it receives each query x before answering, so it observes the adversary's query set in full. For programmability, on a not-yet-seen input the reduction may fill the table entry with any value, and as long as that value is uniform and independent of the adversary's

view so far, the adversary cannot distinguish it from a genuine fresh sample; the table maintains consistency on repeats. □

Example 3.14 (Why Fiat–Shamir needs the ROM)**.** The Fiat–Shamir transform—which renders Halo 2's interactive protocol non-interactive—replaces the verifier's random challenge e by $e = \mathcal{O}(\text{transcript so far})$. We prove soundness of the resulting non-interactive argument by *rewinding*: the reduction observes (capability 1) the transcript the prover commits to, then *reprograms* (capability 2) the oracle to return a fresh, independent challenge on that same transcript, extracting two accepting transcripts that share a prefix and thereby a witness. Neither step is available with a concrete hash, whose outputs the reduction can neither observe selectively nor reprogram. This is the paradigmatic example of a proof that lives only in the ROM, and it is the reason the ROM is indispensable to the cryptography this monograph builds toward.

3.5 What the ROM idealises, and its known limits

Remark 3.15 (The heuristic and its standard justification)*.* A ROM proof is a *heuristic*, not a theorem about the deployed system. The argument for trusting it proceeds: (i) one proves the scheme secure assuming $\mathcal{O}$ is a perfect random oracle; (ii) a hash function believed to have "no exploitable structure" instantiates $\mathcal{O}$; (iii) one *hopes* that any real attack would have to exploit some structure of the concrete hash that a random oracle lacks, and therefore would also break the (presumed structure-free) hash directly. The empirical track record is strong: schemes with ROM proofs (RSA-OAEP, RSA-FDH, Schnorr/EdDSA signatures via Fiat–Shamir, the Boneh–Franklin IBE) have resisted attack for decades, and a ROM proof reliably rules out the entire class of attacks that treat the hash as a black box, which is most of them. This is why the ROM is the dominant model for practical protocol design.

The heuristic is nonetheless known to be *unsound in the worst case*: there exist schemes provably secure in the ROM that become insecure for *every* concrete instantiation. This is not a minor caveat but a theorem, and a graduate reader should understand its shape.

> **Theorem 3.16** (Uninstantiability; Canetti–Goldreich–Halevi)**.** *There exists a (contrived) signature scheme, and likewise an encryption scheme, that is provably secure in the random oracle model but is insecure when* any *efficiently computable function replaces the random oracle. Hence no concrete hash function can soundly instantiate the random oracle for all schemes: a ROM proof does not, in general, imply security of the instantiated scheme.*

Proof. Sketch. The separation exploits the one structural difference between a random oracle and any concrete function: a concrete function H has a *short description* (its code), whereas a random oracle does not. The construction modifies a secure scheme by adding a "backdoor" clause to, say, the signing algorithm: "if the message m, parsed as a program, satisfies $m(\langle\text{description of the hash}\rangle) = H(0)$"—i.e. if the adversary submits a program that correctly predicts the hash's own output on a fixed point given the hash's code—"then output the secret signing key." In the ROM, $\mathcal{O}$ has no finite code for the adversary to feed in, and predicting $\mathcal{O}(0)$ without querying it is infeasible, so the backdoor clause never fires and the scheme inherits the security of the base scheme. But once a concrete H with known code instantiates $\mathcal{O}$, the adversary simply submits the program "compute $H(0)$," the clause fires, and the secret key leaks. The base scheme is secure, the modification is invisible in the ROM, and yet *every* concrete instantiation breaks. (Sharper constructions remove the artificiality, but this version conveys the mechanism: the ROM proof relies on $\mathcal{O}$ being un-codeable, a property no real hash has.) □

Remark 3.17 (How to read the limits)*.* The uninstantiability results are *contrived*: no natural, deployed scheme is known to suffer from them. The community's working stance is therefore: a ROM proof is strong positive evidence and rules out all black-box attacks, but it is not a guarantee, and one should prefer schemes with standard-model proofs when they are available at comparable cost, and treat the

concrete hash's structure with suspicion (e.g. avoid length-extension-prone constructions). A finer caveat relevant to post-quantum security is the *quantum* random oracle model (QROM), in which the adversary may query $\mathcal{O}$ in superposition; observability and programmability are subtler there (one cannot freely "read off" a superposition query), and several classical ROM proofs require genuinely new techniques to port to the QROM. We flag this because Halo 2's long-term security goals include plausible post-quantum arguments, where the QROM is the right model.

3.6 Domain separation and personalisation

A single hash function routinely serves many logically distinct purposes within one protocol—to derive keys, to commit to notes, to tag notes as spent, to seed Fiat–Shamir challenges. If all these uses draw on the literally same function H, an adversary can *replay* an output produced for one purpose as if it were produced for another, since the digests are interchangeable bit strings. *Domain separation* is the discipline of making the same physical hash behave like a family of *independent* oracles, one per purpose.

Definition 3.18 (Domain separation)**.** Let H be a hash (modelled as a random oracle) and let $\{\tau_i\}_{i\in I}$ be a set of distinct, prefix-free *domain tags* (also called *labels* or *personalisation strings*), one per intended use. The use-i hash is

$$H^{(i)}(x) \;=\; H\big(\tau_i \,\|\, x\big).$$

The tags are *prefix-free* (no τ_i is a prefix of another, e.g. all of equal length, or each length-prefixed) so that the input spaces $\tau_i \,\|\, \{0,1\}^*$ are pairwise disjoint.

Proposition 3.19 (Domain separation yields independent oracles)**.** *If H is a random oracle and the tags $\{\tau_i\}$ are prefix-free, then the family $\{H^{(i)}\}_{i\in I}$ is distributed exactly as a collection of* independent *random oracles. Consequently a digest obtained from $H^{(i)}$ is, except with the negligible probability of a generic collision, useless to an adversary against a use $j \neq i$.*

Proof. By prefix-freeness the domains $\tau_i \,\|\, \{0,1\}^*$ are pairwise disjoint, so the inputs queried through different $H^{(i)}$ are never equal. A random oracle assigns independent uniform values to distinct inputs (Definition 3.11); restricting the single oracle H to the disjoint slices indexed by the tags therefore yields independent uniform functions on each slice, which is precisely a family of independent random oracles. The replay statement follows: a value $H^{(i)}(x)$ reveals nothing about any $H^{(j)}(\cdot)$ for $j \neq i$, as the latter are independent of it. □

Example 3.20 (Personalisation in practice)**.** Three idioms implement Definition 3.18. *(a) Prefix tagging:* prepend an ASCII context string, e.g. `"Zcash_nf"` for nullifiers versus `"Zcash_cm"` for note commitments, ensuring distinct slices. *(b) Native personalisation:* BLAKE2b's parameter block carries a 16-byte personalisation field that mixes into the IV, so `BLAKE2b(person="ZcashKDF")` and `BLAKE2b(person="ZcashPRF")` are, by construction, independent functions without touching the message at all—this is why Zcash favours BLAKE2 for its oracles. *(c) Suffix/framing in XOFs:* SHA-3 reserves domain-separation *suffix bits* (01 for SHA-3 hashing, 1111 for SHAKE) appended before padding, so that SHA3-256 and SHAKE256, sharing the same permutation, can never collide across uses. The unifying requirement, in all three, is that distinct uses induce disjoint effective inputs to the underlying primitive—realised in idiom (a) by the prefix-freeness of Proposition 3.19: an ad hoc tag that is a prefix of another silently merges two domains and reopens the replay attack.

3.7 Hashing to a field element

The cryptography ahead lives over a finite field $\mathbb{F}_p$ (and an elliptic curve over it), not over bit strings. We must turn an arbitrary message into a field element—and later a curve point—in a manner that inherits the uniformity of a random oracle. The obstacle is subtle: a hash gives uniform *bits*, but reducing n uniform bits modulo a prime p does *not* give a uniform element of $\mathbb{F}_p$ without care, since 2^n is not a multiple of p.

Definition 3.21 (Hashing to a field, via expand-then-reduce)**.** Let p be a prime with $\lceil \log_2 p \rceil = L$ bits. Fix a XOF or domain-separated hash expand producing arbitrary-length uniform output. To map a message m to a field element $\mathbb{F}_p$:

1. Compute $b \leftarrow \text{expand}\big(\tau \,\|\, m,\ L+k\big)$, a string of $L+k$ uniform bits, where k is a security slack (commonly $k = 128$) and τ is a domain tag.
2. Interpret b as an integer $B \in \{0, \ldots, 2^{L+k} - 1\}$ and output $B \bmod p \in \mathbb{F}_p$.

Proposition 3.22 (Near-uniformity of expand-then-reduce)**.** *If* expand *is a random oracle, the output of Definition 3.21 lies within statistical distance at most $p \cdot 2^{-(L+k)} \le 2^{-k}$ of the uniform distribution on $\mathbb{F}_p$. Hence with $k = 128$ no efficient algorithm can distinguish the map from a uniform field element.*

Proof. Let B be uniform on $\{0, \ldots, M-1\}$ with $M = 2^{L+k}$. For a target residue $r \in \{0, \ldots, p-1\}$, the number of $B \in \{0, \ldots, M-1\}$ with $B \equiv r \pmod p$ is either $\lfloor M/p \rfloor$ or $\lceil M/p \rceil$, differing by one. Thus $\left|\Pr[B \bmod p = r] - 1/p\right| \le 1/M$ for each of the p residues, and the statistical distance to uniform is

$$\tfrac{1}{2} \sum_{r=0}^{p-1} \left| \Pr[B \bmod p = r] - \tfrac{1}{p} \right| \;\le\; \tfrac{1}{2}\, p \cdot \frac{1}{M} \;=\; \frac{p}{2^{L+k+1}} \;\le\; 2^{-(k+1)} \cdot \frac{p}{2^L} \;\le\; 2^{-k},$$

using $p < 2^L$ so $p/2^L < 1$. The bias from naive single-block reduction (taking $k = 0$) can reach as much as 2^{-1} of a residue's mass when p is just above a power of two; the slack k is exactly what reduces it to negligibility. □

Remark 3.23 (The standardised hash-to-field). The IETF "hash-to-curve" specification packages Definition 3.21 as a routine hash_to_field(m, count) that produces count field elements, internally calling an expand_message primitive (either expand_message_xmd built on a fixed-output hash like SHA-256, or expand_message_xof built on SHAKE) with a mandatory domain-separation tag (DST). The specification sets the L parameter there to $\lceil (\lceil \log_2 p \rceil + k)/8 \rceil$ bytes with k the suite's target security level in bits (RFC 9380's own suites for 255-bit fields, such as the curve25519 suites, set $k = 128$, i.e. $L = 48$ bytes; the Zcash suite for Pallas and Vesta is more generous, taking $k = 256$ so that each element is reduced from a full 64-byte BLAKE2b-512 digest), exactly realising the slack of Proposition 3.22. This is the concrete recipe we use to map identities to field elements and to produce the field elements inside hash-to-curve (§3.8); the deployed Fiat–Shamir transcript realises the same wide-output-then-reduce principle of Proposition 3.22 by squeezing a personalised BLAKE2b state (*Halo 2 Guide*).

3.8 Hashing to a curve point

Finally, we must map a message to a point of an elliptic curve $E/\mathbb{F}_p$—to derive the "nothing-up-my-sleeve" generators of a commitment scheme, or to hash to a point in a verifiable random function. Let E be given in short Weierstrass form by $y^2 = g(x) = x^3 + ax + b$ over $\mathbb{F}_p$. Two requirements pull against each other: the output point should be *uniform* (or close to it) in the relevant subgroup, modelling a random oracle into the group; and the map should be *constant-time*, leaking nothing through timing.

Remark 3.24 (The naive try-and-increment map and its flaw). The obvious approach: hash m to a candidate $x \in \mathbb{F}_p$ (via §3.7), test whether $g(x)$ is a quadratic residue (so that a y exists), and if not increment x and retry. This is correct and roughly uniform, but the *number of trials* depends on the message—about half of all x fail the residue test (by the theory of the Legendre symbol $\left(\frac{g(x)}{p}\right)$)—so the running time leaks information about m, an avenue for side-channel attacks. Constant-time hashing to a curve requires a map that succeeds on the *first* try for every input.

The modern solution is an explicit, everywhere-defined algebraic map from $\mathbb{F}_p$ to $E(\mathbb{F}_p)$. The basic tool is the *simplified Shallue–van de Woestijne–Ulas (SWU) map*, which applies directly to curves with $ab \neq 0$. For curves with j-invariant 0 ($a = 0$, such as the Pallas/Vesta curves of Halo 2) or 1728 ($b = 0$) the simplified SWU map does not apply directly; instead one evaluates it on an *isogenous* auxiliary curve with $a'b' \neq 0$—one related to the target curve by an *isogeny*, a nonconstant map between curves, given by polynomial formulas in the coordinates, that respects the group law—and transports the result back through the isogeny map (see Remark 3.27).

Definition 3.25 (Simplified SWU map, high level). Let $E : y^2 = g(x) = x^3 + ax + b$ over $\mathbb{F}_p$ with $ab \neq 0$, and fix a nonsquare $Z \in \mathbb{F}_p$ (a "nothing-up-my-sleeve" constant, e.g. the smallest nonsquare). The simplified SWU map $\mathrm{SWU} : \mathbb{F}_p \to E(\mathbb{F}_p)$ sends a field element u to a point as follows. It computes two candidate x-coordinates,

$$x_1 = \frac{-b}{a}\Big(1 + \frac{1}{Z^2u^4 + Zu^2}\Big), \qquad x_2 = Zu^2\, x_1,$$

(with a defined fallback when the denominator vanishes), and uses the key algebraic identity

$$g(x_2) = Z^3u^6\, g(x_1),$$

together with the fact that Z is a nonsquare, to prove that *exactly one* of $g(x_1), g(x_2)$ is a quadratic residue. It then sets $x = x_1$ if $g(x_1)$ is square and $x = x_2$ otherwise, takes $y = \sqrt{g(x)}$ with a fixed sign convention, and outputs (x, y).

Proposition 3.26 (Correctness of the simplified SWU branch selection). *With notation as in Definition 3.25 and u such that the denominator $D = Z^2u^4 + Zu^2$ is nonzero, the map is well-defined: exactly one of $g(x_1)$ and $g(x_2)$ is a square in $\mathbb{F}_p$, so $\mathrm{SWU}(u)$ is a genuine point of $E(\mathbb{F}_p)$ computed without any trial loop.*

Proof. Sketch. The defining relation $g(x_2) = Z^3u^6\, g(x_1)$ is a polynomial identity in u; substituting $x_2 = Zu^2x_1$ into g and simplifying using the formula for x_1 verifies it, and we take it as given. Now apply the multiplicativity of the Legendre symbol:

$$\left(\frac{g(x_2)}{p}\right) = \left(\frac{Z^3u^6}{p}\right)\left(\frac{g(x_1)}{p}\right) = \left(\frac{Z}{p}\right)\left(\frac{Z^2u^6}{p}\right)\left(\frac{g(x_1)}{p}\right) = \left(\frac{Z}{p}\right)\left(\frac{g(x_1)}{p}\right),$$

since $Z^2u^6 = (Zu^3)^2$ is a nonzero square and contributes a Legendre symbol of $+1$ (assuming $u \neq 0$; the case $u = 0$ maps to a fixed point). Because Z is a *nonsquare*, $\left(\frac{Z}{p}\right) = -1$, hence $\left(\frac{g(x_2)}{p}\right) = -\left(\frac{g(x_1)}{p}\right)$. The two symbols are opposite, so precisely one of $g(x_1), g(x_2)$ is a nonzero square and admits a square root y; the branch selection picks that one. Thus the output is always a valid affine point, on the first try, for every admissible u. □

Remark 3.27 (From a field point to a uniform group point; full hash-to-curve). A single SWU evaluation is *not* surjective onto $E(\mathbb{F}_p)$ and its image is not uniform; one SWU output covers only part of the

curve and with a non-flat distribution. The standardised hash_to_curve(m) therefore composes several stages: (i) hash_to_field($m, 2$) yields two independent near-uniform field elements u_0, u_1 (§3.7); (ii) apply the map to each, $Q_0 = \mathrm{SWU}(u_0)$, $Q_1 = \mathrm{SWU}(u_1)$; (iii) add them on the curve, $R = Q_0 + Q_1$—summing two independent images provably smooths the distribution to within negligible distance of uniform on $E(\mathbb{F}_p)$; and (iv) *clear the cofactor* by multiplying R by $h = \#E(\mathbb{F}_p)/r$ to land in the prime-order subgroup of order r where the cryptography occurs. For curves with $a = 0$ (j-invariant 0), such as Pallas/Vesta, the simplified SWU map of Definition 3.25 does not apply directly—its x_1 formula needs $ab \neq 0$—so one instead evaluates simplified SWU on an isogenous auxiliary curve $E' : y^2 = x^3 + a'x + b'$ with $a'b' \neq 0$ and then transports the resulting point back to E through the *isogeny map* $E' \to E$; the obstruction is intrinsic, for the j-invariant is an isomorphism invariant: a Weierstrass change of variables rescales (a, b) to (u^4a, u^6b), so no model of these curves attains $ab \neq 0$, and the simplified-SWU route always passes through the isogeny. (Curves in Montgomery or twisted Edwards form are hashed by a different standardised map, Elligator 2, with no isogeny; prime-order curves such as Pallas and Vesta admit no such form, their order being odd while Montgomery and Edwards curves have order divisible by four.) The result is a hash $\{0,1\}^* \to E(\mathbb{F}_p)$ that is constant-time, able to stand in for a random oracle into the group in security arguments (no efficient distinguisher can exploit its internal structure to tell the two apart), and the standard way to produce independent generators for Pedersen-style commitments.

Remark 3.28 (Nothing-up-my-sleeve generators). A recurring use of hash-to-curve is generating the public generators $G_1, \ldots, G_k$ of a vector commitment so that *no one knows any discrete-log relation* $\sum_i a_i G_i = 0$ among them—knowledge of such a relation would let a committer open a commitment two ways, breaking binding. The defence is a *nothing-up-my-sleeve* (NUMS) construction: derive each $G_i = \text{hash\_to\_curve}(\tau \,\|\, i)$ from a fixed public domain tag τ (often a human-readable string such as a protocol name) and the index i. Because the generators are outputs of a function modelled as a random oracle, finding a discrete-log relation among them reduces to the discrete-log problem on E, which is hard; and because the seed is a transparent, auditable string with no hidden parameters, no party could have engineered a backdoor relation in advance. This is exactly how Halo 2 produces its Pedersen/inner-product commitment generators, and it is the culminating application that ties this section's three threads—collision-resistant hashing, the random-oracle idealisation, and hashing to a curve—into the commitment machinery the next sections build.

3.9 Arithmetisation-friendly hashing: Poseidon

Every hash described above (SHA-2, BLAKE2) is *bit-oriented*: its round function shuffles and XORs 32- or 64-bit words. Inside an arithmetic circuit over a large prime field $\mathbb{F}_p$ — the setting of the proof systems of the *Halo 2 Guide* — such a function is prohibitively expensive, since field arithmetic and range checks must re-express every Boolean operation. *Arithmetisation-friendly* hashes follow the opposite design: the round function consists of a few low-degree *field* operations, incurring few constraints. *Poseidon* (Grassi, Khovratovich, Rechberger, Roy, and Schofnegger, 2019) is the instance Orchard employs.

The sponge construction. Poseidon is a fixed permutation $f : \mathbb{F}_p^t \to \mathbb{F}_p^t$ on a state of t field elements, operated in *sponge* mode: the state splits into a *rate* of r elements (into which input is added and from which output is read) and a *capacity* of $c = t - r$ elements (never accessed directly — the security reserve). The sponge *absorbs* the message into the rate, r elements at a time with an application of f between blocks, after which it *squeezes* the output from the rate. Collision and preimage resistance hold up to approximately $\sqrt{p^c}$ work, as for a random sponge.

The permutation. f is a sequence of rounds, each comprising three steps: *add round constants* (a fixed vector in $\mathbb{F}_p^t$); a non-linear *S-box* $x \mapsto x^\alpha$ with $\alpha \geq 3$ the smallest integer satisfying $\gcd(\alpha, p-1) = 1$, so that the map is a bijection ($\alpha = 5$ over the Pasta fields, since $3 \mid p - 1$ there); and a linear *mix*

by a fixed maximum-distance-separable matrix $M \in \mathbb{F}_p^{t\times t}$. The cost-saving principle (the "HADES" strategy) is that only a few *full rounds* at the start and end need the full S-box — applied to *all* t elements; in the many *partial rounds* between them the S-box touches a *single* element, while the MDS mixing nonetheless diffuses it across the whole state. The round counts stand against the known algebraic (Gröbner-basis) attacks.

Circuit cost. The only non-linear operation is $x \mapsto x^5$, which an arithmetic circuit verifies with a couple of multiplication gates per S-box; the round-constant additions and the MDS multiplication are linear and almost free. A 2-to-1 Poseidon hash costs a few hundred constraints, against tens of thousands for a bit-oriented hash — the difference between a practical circuit and an impractical one.

The Orchard instance. Orchard fixes $t = 3$ (rate 2, capacity 1), $\alpha = 5$, and 8 full plus 56 partial rounds, over the Pallas base field, at the 128-bit security level. Orchard uses it in "constant-input-length" mode as a two-input hash

$$\mathsf{PoseidonHash}(x, y) = f\big([\,x,\ y,\ 2^{65}\,]\big)_1,$$

the first state element after one permutation of the input padded by the fixed capacity value 2^{65} (so no message padding is then required). Keyed by its first argument it is the pseudorandom function $F_{\mathsf{nk}}(\rho) = \mathsf{PoseidonHash}(\mathsf{nk}, \rho)$ from which Orchard constructs the nullifier (*Ironwood Guide*); unlike the Pedersen and Sinsemilla hashes constructed later (§7.5, §7.7), its security rests on cryptanalytic confidence in the round function rather than on a clean assumption such as discrete log.

4 Pseudorandom functions, block ciphers, and format-preserving encryption

The previous sections developed the machinery of one-way functions, collision-resistant hashing, and the abstract notion of computational indistinguishability. We now assemble these ideas into one of the most versatile tools in symmetric cryptography: the *pseudorandom function* (PRF). A PRF is a keyed family of functions that, to any efficient observer who does not know the key, behaves exactly like a function chosen uniformly at random. From this single primitive one obtains symmetric encryption, message authentication, key derivation, and, as we show at the end, encryption schemes that preserve the format of their input — the keyed bijection on a finite set that Orchard uses to derive the diversifiers behind its shielded addresses.

The development proceeds from first principles. We recall the model of computation and adversaries (§4.1); define pseudorandom functions and permutations and state the PRP/PRF switching lemma that licenses treating a permutation as a function (§4.2); explain the realisation of pseudorandom permutations by block ciphers, with AES as the running example (§4.3); construct PRFs directly from hash functions via keyed BLAKE2 (§4.4); and finally develop format-preserving encryption and the NIST FF1 mode as a Feistel network over a PRF (§4.5).

4.1 The computational model, recalled

$\{0,1\}^\ell$ denotes the set of bit strings of length ℓ, $\{0,1\}^*$ the set of all finite bit strings, and $|x|$ the length of a string x. For a finite set S, the notation $x \xleftarrow{\$} S$ means that x is drawn uniformly at random from S, independently of everything else. The adversarial model is that of Section 1: adversaries are probabilistic polynomial-time (PPT) algorithms in the security parameter λ (Definition 1.3), $\mathrm{negl}(\lambda)$ denotes a negligible function (Definition 1.11), and every security notion below is a distinguishing game between two experiments, scored by the distinguishing advantage of Definition 1.18, written $\mathsf{Adv}(\mathcal{A}) = \left|\Pr[\mathcal{A} \text{ outputs 1 in } \mathsf{Exp}_1] - \Pr[\mathcal{A} \text{ outputs 1 in } \mathsf{Exp}_0]\right|$.

The one device not yet formalised is *oracle access*. We write $\mathcal{A}^f$ for an adversary granted a black box for the function f: it may submit inputs x of its choice and receive $f(x)$ in a single step, but learns nothing about f beyond the answers to the queries it makes. The running time always bounds the number of oracle queries, which is hence polynomial.

4.2 Pseudorandom functions and permutations

We first make a "truly random function" precise, so that we can measure pseudorandomness against it.

> **Definition 4.1** (Random function)**.** Let $\mathsf{Func}(D, R)$ denote the set of all functions from a finite set D to a finite set R. A *random function* from D to R is an element $F \xleftarrow{\$} \mathsf{Func}(D, R)$, chosen uniformly at random. Equivalently, the values $\{F(x)\}_{x \in D}$ are independent and uniformly distributed over R.

There are $|R|^{|D|}$ functions in $\mathsf{Func}(D, R)$, an astronomically large set for the domains of interest (e.g. $D = R = \{0,1\}^{128}$ gives $2^{128 \cdot 2^{128}}$ functions). One can neither store nor transmit the description of such a function. The essential property of a PRF is that it replaces this object by a short key while preserving its observable behaviour. A clean mental model is *lazy sampling*: an oracle for a random function maintains a table, initially empty; on a fresh query x it samples $F(x) \xleftarrow{\$} R$, records the pair, and returns it; on a repeated query it returns the recorded value. This produces exactly the distribution of Definition 4.1 while only ever materialising the queried points.

Definition 4.2 (Keyed function)**.** A *keyed function* is a deterministic, efficiently computable map

$$F : \{0,1\}^{\lambda} \times \{0,1\}^{\ell(\lambda)} \longrightarrow \{0,1\}^{m(\lambda)}, \qquad (k,x) \longmapsto F_k(x),$$

where ℓ, m are polynomially bounded functions of the key length λ. Here λ is the key length, $\ell(\lambda)$ the input length, and $m(\lambda)$ the output length. Fixing the first argument to a key k yields a function $F_k : \{0,1\}^{\ell(\lambda)} \to \{0,1\}^{m(\lambda)}$.

Definition 4.3 (Pseudorandom function)**.** A keyed function F (Definition 4.2) is a *pseudorandom function* (PRF) if for every PPT adversary $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{prf}}_{F}(\mathcal{A}) \;=\; \Big|\; \Pr_{k \xleftarrow{\$} \{0,1\}^{\lambda}} \left[\mathcal{A}^{F_k}(1^{\lambda}) = 1\right] \;-\; \Pr_{R \xleftarrow{\$} \mathsf{Func}(\ell(\lambda), m(\lambda))} \left[\mathcal{A}^{R}(1^{\lambda}) = 1\right] \Big| \;\leq\; \mathrm{negl}(\lambda),$$

where $\mathsf{Func}(\ell, m) = \mathsf{Func}(\{0,1\}^{\ell}, \{0,1\}^{m})$, and the input 1^{λ} (the security parameter in unary) merely conveys to $\mathcal{A}$ the parameter size.

Stated informally: an adversary handed a black box that is either F_k for a secret random key, or a freshly sampled truly random function, and allowed to query it adaptively, cannot determine which it has. Two features merit emphasis. First, the adversary obtains *oracle* access only — it never sees the key, and it cannot read the code of the function, only its input–output behaviour. Second, the comparison object is the gigantic, unstorable random function; the PRF compresses this into λ bits while remaining computationally indistinguishable from it. Information theory cannot afford this trade: a keyed function takes at most 2^{λ} values as k ranges over the keys, a vanishing fraction of all $|R|^{|D|}$ functions, so an *unbounded* adversary could in principle distinguish. Pseudorandomness is an inherently computational notion.

Remark 4.4 (Necessity of oracle access and of adaptivity)*.* The adversary chooses its queries; it need not commit to them in advance and may let later queries depend on earlier answers. This *adaptive* access is the strongest reasonable model and the one that downstream constructions rely on. A weaker "non-adaptive" definition, in which the adversary fixes all queries before seeing any answer, is genuinely weaker and insufficient for many applications.

A block cipher is not merely a function but a *permutation* for each key — it must be invertible so that decryption is possible. The relevant ideal object is therefore a random *permutation*.

Definition 4.5 (Keyed permutation)**.** A keyed function $F : \{0,1\}^{\lambda} \times \{0,1\}^{\ell} \to \{0,1\}^{\ell}$ (with equal input and output length) is a *keyed permutation* if for every key k the map $F_k : \{0,1\}^{\ell} \to \{0,1\}^{\ell}$ is a bijection, and one can efficiently compute both F_k and its inverse F_k^{-1} given k. $\mathsf{Perm}(\ell)$ denotes the set of all permutations of $\{0,1\}^{\ell}$, of which there are $2^{\ell}!$.

Definition 4.6 (Pseudorandom permutation)**.** A keyed permutation F is a *pseudorandom permutation* (PRP) if for every PPT adversary $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{prp}}_{F}(\mathcal{A}) \;=\; \Big|\Pr_{k}\left[\mathcal{A}^{F_k}(1^{\lambda}) = 1\right] - \Pr_{P \xleftarrow{\$} \mathsf{Perm}(\ell)} \left[\mathcal{A}^{P}(1^{\lambda}) = 1\right] \Big| \;\leq\; \mathrm{negl}(\lambda).$$

If in addition no PPT adversary can distinguish even when given oracle access to *both* the permu-

tation and its inverse,

$$\mathsf{Adv}^{\mathrm{sprp}}_{F}(\mathcal{A}) \;=\; \Big|\Pr_{k}\big[\mathcal{A}^{F_k,F_k^{-1}}(1^\lambda)=1\big] - \Pr_{P\xleftarrow{\$}\mathsf{Perm}(\ell)}\big[\mathcal{A}^{P,P^{-1}}(1^\lambda)=1\big]\Big| \;\le\; \mathrm{negl}(\lambda),$$

then F is a *strong* pseudorandom permutation (SPRP).

The distinction between a PRP and an SPRP is the availability of a decryption oracle. A block cipher used for encryption where the attacker can also obtain decryptions of chosen ciphertexts must be an SPRP; this is the natural security target for a block cipher.

Example 4.7 (A random permutation is almost a random function)**.** Consider an adversary that queries a length-preserving oracle on q distinct inputs. If the oracle is a random function $R \in \mathsf{Func}(\ell,\ell)$, the answers are q independent uniform strings, and collisions (two queries with equal answers) occur. If the oracle is a random permutation $P \in \mathsf{Perm}(\ell)$, the answers are q *distinct* uniform strings, and no collision can occur. This is the *only* statistical difference between the two ideal objects from the outside, and it is the entire content of the PRP/PRF switching lemma.

Lemma 4.8 (PRP/PRF switching lemma)**.** *Let $\ell \ge 1$ and let $\mathcal{A}$ be any adversary — even a computationally unbounded one — making at most q queries to its oracle. Then*

$$\Big|\Pr_{P\xleftarrow{\$}\mathsf{Perm}(\ell)}\big[\mathcal{A}^{P}=1\big] - \Pr_{R\xleftarrow{\$}\mathsf{Func}(\ell,\ell)}\big[\mathcal{A}^{R}=1\big]\Big| \;\le\; \frac{q(q-1)}{2^{\ell+1}}.$$

We use the lemma as a statement only; its proof is a standard game-hopping argument bounding the probability of the single distinguishing event of Example 4.7, an output collision. The consequence is that a secure PRP on a large domain is also a secure PRF whenever the number of queries stays well below the birthday bound $2^{\ell/2}$: for AES, $q^2/2^{129}$ is negligible for any realistic query budget. This is the licence by which the FF1 construction of §4.5 treats AES — a permutation — as the PRF its round function requires.

4.3 Block ciphers and AES

Definition 4.9 (Block cipher)**.** A *block cipher* with block length b and key length κ is a keyed permutation

$$E:\ \{0,1\}^{\kappa}\times\{0,1\}^{b} \longrightarrow \{0,1\}^{b},$$

together with its inverse $D = E^{-1}$ satisfying $D_k(E_k(x)) = x$ for all k, x. Here E_k is *encryption* and D_k is *decryption* under key k. The security goal is that E be a strong pseudorandom permutation (Definition 4.6).

A block cipher is thus a concrete, fast, fixed-parameter instantiation of the SPRP abstraction. Whereas the PRP definition is asymptotic (a family indexed by λ), a real block cipher fixes b and κ once and for all (e.g. $b = 128$, $\kappa \in \{128, 192, 256\}$ for AES) and offers *concrete security*: an explicit function of the adversary's resources bounds the advantage of the best known attacks. We state pseudorandomness for block ciphers in concrete terms.

Definition 4.10 (Concrete (t,q,ε)-security)**.** A block cipher E is a (t,q,ε)-*secure* PRP if every adversary running in time at most t and making at most q oracle queries has $\mathsf{Adv}^{\mathrm{prp}}_{E}(\mathcal{A}) \le \varepsilon$; analogously for SPRP and PRF security.

The Advanced Encryption Standard. AES (the Rijndael cipher of Daemen and Rijmen, standardised in FIPS 197) is the canonical block cipher. It has block length $b = 128$ bits and three key lengths $\kappa \in \{128, 192, 256\}$, with $N_r \in \{10, 12, 14\}$ rounds respectively. Structurally it is a *substitution–permutation network* (SPN): the 128-bit state, viewed as a 4×4 array of bytes (elements of the field $\mathbb{F}_{2^8}$), passes through N_r rounds, each composing a nonlinear byte substitution (**SubBytes**, built on field inversion $a \mapsto a^{-1}$, chosen for its resistance to differential and linear cryptanalysis), a byte transposition (**ShiftRows**), an invertible column-wise linear mix (**MixColumns**), and the XOR of a 128-bit round key (**AddRoundKey**) that a key schedule expands from k. The two permutation layers together guarantee that after two rounds every output byte depends on every input byte. Each layer is individually invertible, so E_k is a permutation as Definition 4.9 requires, and decryption applies the inverses in reverse order.

Remark 4.11 (Confusion and diffusion). The SPN design is the modern embodiment of Shannon's two principles. The nonlinear S-box supplies *confusion* — making the relationship between key and ciphertext as complex as possible. ShiftRows and MixColumns supply *diffusion* — spreading the influence of each input bit over the whole output. Iterating these for ten or more rounds is what gives AES its empirical resistance to all known attacks. No proof that AES is a PRP exists; like every practical block cipher, its security is a *conjecture* corroborated by decades of failed cryptanalysis. The theorems below therefore take "E is a (P)PRP/SPRP" as an assumption and reason from it.

4.4 PRFs from hash functions: length extension and keyed BLAKE2

Block ciphers are one source of PRFs; cryptographic hash functions are another, often more convenient one (hash functions are unkeyed and ubiquitous). The naive approach — keying a hash function H by prepending the key, $F_k(m) = H(k \,\|\, m)$ — is unsafe for the iterated "Merkle–Damgård" hashes (MD5, SHA-1, SHA-256) because of the *length-extension* attack: from $H(k \,\|\, m)$ and $|m|$ one can compute $H(k \,\|\, m \,\|\, \text{pad} \,\|\, m')$ for attacker-chosen m' *without knowing* k, since the hash output is exactly the internal chaining state after absorbing $k \,\|\, m$.

The classical repair is *HMAC*, the nested double call $\mathrm{HMAC}_k(m) = H\big((k \oplus \text{opad}) \,\|\, H((k \oplus \text{ipad}) \,\|\, m)\big)$ for two fixed pad constants: the inner hash absorbs the message under one key-derived value, and the outer hash re-keys the fixed-length result, so the emitted digest is no longer a resumable chaining state. Bellare's analysis shows that HMAC is a PRF whenever the underlying compression function is, up to a birthday term $q^2/2^\ell$; as a PRF it is in particular a secure message authentication code, and it is the building block of the standardised key-derivation function HKDF (§5.6). We need no more than this summary, because the shielded protocol uses HMAC nowhere: its hash of choice admits direct keying.

Keyed BLAKE2. Modern hash functions of the *sponge* or *HAIFA* families do not suffer length extension and so admit a far simpler keying. BLAKE2 (and its successor BLAKE3) is built so that it can be keyed natively: the construction loads the key as a parameter and prepends it as a padded first block, and the function's compression mixes a counter and finalisation flags that make the output depend on the whole input in a way that does not expose the internal state. One simply writes

$$F_k(m) \;=\; \mathrm{BLAKE2}\big(m;\ \text{key} = k\big),$$

and obtains a PRF directly — no nested double call is needed. No length-extension attack remains to repair, because the finalisation step (the flag XORed in for the last block) means the emitted digest is *not* the raw chaining value and cannot be used to resume the computation.

Remark 4.12 (Zcash's use of keyed/personalised BLAKE2). The Zcash protocol uses BLAKE2b and BLAKE2s pervasively, exploiting two parameters: the *key* (for PRF/MAC use) and the *personalisation* string, a domain-separation tag baked into the initial state (16 bytes for BLAKE2b, 8 bytes for BLAKE2s). Personalisation ensures that the same input hashed for two different purposes (say, deriving a nullifier versus deriving a note commitment) yields independent-looking outputs, so each use

site is effectively an independent PRF. This is the practical realisation of the "independent random oracle per context" idiom and is cheaper and cleaner than HMAC because BLAKE2's design already provides keying and domain separation as first-class features.

4.5 Format-preserving encryption and FF1

All constructions so far operate on bit strings of a length the cipher's block size dictates. Many applications, however, must encrypt data drawn from a peculiar finite domain — a 16-digit credit-card number, a 9-digit national identifier, a license plate — and require the ciphertext to lie in the *same* domain, so that legacy databases, field-length constraints, and checksum formats continue to accept it. This is the problem that *format-preserving encryption* (FPE) solves.

Definition 4.13 (Format-preserving encryption)**.** Let $\mathcal{X}$ be a finite set (the *format*, e.g. $\mathcal{X} = \{0,1,\dots,9\}^{16}$, the set of 16-digit strings). A *format-preserving encryption scheme* on $\mathcal{X}$ is a keyed permutation $\mathcal{E} : \{0,1\}^{\kappa} \times \mathcal{X} \to \mathcal{X}$ such that for every key k, $\mathcal{E}_k : \mathcal{X} \to \mathcal{X}$ is a bijection with efficiently computable inverse $\mathcal{D}_k$. Security requires $\mathcal{E}$ to be a (strong) pseudorandom permutation on $\mathcal{X}$ (Definition 4.6, with $\mathsf{Perm}(\mathcal{X})$ in place of $\mathsf{Perm}(\{0,1\}^{\ell})$): no efficient adversary distinguishes $\mathcal{E}_k$ from a uniformly random permutation of $\mathcal{X}$, even with access to the inverse.

The defining feature is that the domain $\mathcal{X}$ is *arbitrary* — in particular $|\mathcal{X}|$ need not be a power of two, so a plain block cipher does not apply. What we require is a keyed pseudorandom *bijection* on a set whose size is, say, 10^{16}. The deterministic nature is essential and distinguishes FPE from randomised block-cipher modes: $\mathcal{X}$ has no room to store a random IV, so FPE is the deterministic, length-preserving (indeed format-preserving) analogue of a block cipher on the exotic alphabet of $\mathcal{X}$. It necessarily leaks equality of plaintexts, as any deterministic cipher must, but this is the unavoidable cost of preserving format with no expansion, and it is acceptable for the tokenisation use cases FPE targets.

Reduction to integers. Any format $\mathcal{X}$ of the form "strings of length v over an alphabet of radix ρ" is in bijection with the integer interval $\{0,1,\dots,\rho^v - 1\}$ by reading the string as a base-ρ numeral. It therefore suffices to build a keyed pseudorandom permutation of $\mathbb{Z}/M\mathbb{Z} = \{0,\dots,M-1\}$ for an arbitrary modulus M (here $M = \rho^v$). FF1 splits $M = a \cdot b$ into two factors of nearly equal "size" and runs a Feistel network whose two halves live in $\mathbb{Z}/a\mathbb{Z}$ and $\mathbb{Z}/b\mathbb{Z}$.

The Feistel network. A Feistel network turns a (not necessarily invertible) round function into a permutation. We present the classical bitwise case first, then the modular generalisation that FF1 uses.

Definition 4.14 (Balanced Feistel network)**.** Let $F_1,\dots,F_r : \{0,1\}^w \to \{0,1\}^w$ be arbitrary *round functions*. The r-round Feistel network $\Psi[F_1,\dots,F_r]$ is the permutation of $\{0,1\}^{2w}$ defined as follows. Write the input as (L_0, R_0) with $L_0, R_0 \in \{0,1\}^w$. For $i = 1,\dots,r$ set

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus F_i(R_{i-1}),$$

and output (L_r, R_r).

Proposition 4.15 (Feistel is a bijection regardless of the round functions)**.** *For any choice of round functions $F_1,\dots,F_r$ (which need not be injective), $\Psi[F_1,\dots,F_r]$ is a bijection of $\{0,1\}^{2w}$, and one com-*

putes its inverse by running the rounds backwards:

$$R_{i-1} = L_i, \qquad L_{i-1} = R_i \oplus F_i(L_i), \qquad i = r, \dots, 1.$$

Proof. It suffices to show each round is invertible; the composition of bijections is a bijection. The i-th round map sends $(L_{i-1}, R_{i-1}) \mapsto (R_{i-1},\ L_{i-1} \oplus F_i(R_{i-1}))$. Given the output (L_i, R_i), one recovers $R_{i-1} = L_i$ immediately, and then $L_{i-1} = R_i \oplus F_i(R_{i-1}) = R_i \oplus F_i(L_i)$, using that XOR is its own inverse. The recovery uses only forward evaluations of F_i, so it *never requires an inverse of the round function* — this is the crucial point that permits building a permutation out of a one-way-ish PRF. The displayed backward recursion is exactly this inversion applied round by round. □

This proposition is the structural heart of FPE: it manufactures a *keyed bijection* from a keyed function that is not itself a bijection. The round functions derive from a PRF (in FF1, AES iterated in cipher-block-chaining fashion; the FF1 description below gives the construction), so the network is invertible without ever inverting the PRF — the construction only ever evaluates AES in the forward direction.

The Luby–Rackoff theorem explains why enough Feistel rounds over a PRF yield a pseudorandom permutation, justifying the construction's security.

Theorem 4.16 (Luby–Rackoff)**.** *If the round functions $F_1, \dots, F_r$ are independent PRFs on $\{0,1\}^w$, then the Feistel network Ψ is, against an adversary making q queries:*

- *with $r = 3$ rounds, a pseudorandom permutation (CPA-secure: forward queries only), and*
- *with $r = 4$ rounds, a* strong *pseudorandom permutation (CCA-secure: forward and inverse queries),*

with distinguishing advantage $O(q^2/2^w)$ above the PRF advantage of the round functions.

Proof. Sketch. Replace each PRF round function by a truly random function, at the cost of r PRF terms. For the three-round case one shows that, unless two of the at most q queries induce a collision in the intermediate Feistel values (a birthday event of probability $O(q^2/2^w)$), the outputs of the network are distributed exactly as those of a random permutation: the middle round's random function, evaluated at inputs that are distinct with high probability, produces independent uniform values that perfectly mask the relationship between input and output halves. The fourth round adds the symmetry needed to also resist inverse queries, upgrading PRP to SPRP. The full argument is a careful coupling/transcript analysis bounding the probability of any internal collision; the leftover term is the stated birthday bound. □

Remark 4.17 (FF1's use of ten rounds rather than four). Luby–Rackoff gives security only up to $q \approx 2^{w/2}$ queries, and for small domains (the FPE setting, where w is tiny — consider one half of a 16-digit number) the birthday bound $q^2/2^w$ is weak. FF1 therefore uses *ten* rounds rather than the minimal four, providing a comfortable security margin against the more powerful attacks known for small-domain Feistel (which can sometimes beat the generic birthday bound). The number of rounds buys security; it does not affect invertibility, which Proposition 4.15 guarantees for any round count.

The FF1 mode (NIST SP 800-38G). FF1 instantiates an *unbalanced, modular* Feistel network of ten rounds to build a PRP on $\mathbb{Z}/M\mathbb{Z}$ with $M = \rho^\nu$, the radix-ρ numeral interpretation of a length-ν string. It supports an associated *tweak* T — a non-secret string that selects, in effect, one permutation from a family indexed by T, so that the same key encrypts the same plaintext to different ciphertexts under different tweaks (useful for binding a token to a context). The following describes its operation.

1. **Split.** Set $u = \lfloor \nu/2 \rfloor$, $v = \nu - u$. Split the plaintext numeral string X into a left part A of u symbols and a right part B of v symbols, with integer values in $\{0, \dots, \rho^u - 1\}$ and $\{0, \dots, \rho^v - 1\}$ respectively. (Unbalanced when ν is odd; the two halves live in moduli $a = \rho^u$ and $b = \rho^v$.)

2. **Round function from a PRF.** For round $i = 0, 1, \ldots, 9$, assemble a byte string Q encoding the tweak T, the round index i, and the current right half B, prepend a fixed parameter block P (encoding the radix, lengths, and tweak length), and apply $\mathrm{PRF}(P \parallel Q)$, where PRF is AES in CBC-MAC mode: set $Y_0 = 0^{128}$, iterate $Y_j = \mathrm{AES}_k(Y_{j-1} \oplus X_j)$ over the 16-byte blocks X_j of $P \parallel Q$, and output the final block Y_m (SP 800-38G, Algorithm 6). The iteration chains AES through an evolving state exactly as the Merkle–Damgård iteration of §4.4 chains a compression function; emitting the final state is safe here because all inputs share one fixed length, so no length-extension query arises. Further AES calls then expand the result to a long enough digest, which we read as an integer y.

3. **Modular Feistel step.** Update the halves by the modular analogue of Definition 4.14:

$$C = \big(A + y\big) \bmod m, \qquad A \leftarrow B, \qquad B \leftarrow C,$$

   where the modulus m alternates between $a = \rho^u$ and $b = \rho^v$ according to the parity of the round (so that the addition lands in the correct half's domain). Here modular addition replaces XOR.

4. **Output.** After ten rounds, concatenate the final A and B and render back to a length-ν radix-ρ string.

Decryption runs the rounds in reverse, replacing modular addition by modular subtraction $A \leftarrow (C - y) \bmod m$, exactly mirroring the backward recursion of Proposition 4.15.

Proposition 4.18 (FF1 is a keyed bijection on $\mathcal{X}$)**.** *For every key k and tweak T, the FF1 map $\mathcal{E}_{k,T} : \mathcal{X} \to \mathcal{X}$ is a bijection of the format set $\mathcal{X}$, and $\mathcal{D}_{k,T}$ is its two-sided inverse.*

Proof. Each round is the map $(A, B) \mapsto (B, (A + y) \bmod m)$, where $y = y(i, T, B)$ depends only on the round index, the tweak, and the right half B — *not* on A. Fix the inputs to the round function (i.e. fix B, i, T); then y is a fixed constant and the map $A \mapsto (A + y) \bmod m$ is a bijection of $\mathbb{Z}/m\mathbb{Z}$ (translation by a constant in a cyclic group is a bijection, since gcd-free addition is invertible by subtracting the same constant). The full round is thus invertible: from (B, C) recover B directly as the new left value's source, recompute $y = y(i, T, B)$ by a *forward* PRF evaluation, and set $A = (C - y) \bmod m$. Each even-indexed round is a bijection of $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ onto $\mathbb{Z}/b\mathbb{Z} \times \mathbb{Z}/a\mathbb{Z}$ and each odd-indexed round a bijection back onto $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ (the two products coincide when ν is even); the composition of the ten rounds, an even number, is therefore a bijection of $\mathcal{X}' = \mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$. The numeral map $(A, B) \mapsto Ab + B$ identifies $\mathcal{X}'$ with $\mathbb{Z}/M\mathbb{Z}$ as a bijection of sets (not of groups: $\gcd(a, b) = \rho^{\min(u,v)} > 1$, so the Chinese remainder theorem does not apply; only bijectivity is used), and $\mathcal{E}_{k,T}$ is a bijection of $\mathbb{Z}/M\mathbb{Z}$. Transporting through the numeral bijection $\mathcal{X} \cong \mathbb{Z}/M\mathbb{Z}$ (which is itself a bijection), $\mathcal{E}_{k,T}$ is a bijection of $\mathcal{X}$. Running the rounds backward with subtraction inverts it on both sides, so $\mathcal{D}_{k,T} = \mathcal{E}_{k,T}^{-1}$. □

Remark 4.19 ("Exactly a keyed bijection", and the locus of security). Two points complete the picture. First, FF1 is a keyed bijection by *structure*: invertibility comes for free from the Feistel construction (Proposition 4.15 and 4.18) and holds for *any* round function whatsoever, including a broken or constant one. The PRF (AES) is responsible not for invertibility but for *pseudorandomness* — making $\mathcal{E}_{k,T}$ look like a uniformly random permutation of $\mathcal{X}$ (Definition 4.13), per the Luby–Rackoff heuristic of Theorem 4.16 adapted to the modular, multi-round, tweaked setting. This clean separation — bijectivity from algebra, security from the PRF — is precisely why Feistel is the standard route to FPE.

Second, the domain-size caveat is real: the published SP 800-38G requires $\rho^\nu \geq 100$ and recommends $\rho^\nu \geq 10^6$ (its draft Revision 1 would make 10^6 mandatory), because for very small domains an attacker can recover or distinguish the keyed permutation by exhaustively cataloguing input/output pairs, and known message-recovery attacks on small-domain FPE (Bellare–Hoang–Tessaro and successors) erode the security margin. For the moderately large domains FPE is meant for (card numbers,

identifiers), and with ten rounds, FF1 is the NIST-standardised, AES-backed realisation of a keyed pseudorandom bijection on an arbitrary finite format. Its single use in Orchard is the derivation of *diversifiers*: FF1-AES256 maps an address index to the 11-byte diversifier embedded in a shielded address, so that one spending key yields many unlinkable addresses (*Ironwood Guide*).

5 Symmetric encryption, AEAD, and key derivation

This section develops symmetric-key cryptography from its information-theoretic baseline to the concrete authenticated-encryption scheme used throughout the Zcash protocol. We first recall the one-time pad and the eavesdropper (EAV) security notion that relaxes its perfect secrecy to computational adversaries. We then instantiate the resulting pseudo-one-time pad by the nonce-based stream cipher ChaCha20 and formalise chosen-plaintext security. Encryption alone guarantees confidentiality but not integrity; we therefore develop message authentication codes, the universal-hash MAC Poly1305, and the notion of authenticated encryption with associated data (AEAD), culminating in the ChaCha20-Poly1305 construction. The section concludes with key-derivation functions, which manufacture uniform keys from non-uniform secrets such as Diffie–Hellman shared values: we state the general notion, note the standardised HKDF in passing, and present the single personalised BLAKE2b call by which Zcash actually derives its note-encryption keys.

Throughout, an *adversary* is a (possibly probabilistic) algorithm $\mathcal{A}$, and $y \leftarrow \mathcal{A}(x)$ denotes running $\mathcal{A}$ on input x and assigning its output to y. For a finite set S, $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from S. Every adversary in our computational definitions is *probabilistic polynomial time* (PPT) in a security parameter λ, and $\mathrm{negl}(\lambda)$ denotes a negligible function (Definition 1.11).

The one-time pad and EAV security. The information-theoretic baseline is the *one-time pad*: to encrypt $m \in \{0,1\}^L$ under a uniformly random key $k \in \{0,1\}^L$ used exactly once, output $c = m \oplus k$. Since $m \oplus k$ is uniform whatever m is, the ciphertext distribution is independent of the message, and even an unbounded eavesdropper learns nothing—Shannon's *perfect secrecy*. The price is severe: the key must be as long as the message and must never be reused, for two ciphertexts under one pad reveal $c \oplus c' = m \oplus m'$, the *two-time-pad* failure. The computational relaxation is *EAV security* (security against an eavesdropper): an adversary who chooses two equal-length messages m_0, m_1 and sees a single ciphertext $\mathrm{Enc}_k(m_b)$ for a hidden uniform bit b guesses b with advantage at most $\mathrm{negl}(\lambda)$. Replacing the truly random pad by the output of a pseudorandom generator stretched from a short seed—the *pseudo-one-time pad*—achieves EAV security with a fixed-length key, at the cost of resting on a pseudorandomness assumption.

5.1 Stream ciphers and ChaCha20

A *stream cipher* is the practical instantiation of the pseudo-one-time pad: it is a keyed, seekable PRG producing a *keystream* that XORs into the plaintext. To support encrypting many messages under one key without violating the one-time-pad no-reuse rule, modern stream ciphers take an additional public input called a *nonce* (number used once) and behave like a pseudorandom function of the nonce.

> **Definition 5.1** (Nonce-based stream cipher)**.** A *nonce-based stream cipher* is a deterministic function $\mathrm{KS} : \mathcal{K}\times\mathcal{N}\times\mathbb{N} \to \{0,1\}^*$ producing, for key k, nonce ν, and length L, a keystream $\mathrm{KS}(k,\nu,L) \in \{0,1\}^L$. Encryption of $m \in \{0,1\}^L$ is $\mathrm{Enc}_k(\nu,m) = m \oplus \mathrm{KS}(k,\nu,|m|)$ with ciphertext (ν, c); decryption recomputes the keystream and XORs. The security goal is that for distinct nonces the keystreams are jointly indistinguishable from independent uniform strings, provided no nonce repeats under a fixed key.

ChaCha20, designed by Bernstein, is the stream cipher that Zcash uses (in the note-encryption layer) and that RFC 8439 standardises. Its core is the *ChaCha quarter-round*, a reversible mixing of four 32-bit words built only from modular addition, XOR, and rotation (an "ARX" design), which avoids data-dependent table lookups and hence timing side channels.

Definition 5.2 (ChaCha state and quarter-round)**.** The ChaCha20 state is a 4×4 matrix of 32-bit words, viewed as a vector $(x_0, \ldots, x_{15}) \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}$. Arithmetic $+$ is addition modulo 2^{32}, $\oplus$ is XOR, and $x \lll r$ is left rotation by r bits. The *quarter-round* $\mathrm{QR}(a, b, c, d)$ updates four words by

$$\begin{aligned} a \mathrel{+}= b;\quad d \mathrel{\oplus}= a;\quad d \lll= 16;\\ c \mathrel{+}= d;\quad b \mathrel{\oplus}= c;\quad b \lll= 12;\\ a \mathrel{+}= b;\quad d \mathrel{\oplus}= a;\quad d \lll= 8;\\ c \mathrel{+}= d;\quad b \mathrel{\oplus}= c;\quad b \lll= 7. \end{aligned}$$

The cipher initialises the state from the 128-bit constant "`expand 32-byte k`" (words $x_0 - x_3$), a 256-bit key ($x_4 - x_{11}$), a 32-bit block counter (x_{12}), and a 96-bit nonce ($x_{13} - x_{15}$). One *double round* applies QR to the four columns and then to the four diagonals; the 20 rounds of ChaCha20 are ten double rounds. Writing X for the initial state and $R(X)$ for the state after the rounds, the 64-byte keystream block is the *feed-forward* sum $R(X) + X$ (word-wise, modulo 2^{32}), serialised in little-endian order.

Remark 5.3 (Feed-forward and counter mode)*.* The rounds R form a bijection (each quarter-round is invertible), so without the final addition $R(X) + X$ the block function would be a permutation and thus let the adversary invert it trivially; the feed-forward sum destroys invertibility and is the standard "Davies–Meyer"-style construction for turning a permutation into a one-way mixing function. Incrementing the block counter x_{12} produces successive 64-byte keystream blocks, so $\mathrm{KS}(k, \nu, L)$ is the concatenation of blocks for counters $0, 1, 2, \ldots$; this *counter mode* is what makes the cipher seekable and parallelisable. We conjecture ChaCha20 to be a secure PRF of $(\nu, \text{counter})$: no attack better than brute force over the 256-bit key is known. Reusing a (k, ν) pair reproduces the same keystream and re-creates the two-time-pad failure; nonce uniqueness is a hard requirement.

5.2 Chosen-plaintext security

EAV-security models only an adversary who sees a single ciphertext. A realistic adversary may influence which messages get encrypted and observe many ciphertexts. The corresponding notion is *chosen-plaintext attack* (CPA) security, which we model by granting the adversary oracle access to the encryption algorithm.

Definition 5.4 (CPA security)**.** For scheme Π, adversary $\mathcal{A}$, and $b \in \{0, 1\}$, the experiment $\mathrm{Exp}^{\mathrm{cpa}}_{\Pi,\mathcal{A}}(\lambda, b)$ is: sample $k \leftarrow \mathrm{Gen}(1^\lambda)$; give $\mathcal{A}$ oracle access to $\mathrm{Enc}_k(\cdot)$; at some point $\mathcal{A}$ submits a challenge pair (m_0, m_1) of equal length and receives $c^* \leftarrow \mathrm{Enc}_k(m_b)$; $\mathcal{A}$ may continue querying the oracle and finally outputs a bit b', which is the output. Π is *CPA-secure* if for every PPT $\mathcal{A}$,

$$\mathrm{Adv}^{\mathrm{cpa}}_{\Pi,\mathcal{A}}(\lambda) := \left|\Pr[\mathrm{Exp}^{\mathrm{cpa}}_{\Pi,\mathcal{A}}(\lambda, 0) = 1] - \Pr[\mathrm{Exp}^{\mathrm{cpa}}_{\Pi,\mathcal{A}}(\lambda, 1) = 1]\right| \le \mathrm{negl}(\lambda).$$

Proposition 5.5 (Determinism precludes CPA security)**.** *No scheme with a deterministic, stateless* Enc *is CPA-secure.*

Proof. The adversary queries the oracle on two distinct messages $m_0 \neq m_1$, recording $c_0 = \mathrm{Enc}_k(m_0)$ and $c_1 = \mathrm{Enc}_k(m_1)$. It submits the challenge (m_0, m_1), receives $c^* = \mathrm{Enc}_k(m_b)$, and outputs 0 if $c^* = c_0$ and 1 otherwise. Since Enc is deterministic, $c^* = c_b$ always, so $\mathcal{A}$ is always correct and its advantage is 1. □

Consequently, CPA-secure schemes must be randomised or nonce-based. A nonce-based stream cipher that uses a fresh (e.g. random or counter) nonce per encryption attains CPA security under the

PRF conjecture for the keystream generator, because each encryption is effectively a fresh one-time pad. We record here the security notion toward which the development proceeds; we defer the routine PRF-based proof to the AEAD analysis below, where we treat confidentiality and integrity together.

Remark 5.6 (Semantic security). Goldwasser and Micali's original notion, *semantic security*, demands that whatever an efficient adversary can compute about the plaintext from the ciphertext, it could compute without the ciphertext (from the message length alone). Semantic security is provably equivalent to the indistinguishability notions above; we use indistinguishability throughout because it is far easier to manipulate in reductions. The content is the same: the ciphertext is computationally useless for learning anything about the message beyond its length.

5.3 Message authentication codes

CPA security protects *confidentiality* but says nothing about *integrity*: an adversary who tampers with a ciphertext may produce a different valid plaintext without detection. Moreover, for the malleable XOR-based stream ciphers above, flipping a bit of the ciphertext flips the corresponding plaintext bit. To detect tampering, we attach a *message authentication code*.

Definition 5.7 (Message authentication code)**.** A *message authentication code* (MAC) is a pair $(\mathrm{Mac}, \mathrm{Vrfy})$ with key space $\mathcal{K}$: $\mathrm{Mac}_k(m)$ outputs a tag t, and $\mathrm{Vrfy}_k(m,t) \in \{0,1\}$ accepts (1) or rejects (0). Correctness requires $\mathrm{Vrfy}_k(m, \mathrm{Mac}_k(m)) = 1$ for all k, m. When Mac is deterministic, canonical verification recomputes the tag and checks equality.

Definition 5.8 (Existential unforgeability, EUF-CMA)**.** Consider the experiment $\mathrm{Exp}^{\mathrm{forge}}_{\Pi,\mathcal{A}}(\lambda)$: sample $k \leftarrow \mathrm{Gen}(1^\lambda)$; give $\mathcal{A}$ oracle access to $\mathrm{Mac}_k(\cdot)$; let Q be the set of messages $\mathcal{A}$ queries; $\mathcal{A}$ outputs a pair (m^*, t^*) and wins iff $\mathrm{Vrfy}_k(m^*, t^*) = 1$ and $m^* \notin Q$. The MAC is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every PPT $\mathcal{A}$,

$$\mathrm{Adv}^{\mathrm{forge}}_{\Pi,\mathcal{A}}(\lambda) := \Pr[\mathcal{A} \text{ wins}] \le \mathrm{negl}(\lambda).$$

A *strongly* unforgeable (sUF-CMA) MAC additionally forbids producing a new valid tag for an already-queried message: $\mathcal{A}$ wins on (m^*, t^*) as long as the oracle never returned that exact pair; write $\mathrm{Adv}^{\text{s-forge}}_{\Pi,\mathcal{A}}$ for the advantage in this variant.

The MAC that Zcash uses, Poly1305, is not a pseudorandom-function MAC but a *one-time*, *universal-hash* MAC, which provides information-theoretic security per nonce and is extremely fast. The key abstraction is a universal hash family.

Definition 5.9 (ε-almost-Δ-universal hash family)**.** Let $(\mathcal{G}, +)$ be a finite abelian group. A family of functions $\{h_r : \mathcal{X} \to \mathcal{G}\}_{r \in \mathcal{R}}$ is *ε-almost-Δ-universal* (ε-AΔU) if for all distinct $x, x' \in \mathcal{X}$ and every $\delta \in \mathcal{G}$,

$$\Pr_{r \xleftarrow{\$} \mathcal{R}} [\, h_r(x) - h_r(x') = \delta \,] \le \varepsilon.$$

Taking $\delta = 0$ recovers ordinary ε-universality (low collision probability); the Δ-universality condition controls additive differences, which is what renders the one-time MAC below secure. With $\mathcal{G} = \{0,1\}^n$ under XOR this is the classical almost-XOR-universality; Poly1305 below takes $\mathcal{G} = \mathbb{Z}/2^{128}\mathbb{Z}$, addition modulo 2^{128}.

The Carter–Wegman paradigm turns any AΔU family into a one-time MAC by masking the hash with a fresh pad.

Theorem 5.10 (Carter–Wegman one-time MAC). *Let $\{h_r\}$ be ε-AΔU into $(\mathcal{G}, +)$. Define a MAC with key (r, s), where $r \xleftarrow{\$} \mathcal{R}$ and the pad $s \xleftarrow{\$} \mathcal{G}$, by $\mathrm{Mac}_{r,s}(m) = h_r(m) + s$. If (r, s) authenticates at most one message, then any (even computationally unbounded) adversary who sees one message–tag pair (m, t) forges a valid pair (m^*, t^*) with $m^* \neq m$ with probability at most ε.*

Proof. The adversary observes $t = h_r(m)+s$ and must output (m^*, t^*) with $m^* \neq m$ and $t^* = h_r(m^*)+s$. Subtracting the two tag equations eliminates the pad:

$$t^* - t = h_r(m^*) - h_r(m).$$

Thus the forgery succeeds iff $h_r(m^*) - h_r(m) = t^* - t$. Condition on the adversary's view. The pad s is uniform and independent of r; given the single observed tag t, every value of $h_r(m)$ is equally consistent (each determines a unique s), so the observation t leaks nothing about r, and the posterior on r remains uniform on $\mathcal{R}$. The adversary fixes its chosen $\delta := t^* - t$ and the distinct pair (m, m^*) independently of r; hence by the AΔU property

$$\Pr[h_r(m^*) - h_r(m) = \delta] \leq \varepsilon. \qquad \square$$

Poly1305 instantiates this with a polynomial-evaluation hash over the prime field $\mathbb{F}_p$ for $p = 2^{130} - 5$.

Definition 5.11 (Poly1305). Fix the prime $p = 2^{130} - 5$ and work in $\mathbb{F}_p$. The key is a pair (r, s): r is a 128-bit value subjected to a fixed bit-*clamping* (certain bits are forced to zero) and s is a 128-bit pad. To authenticate a message, split it into ℓ blocks $c_1, \ldots, c_\ell$ of at most 16 bytes; encode each block as an integer in $[0, 2^{128})$ and add 2^{8b} where b is the block's byte length (this appends a leading 1 bit, making the encoding prefix-free). Interpreting r and each c_i as elements of $\mathbb{F}_p$, the hash is the polynomial

$$h_r(m) = \sum_{i=1}^{\ell} c_i \, r^{\ell - i + 1} \in \mathbb{F}_p,$$

i.e. Horner evaluation $\big(\ldots((c_1 r + c_2)r + c_3)r \cdots\big)r$. The tag is

$$\mathrm{Mac}_{r,s}(m) = \big((h_r(m) \bmod p) + s\big) \bmod 2^{128},$$

the field hash reduced and added to the pad modulo 2^{128}.

Proposition 5.12 (Poly1305 is almost-Δ-universal). *Restricted to messages of at most L bytes (hence at most $\ell_{\max} = \lceil L/16 \rceil$ blocks), the reduced Poly1305 hash family $\{m \mapsto h_r(m) \bmod p\}$, read into $\mathbb{Z}/2^{128}\mathbb{Z}$, is ε-almost-Δ-universal with respect to addition modulo 2^{128}, with $\varepsilon \leq 8\ell_{\max}/2^{106}$; the loss from the ideal $\ell_{\max} \cdot p^{-1}$ accounts for the 128-bit clamping of r (which restricts r to a subset of size 2^{106}) and the final reduction modulo 2^{128}.*

Proof. Sketch. Fix two distinct messages m, m' with block encodings (c_i) and (c'_j), padded to a common length ℓ by leading zero-coefficients without changing their values (the prefix-free encoding guarantees that $m \neq m'$ yields distinct coefficient vectors). The difference of hashes is

$$h_r(m) - h_r(m') = \sum_{i=1}^{\ell} (c_i - c'_i)\, r^{\ell - i + 1} =: P(r) \in \mathbb{F}_p,$$

a nonzero polynomial in r of degree at most ℓ (nonzero because the coefficient vectors differ and the encoding is injective). For any target difference $\delta \in \mathbb{F}_p$, the equation $P(r) = \delta$ is a polynomial

equation of degree $\leq \ell$ over the field $\mathbb{F}_p$, so it has at most ℓ roots r. Since r is (clamped) uniform over a set of 2^{106} values, a uniformly chosen r is a root with probability at most $\ell/2^{106}$. For a target difference δ taken modulo 2^{128}, each residue class modulo 2^{128} corresponds to at most eight field differences (since $p = 2^{130} - 5$), multiplying the root-counting bound by a small constant; this is the AΔU property with $\varepsilon \leq 8\ell_{\max}/2^{106}$. □

Combining Proposition 5.12 with the Carter–Wegman Theorem 5.10 shows that Poly1305, with a *fresh, unpredictable pad s per message*, is a one-time MAC with forgery probability at most $8\ell_{\max}/2^{106}$ per verification attempt. In the AEAD scheme below, ChaCha20 itself generates the one-time key (r, s) pseudorandomly from the nonce, so the per-nonce one-time guarantee suffices.

5.4 Authenticated encryption with associated data

We now combine confidentiality and integrity. The appropriate composite goal is *authenticated encryption*: the adversary should be unable both to learn anything about plaintexts (CPA security) and to produce any new ciphertext that decrypts to a non-$\perp$ value (ciphertext integrity). Real protocols also require ciphertexts to bind to public context (packet headers, version numbers) that is sent in the clear but must be authenticated; this is the *associated data*.

Definition 5.13 (AEAD scheme)**.** An *authenticated encryption with associated data* (AEAD) scheme is a pair $(\mathrm{Enc}, \mathrm{Dec})$ with key space $\mathcal{K}$, nonce space $\mathcal{N}$, and associated-data space $\mathcal{D}$: $\mathrm{Enc}_k(\nu, a, m)$ returns a ciphertext c, and $\mathrm{Dec}_k(\nu, a, c)$ returns a message in $\mathcal{M}$ or $\perp$. Correctness: $\mathrm{Dec}_k(\nu, a, \mathrm{Enc}_k(\nu, a, m)) = m$ for all k, ν, a, m. The scheme authenticates but does not encrypt the associated data a.

Definition 5.14 (Ciphertext integrity, INT-CTXT)**.** In experiment $\mathrm{Exp}^{\mathrm{ctxt}}_{\Pi,\mathcal{A}}(\lambda)$: sample $k \leftarrow \mathrm{Gen}(1^\lambda)$; give $\mathcal{A}$ an encryption oracle $\mathrm{Enc}_k(\cdot,\cdot,\cdot)$, recording the set Q of returned triples (ν, a, c); $\mathcal{A}$ wins if it outputs $(\nu^*, a^*, c^*) \notin Q$ with $\mathrm{Dec}_k(\nu^*, a^*, c^*) \neq \perp$. The adversary must never repeat a nonce to its encryption oracle (the *nonce-respecting* model). Π has *ciphertext integrity* if $\Pr[\mathcal{A} \text{ wins}] \leq \mathrm{negl}(\lambda)$ for all PPT $\mathcal{A}$.

Definition 5.15 (Authenticated encryption)**.** A nonce-respecting AEAD scheme is *secure* (an AEAD) if it is both CPA-secure (Definition 5.4, with nonces and associated data included in the challenge) and has ciphertext integrity (Definition 5.14).

The standard way to build an AEAD from a CPA-secure cipher and a strongly unforgeable MAC is *encrypt-then-MAC*: encrypt the message, then compute a MAC over the ciphertext (together with the associated data and nonce), appending the tag. Decryption verifies the tag *before* decrypting, and any tag failure returns $\perp$. The contrasting orders (MAC-then-encrypt and encrypt-and-MAC) do not generically achieve authenticated encryption; encrypt-then-MAC does.

Theorem 5.16 (Encrypt-then-MAC yields authenticated encryption)**.** *Let $\Pi_E = (\mathrm{Enc}, \mathrm{Dec})$ be a CPA-secure encryption scheme and let $\Pi_M = (\mathrm{Mac}, \mathrm{Vrfy})$ be a strongly unforgeable (sUF-CMA) MAC, with independent keys k_E, k_M. Define the composite AEAD*

$$\mathrm{Enc}'_{k_E,k_M}(\nu, a, m) = (c, t), \quad c \leftarrow \mathrm{Enc}_{k_E}(\nu, m),\ t \leftarrow \mathrm{Mac}_{k_M}(\nu \| a \| c),$$

with Dec' returning $\perp$ unless $\mathrm{Vrfy}_{k_M}(\nu \| a \| c, t) = 1$, in which case it returns $\mathrm{Dec}_{k_E}(\nu, c)$. Then Π' is

a secure AEAD: it is CPA-secure and has ciphertext integrity.

Proof. Ciphertext integrity. Suppose a PPT $\mathcal{A}$ outputs a fresh $(\nu^*, a^*, (c^*, t^*)) \notin Q$ that decrypts successfully, i.e. $\mathrm{Vrfy}_{k_M}(\nu^*\|a^*\|c^*, t^*) = 1$. Each encryption-oracle call on (ν, a, m) returns (c, t) and corresponds to a MAC oracle query on the string $w = \nu\|a\|c$ returning tag t. Because the framing $\nu\|a\|c$ parses uniquely (lengths are fixed or prepended), the pair (ν^*, a^*, c^*) being fresh and yet verifying means the MAC oracle never produced the string–tag pair $(\nu^*\|a^*\|c^*,\ t^*)$: either $w^* := \nu^*\|a^*\|c^*$ was never queried, or it was queried but returned a tag $\neq t^*$. In both cases (w^*, t^*) is a fresh valid string–tag pair, a strong forgery against Π_M. Hence we build $\mathcal{B}$ that runs $\mathcal{A}$, answering encryption queries using its own MAC oracle and a self-chosen k_E, and outputs (w^*, t^*); $\mathcal{B}$ wins whenever $\mathcal{A}$ does, so $\Pr[\mathcal{A}\text{ wins}] \le \mathrm{Adv}^{\text{s-forge}}_{\Pi_M,\mathcal{B}}(\lambda) = \mathrm{negl}(\lambda)$.

CPA security. We argue indistinguishability of the challenge by a hybrid that replaces the genuine tag's coupling with the message. Consider the challenge (m_0, m_1); the ciphertext is (c, t) with $c \leftarrow \mathrm{Enc}_{k_E}(\nu, m_b)$ and $t = \mathrm{Mac}_{k_M}(\nu\|a\|c)$. The tag t is a deterministic function of c (and k_M, ν, a) and so carries no information about b beyond what c already carries. Formally, we reduce to the CPA security of Π_E: a distinguisher $\mathcal{B}'$ samples k_M itself, forwards $\mathcal{A}$'s encryption and challenge queries to its Π_E-oracle to obtain the c-components, and appends MACs it computes with k_M. Then $\mathcal{B}'$ perfectly simulates Π' to $\mathcal{A}$, and $\mathrm{Adv}^{\text{cpa}}_{\Pi',\mathcal{A}}(\lambda) = \mathrm{Adv}^{\text{cpa}}_{\Pi_E,\mathcal{B}'}(\lambda) = \mathrm{negl}(\lambda)$. □

Remark 5.17 (Integrity of ciphertexts and the choice of ordering). Encrypt-then-MAC permits the receiver to reject forged ciphertexts *without decrypting*, which is what blocks chosen-ciphertext and padding-oracle attacks: a rejected ciphertext never reaches the decryption logic, so it cannot leak. By contrast, MAC-then-encrypt verifies only after decrypting, exposing the decryption routine to attacker-chosen ciphertexts. Ciphertext integrity is also exactly the property that upgrades CPA security to CCA (chosen-ciphertext) security: an authenticated-encryption scheme is automatically CCA-secure, because the decryption oracle is useless to the adversary (every ciphertext it did not legitimately obtain decrypts to $\perp$ except with negligible probability).

5.5 The ChaCha20-Poly1305 AEAD

We now assemble the concrete scheme of RFC 8439, which Zcash uses. It is an encrypt-then-MAC composition of the ChaCha20 stream cipher with the Poly1305 one-time MAC, with the feature that the cipher derives *both* the key stream and the one-time MAC key from the single (k, ν) input.

Definition 5.18 (ChaCha20-Poly1305 AEAD)**.** Fix a 256-bit key k and a 96-bit nonce ν. Encryption of message m with associated data a proceeds as follows.

1. *One-time MAC key.* Run ChaCha20 with key k, nonce ν, and block counter 0; take the first 32 bytes of the keystream block as the Poly1305 one-time key (r, s) (clamp the first 16 bytes to form r; the next 16 are s). Discard the rest of this block.
2. *Encrypt.* Run ChaCha20 with key k, nonce ν, starting at block counter 1, to produce the keystream KS_1 (the keystream of Remark 5.3 taken from block counter 1 onward), and set $c = m \oplus \mathrm{KS}_1(k, \nu, |m|)$.
3. *Authenticate.* Form the Poly1305 input by concatenating: a padded with zeros to a 16-byte boundary, c padded with zeros to a 16-byte boundary, the 8-byte little-endian length of a, and the 8-byte little-endian length of c. Compute the tag $t = \mathrm{Poly1305}_{r,s}$(that input).

The ciphertext is (c, t). Decryption recomputes (r, s) and the tag from (k, ν, a, c), compares it to t in constant time, returns $\perp$ on mismatch, and otherwise outputs $m = c \oplus \mathrm{KS}_1(k, \nu, |c|)$.

Remark 5.19 (Length encoding prevents splicing). The trailing 8-byte lengths of a and c are essential. Without them, the zero-padding that aligns a and c to 16-byte boundaries would permit an adversary to shift bytes between the associated data and the ciphertext (or pad-extend one of them) while leaving the Poly1305 input unchanged, breaking the binding between a and c. Appending the explicit lengths makes the MAC input an injective encoding of (a, c), which is what the prefix-free requirement in the AΔU analysis (Proposition 5.12) demands.

Theorem 5.20 (Security of ChaCha20-Poly1305)**.** *Model ChaCha20 as a pseudorandom function $F_k(\nu, j)$ from (ν, j) (nonce and block counter) to 64-byte blocks. In the nonce-respecting model, ChaCha20-Poly1305 is a secure AEAD: any PPT adversary making q encryption queries and q_v verification (decryption) queries whose Poly1305 input (padded associated data, padded ciphertext, and the length block) comprises at most $\ell_{\max}$ sixteen-byte blocks, $\ell_{\max} = \lceil |a|/16 \rceil + \lceil |c|/16 \rceil + 1$, has CPA advantage at most $\mathrm{Adv}^{\mathrm{prf}}_F(\lambda)$ + (negligible) and forges (breaks ciphertext integrity) with probability at most*

$$\mathrm{Adv}^{\mathrm{prf}}_F(\lambda) + \frac{8\, q_v\, \ell_{\max}}{2^{106}}.$$

Proof. Sketch. Replace F_k by a truly random function Φ from (ν, j) to 64-byte blocks; this costs $\mathrm{Adv}^{\mathrm{prf}}_F(\lambda)$ by definition of the PRF, and we analyse the idealised scheme.

Confidentiality. Since the adversary is nonce-respecting, every encryption query uses a fresh ν. For a fresh nonce, the blocks $\Phi(\nu, 1), \Phi(\nu, 2), \ldots$ used as keystream are uniform and independent of everything the adversary has seen, so $c = m \oplus \mathrm{KS}$ is a genuine one-time pad and reveals nothing about m beyond its length; the tag is a deterministic function of c, a, ν and adds no information. Thus the idealised scheme has zero CPA advantage, and the real scheme has CPA advantage at most $\mathrm{Adv}^{\mathrm{prf}}_F(\lambda)$.

Integrity. For each nonce ν, the one-time key $(r, s) = \Phi(\nu, 0)[0\!:\!32]$ is uniform and independent across distinct nonces, and—because counter 0 is reserved for the MAC key while encryption uses counters ≥ 1—independent of the keystream that produced the ciphertexts. Hence per nonce the setting is exactly the Carter–Wegman one-time-MAC setting of Theorem 5.10 with the ε-AΔU Poly1305 hash of Proposition 5.12, $\varepsilon \leq 8\ell_{\max}/2^{106}$. Consider any single verification query (ν^*, a^*, c^*, t^*) that is fresh. If ν^* was never used for encryption, the pad s for ν^* is uniform and entirely unknown to the adversary, so the tag t^* is correct with probability 2^{-128}. If ν^* equals the nonce of some encryption query (the adversary may reuse a nonce in a *verification* query, though not in encryption), then the adversary has seen exactly one tag under (r, s), and the freshness of (a^*, c^*) means it is attempting a forgery on a new message under that one-time key; by Theorem 5.10 this succeeds with probability at most $\varepsilon \leq 8\ell_{\max}/2^{106}$. A union bound over the q_v verification queries gives forgery probability at most $8\, q_v\, \ell_{\max}/2^{106}$ in the idealised scheme, and adding back the PRF-switching cost yields the stated bound. □

Remark 5.21 (Single key, two roles). ChaCha20-Poly1305 reuses one 256-bit key for both encryption and authentication, seemingly violating the independent-keys hypothesis of the encrypt-then-MAC Theorem 5.16. Reserving *block counter* 0 for the Poly1305 key and counters ≥ 1 for the keystream is exactly what restores independence: modelling ChaCha20 as a PRF, the outputs at distinct counters are independent, so (r, s) is independent of the keystream, and the two-key analysis applies. This is a recurring design pattern—domain separation by a counter or label permits one master key to safely play several roles.

5.6 Key-derivation functions

The AEAD above assumes a uniformly random 256-bit key. In practice keys come from sources that are *high-entropy but not uniform*: a Diffie–Hellman shared secret g^{xy} is a group element, not a uniform bit string; a passphrase or a hardware noise source has min-entropy spread unevenly across its

bits. A *key-derivation function* (KDF) bridges this gap, turning a non-uniform secret into one or more uniformly random keys. The relevant entropy measure is min-entropy.

Definition 5.22 (Min-entropy)**.** The *min-entropy* of a random variable X is $H_\infty(X) = -\log_2 \max_x \Pr[X = x]$. Equivalently, the best chance of guessing X in one try is $2^{-H_\infty(X)}$. The *statistical distance* between distributions X, Y on a set S is $\Delta(X, Y) = \frac{1}{2}\sum_{u \in S} |\Pr[X = u] - \Pr[Y = u]|$; we say X is ϵ-close to uniform if $\Delta(X, U_S) \le \epsilon$.

Definition 5.23 (Key-derivation function and its security)**.** A *key-derivation function* is a function $\mathrm{KDF} : \mathcal{S} \times \mathcal{I} \times \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$ taking a source secret (sample of a random variable Σ), an optional non-secret *salt* slt, an *info*/context string ctx, and a desired output length L. Its security goal: whenever Σ has min-entropy at least γ (relative to the adversary's side information), the output $\mathrm{KDF}(\Sigma, \mathrm{slt}, \mathrm{ctx}, L)$ should be computationally indistinguishable from U_L, even given slt, ctx, and that side information.

The notion a KDF targets is that of a (computational) randomness extractor: a seeded function whose output looks uniform whenever its input has enough min-entropy.

Definition 5.24 (Computational extractor)**.** A keyed function $\mathrm{Ext} : \mathcal{S} \times \mathcal{X} \to \{0,1\}^n$ is a *strong* (γ, ϵ)*-computational extractor* if for every source Σ (with auxiliary information) of min-entropy at least γ, and for a uniformly random salt $\mathrm{slt} \xleftarrow{\$} \mathcal{S}$,

$$\big(\mathrm{slt}, \mathrm{Ext}_{\mathrm{slt}}(\Sigma)\big) \approx_c \big(\mathrm{slt}, U_n\big),$$

with distinguishing advantage at most ϵ; "strong" means the construction reveals the salt to the distinguisher.

Information theory can realise this notion unconditionally. The *leftover hash lemma* states that any 2^{-n}-universal family $\{g_r : \mathcal{X} \to \{0,1\}^n\}$ with a uniform public seed R satisfies $\Delta\big((R, g_R(X)), (R, U_n)\big) \le \frac{1}{2}\sqrt{2^{n-\gamma}}$ whenever $H_\infty(X) \ge \gamma$; thus investing $\gamma \ge n + 2\log_2(1/\epsilon)$ bits of min-entropy buys n bits that are ϵ-close to uniform. We state this without proof, since the deployed mechanism below rests on a random-oracle modelling instead.

HKDF, in passing. The standardised general-purpose KDF is Krawczyk's HKDF (RFC 5869), built from HMAC (§4.4) in an *extract-then-expand* architecture: a salted HMAC call first distils the non-uniform secret into a short uniform *pseudorandom key* (the extractor stage, justified along leftover-hash lines), and a counter-chained sequence of HMAC calls then stretches that key, under a context string, into as many output bytes as required (the expander stage, justified by HMAC's PRF security). HKDF is the right tool when one KDF must serve arbitrary sources and output lengths. Zcash does not use it—indeed the shielded protocol uses HMAC nowhere—because its requirements are far narrower and a single hash call meets them.

The Zcash KDF. Orchard derives the note-encryption key by one personalised BLAKE2b call:

$$K_{\mathrm{enc}} = \mathrm{KDF}(\mathsf{sharedSecret}, \mathsf{epk}) = \text{BLAKE2b-256}\big(\texttt{"Zcash\_OrchardKDF"};\ \mathsf{sharedSecret} \,\|\, \mathsf{epk}\big),$$

where sharedSecret is the Diffie–Hellman shared point of the key agreement in Section 6, epk is the sender's ephemeral public key, and the quoted string occupies BLAKE2b's personalisation field. The analysis models this personalised hash as a random oracle: by the domain-separation principle of

§3.6 (Proposition 3.19 and Example 3.20), the personalisation makes it an oracle independent of every other hash use in the protocol, and on an input containing the high-min-entropy, hard-to-guess sharedSecret its output is uniform and independent of the adversary's view—which is exactly the security goal of Definition 5.23. Folding epk into the input binds the derived key to this particular ephemeral, the role the context string plays in HKDF; no salt is needed because the random-oracle modelling already rules out source distributions tuned against the function.

Remark 5.25 (The role of the KDF in the larger protocol). In the Zcash note-encryption layer, a Diffie–Hellman exchange on the Pallas curve produces a shared group element, and the single BLAKE2b call above condenses that element into the symmetric key fed to ChaCha20-Poly1305. The two halves of this section thus meet exactly here: key derivation manufactures the uniform key that the AEAD's security proof (Theorem 5.20) takes as a hypothesis, and the chain from a non-uniform shared secret to an authenticated ciphertext is complete. Section 6 assembles this chain end to end as hybrid public-key encryption.

6 Key agreement

Every primitive constructed so far solves a problem *after* the parties already share a secret. A block cipher, a message-authentication code, and an authenticated encryption scheme each presuppose a key both endpoints already hold. The most basic question remains open: how do two parties who have never met, communicating over a channel that an adversary fully observes, agree on a secret the adversary does not learn? This is the problem of *key agreement*, whose first solution—the Diffie–Hellman protocol of 1976—inaugurated public-key cryptography. This section develops key agreement from the ground up: the bare protocol, the abstraction that captures it, the precise sense in which it is secure, its elliptic-curve incarnation, and the composition with symmetric encryption that yields the in-band public-key encryption used to deliver shielded notes.

Throughout, $\mathbb{G}$ denotes a finite cyclic group, written multiplicatively, with a fixed generator g and order $n = \mathrm{ord}(g) = |\mathbb{G}|$. We assume all parties know the order n. Two concrete instances serve as references: the multiplicative group $\mathbb{F}_p^\times$ of a prime field (or a large prime-order subgroup of it), and the group of $\mathbb{F}$-points of an elliptic curve. The protocol depends only on the group structure together with the hardness of certain computational problems, and not on the presentation of the group.

6.1 The Diffie–Hellman protocol

The protocol rests on a single algebraic asymmetry. In a cyclic group $\mathbb{G} = \langle g \rangle$, the map

$$\exp_g : \mathbb{Z}/n\mathbb{Z} \to \mathbb{G}, \qquad x \mapsto g^x$$

is a group isomorphism: it is cheap to evaluate (by repeated squaring, in $O(\log n)$ group operations) but, in the groups used here, believed infeasible to invert. Inverting it is the *discrete-logarithm problem* (DLOG) of Definition 2.2; as there, $\log_g h$ denotes the unique $x \in \mathbb{Z}/n\mathbb{Z}$ with $g^x = h$.

The Diffie–Hellman insight is that two parties can each contribute one exponent, exchange the corresponding group elements in the clear, and—by the commutativity of exponentiation—arrive at a common value an eavesdropper, seeing only the two transmitted elements, cannot compute. We describe the protocol between two parties, *Alice* and *Bob*.

Definition 6.1 (Diffie–Hellman key exchange)**.** Fix public parameters $(\mathbb{G}, g, n)$ with $\mathbb{G} = \langle g \rangle$ of order n. The protocol proceeds as follows.

1. Alice samples $a \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ uniformly, computes $A = g^a$, and sends A to Bob.
2. Bob samples $b \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ uniformly, computes $B = g^b$, and sends B to Alice.
3. Alice computes $k_A = B^a$; Bob computes $k_B = A^b$.

The value $k_A = k_B$ is the *shared secret*; the elements A and B are the parties' *public shares* (or *public keys*), and a, b their *private keys*.

Proposition 6.2 (Correctness)**.** *In the protocol of Definition 6.1, both parties compute the same group element, namely*

$$k_A = k_B = g^{ab}.$$

Proof. By the laws of exponents in an abelian group,

$$k_A = B^a = (g^b)^a = g^{ba} = g^{ab} = (g^a)^b = A^b = k_B,$$

where $ba = ab$ because integer multiplication is commutative and a group of order n takes its exponents modulo n. □

The shared secret g^{ab} is the foundational object of the section: a value determined by A and B, computable by anyone who knows *one* of the two private keys, but conjecturally not by anyone who knows only the two public shares. Subsection 6.4 names this conjecture.

Remark 6.3 (Why a generator, and why prime order). If $\mathbb{G}$ has small subgroups, an adversary can confine a victim's contribution to one of them and learn the shared secret modulo a small factor of n (a *small-subgroup* attack). For this reason we prefer $\mathbb{G}$ of *prime* order n, in which the only subgroups are $\{1\}$ and $\mathbb{G}$ itself, so every non-identity element is a generator and no nontrivial confinement is possible. When the natural group (such as $\mathbb{F}_p^\times$, of composite order $p-1$) lacks prime order, one works inside a prime-order subgroup, and a recipient validates incoming elements by checking they lie in that subgroup—for instance by verifying $B^n = 1$ and $B \neq 1$. The analogue of this check for elliptic curves appears in Subsection 6.6.

6.2 Static and ephemeral keys

Definition 6.1 is symmetric and interactive: both parties sample fresh randomness and both speak. In practice key pairs differ by their lifetime.

Definition 6.4 (Static and ephemeral keys)**.** We call a Diffie–Hellman key pair (x, g^x)

- *static* (or *long-term*) if its owner generates it once and reuses it across many key-agreement sessions, typically publishing it and binding it to an identity; and
- *ephemeral* if its owner generates it freshly for a single session and discards it immediately afterwards.

The two notions combine into a small taxonomy of protocols, each with its own security character.

- **Ephemeral–ephemeral (DHE).** Both parties use fresh ephemeral keys, as in Definition 6.1 read literally. Once the session ends and a, b are erased, no one—not even the parties themselves—can recover the shared secret g^{ab}. This is the defining feature of *forward secrecy*: an adversary who later compromises both parties and steals all stored long-term state still cannot decrypt recorded past sessions, because the only secrets that ever existed were the now-deleted ephemerals.
- **Static–ephemeral.** One party (say the recipient, Bob) holds a static key pair $(b, B = g^b)$ known to Alice in advance, while Alice contributes a fresh ephemeral $(a, A = g^a)$. The shared secret is again g^{ab}. Crucially this requires *no interaction from Bob at agreement time*: Alice computes g^{ab} from Bob's published B, attaches A to her message, and sends everything in a single non-interactive flow. Bob recovers $g^{ab} = A^b$ when he later reads the message. This is precisely the shape of the hybrid encryption of Subsection 6.7 and of shielded-note delivery: the sender is online once, the recipient need not be online at all, and there is no round trip.
- **Static–static.** Both keys are long-term. Then g^{ab} is a fixed function of the two identities and stays the *same in every session* between them. This is occasionally useful but offers no forward secrecy and, because the secret never varies, one must combine it with fresh per-session randomness (a nonce) before using it to key any symmetric scheme.

Remark 6.5 (Static keys make the protocol non-interactive). The static–ephemeral case is the conceptual bridge from *key exchange* (an interactive protocol) to *public-key encryption* (a one-shot algorithm). A static public key $B = g^b$ functions exactly as an encryption public key: anyone can derive a shared secret with its owner by sending a single ephemeral A. The remainder of this section formalises this view through the key-agreement abstraction and then builds encryption on top of it.

6.3 The key-agreement scheme abstraction

We now abstract away from the particular group and isolate the two operations a key-agreement scheme must provide: deriving a public key from a private one, and combining one's own private key with a counterparty's public key to produce a shared secret. Higher-level constructions program against this abstraction.

Definition 6.6 (Key-agreement scheme)**.** A *(non-interactive) key-agreement scheme* over a parameter set Π consists of three algorithms together with a private-key space $\mathcal{S}$, a public-key space $\mathcal{P}$, and a shared-secret space $\mathcal{K}$:

- $\mathsf{KeyGen}(\Pi) \to s$: a randomised algorithm outputting a private key $s \in \mathcal{S}$;
- $\mathsf{DerivePublic}(\Pi, s) \to p$: a deterministic algorithm mapping a private key $s \in \mathcal{S}$ to a public key $p \in \mathcal{P}$;
- $\mathsf{Agree}(\Pi, s, p') \to k$: a deterministic algorithm mapping a private key $s \in \mathcal{S}$ and a public key $p' \in \mathcal{P}$ to a shared secret $k \in \mathcal{K}$.

The scheme is *correct* if for all parameters Π and all private keys s, t in the support of $\mathsf{KeyGen}(\Pi)$,

$$\mathsf{Agree}\big(\Pi,\ s,\ \mathsf{DerivePublic}(\Pi, t)\big) \ =\ \mathsf{Agree}\big(\Pi,\ t,\ \mathsf{DerivePublic}(\Pi, s)\big). \tag{1}$$

That is, the two parties holding s and t compute the same shared secret from the other's public key.

Example 6.7 (Diffie–Hellman as a key-agreement scheme)**.** Fix $\Pi = (\mathbb{G}, g, n)$ with $\mathbb{G} = \langle g \rangle$ of prime order n. Set $\mathcal{S} = \mathbb{Z}/n\mathbb{Z}$ (or, to avoid the degenerate identity-only case, $\mathcal{S} = (\mathbb{Z}/n\mathbb{Z}) \setminus \{0\}$), $\mathcal{P} = \mathcal{K} = \mathbb{G}$, and define

$$\mathsf{KeyGen}(\Pi) = a \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}, \qquad \mathsf{DerivePublic}(\Pi, a) = g^a, \qquad \mathsf{Agree}(\Pi, a, B) = B^a.$$

Correctness is exactly Proposition 6.2: with $p' = \mathsf{DerivePublic}(\Pi, t) = g^t$ and $p = \mathsf{DerivePublic}(\Pi, s) = g^s$,

$$\mathsf{Agree}(\Pi, s, g^t) = (g^t)^s = g^{ts} = g^{st} = (g^s)^t = \mathsf{Agree}(\Pi, t, g^s),$$

which is (1). This is the *Diffie–Hellman key-agreement scheme* $\mathsf{DH}_{\mathbb{G},g}$.

Remark 6.8 (Why this is the right abstraction)*.* The interface deliberately hides the group. A protocol designer who programs against DerivePublic and Agree can swap in any concrete scheme satisfying (1)—a prime-field group, an elliptic curve, or even a post-quantum non-interactive key agreement such as the CSIDH group action—without changing a line of the surrounding logic. The Zcash note-encryption machinery follows this pattern with one generalisation: the deployed DerivePublic takes the base point as an explicit argument, so that Sapling and Orchard derive keys over per-address *diversified bases* rather than a fixed generator (*Ironwood Guide*). It still invokes an abstract key agreement and never inspects the underlying group, which is why the construction transports from Sprout's Curve25519 to Sapling's Jubjub and Orchard's Pallas. A key-encapsulation mechanism, by contrast, cannot satisfy (1): encapsulation requires the recipient's public key before producing its ciphertext, so no counterparty-independent DerivePublic exists. The static–ephemeral one-flow pattern of Subsection 6.7 does generalise to any KEM, with the encapsulation ciphertext playing the role of the ephemeral public key, at the cost of changing the interface the surrounding logic programs against.

The output of Agree is a group element, but downstream symmetric primitives require a uniform bit string of fixed length. Bridging this gap—turning a structured, non-uniform group element into key material—is the job of a key-derivation function, introduced in §5.6 and put to work in Subsection 6.7. First we must state the precise sense in which g^{ab} is secret.

6.4 Passive security and the Diffie–Hellman assumptions

The relevant notion for the bare protocol is security against a *passive* adversary—an eavesdropper who reads the channel but does not alter it. The adversary's entire view is the pair of public shares (g^a, g^b), and security demands that this view reveal nothing useful about g^{ab}. The two formalisations are exactly the Diffie–Hellman problems of §2.3: the *computational* problem CDH (Definition 2.4)—compute g^{ab} from (g^a, g^b)—and the *decisional* problem DDH (Definition 2.5)—distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c). We write $\mathrm{Adv}^{\mathrm{cdh}}_{\mathbb{G},g}(\mathcal{A}) = \Pr[\mathcal{A}(g^a, g^b) = g^{ab}]$ and $\mathrm{Adv}^{\mathrm{ddh}}_{\mathbb{G},g}(\mathcal{A})$ for the corresponding advantages. Section 2.4 establishes the hierarchy DDH $\leq$ CDH $\leq$ DLOG (Theorems 2.7 and 2.8) together with the pairing-based separation of DDH from CDH (Example 2.9).

CDH asserts only that the adversary cannot compute the *whole* secret. Keying a symmetric cipher requires more: g^{ab} must be *indistinguishable* from a uniformly random group element, so that no individual bit, nor any predicate, of it leaks. The following concrete break shows the gap is real even without pairings.

Remark 6.9 (A concrete DDH break: the Legendre symbol). In the full multiplicative group $\mathbb{F}_p^\times$ the Legendre symbol $\left(\frac{\cdot}{p}\right)$ is efficiently computable and multiplicative, so (g^a, g^b) reveals the quadratic-residue characters of g^a and g^b, and these determine the character of g^{ab}; comparing the predicted character with that of the third component distinguishes $\mathcal{D}_{\mathrm{dh}}$ from $\mathcal{D}_{\mathrm{rand}}$ with constant advantage, even though CDH in $\mathbb{F}_p^\times$ is believed hard. This motivates the restriction to a prime-order subgroup (Remark 6.3), in which every element is a square and the character leak vanishes.

We can now state the passive-security theorem precisely. The natural target is that the shared secret is, to a passive eavesdropper, indistinguishable from a fresh random group element.

> **Theorem 6.10** (Passive security of Diffie–Hellman). *Let $\mathsf{DH}_{\mathbb{G},g}$ be the scheme of Example 6.7. Consider a passive adversary $\mathcal{A}$ who, given the transcript $(A, B) = (g^a, g^b)$ of an honest ephemeral–ephemeral session, must distinguish the true shared secret g^{ab} from a uniformly random element of $\mathbb{G}$. Any such distinguisher $\mathcal{A}$ has distinguishing advantage exactly $\mathrm{Adv}^{\mathrm{ddh}}_{\mathbb{G},g}(\mathcal{A})$, and the reduction preserves running time exactly. In particular, under the DDH assumption the shared secret is pseudorandom, and a passive adversary learns nothing about it that it could not have guessed about a random group element.*

Proof. The adversary's challenge is, by definition, to distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) with a, b, c uniform and independent—verbatim the DDH distinguishing game of Definition 2.5. Hence any distinguisher's advantage in the passive-security game equals its DDH advantage, which is negligible under the assumption. Pseudorandomness of g^{ab} follows immediately: no efficient test separates it from uniform. □

Remark 6.11 (Why pseudorandomness, not just unpredictability). Theorem 6.10 is the property a key-agreement scheme requires. Feeding g^{ab} into a key-derivation function to produce a symmetric key, we model that function (in Subsection 6.7) as extracting near-uniform bits from an input of high min-entropy that is hard to guess. CDH alone guarantees that g^{ab} is hard to guess in full but not that it lacks efficiently-predictable structure; DDH guarantees it is as good as random. In practice one often relies only on CDH together with a *hash* key-derivation function modelled as a random oracle, which manufactures the missing pseudorandomness; we make this precise below. In either case the design goal is that the derived symmetric key be indistinguishable from uniform.

6.5 Active attacks and the role of authentication

Passive security is the most the bare protocol can offer, and it is fundamentally limited. The protocol of Definition 6.1 provides *no* authentication: nothing in g^a or g^b testifies to who sent it. An *active* adversary who controls the channel can mount the *man-in-the-middle* attack.

Definition 6.12 (Man-in-the-middle attack)**.** An active adversary M sitting on the channel between Alice and Bob runs two independent Diffie–Hellman sessions:

1. M samples $m \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ and computes g^m. When Alice sends $A = g^a$ (intended for Bob), M intercepts it and instead forwards g^m to Bob. When Bob sends $B = g^b$ (intended for Alice), M intercepts it and forwards g^m to Alice.
2. Alice, believing she shares with Bob, computes $k_A = (g^m)^a = g^{am}$. M computes the same value as $A^m = g^{am}$. Symmetrically, Bob computes $k_B = (g^m)^b = g^{bm}$, which M also computes as B^m.

Proposition 6.13 (Man-in-the-middle defeats unauthenticated DH)**.** *After the attack of Definition 6.12, Alice and Bob each hold a shared secret that they believe is shared with the other but is in fact shared with M: M knows both Alice's g^{am} and Bob's g^{bm}. M can therefore decrypt, read, and re-encrypt every subsequent message in each direction, remaining completely transparent to both parties. No assumption on $\mathbb{G}$—not even ideal hardness of DLOG—prevents this.*

Proof. By Proposition 6.2 applied to each of the two sessions, Alice and M agree on g^{am} and Bob and M agree on g^{bm}. M knows m and both A and B, so it computes both secrets directly. Since neither Alice nor Bob verifies that the group element received originated from the intended peer, neither can detect the attack from their transcripts: each sees a syntactically valid run. The hardness of DLOG is irrelevant because M never inverts anything—it knows its own exponent m. □

The distinction is structural: **passive security concerns secrecy of the secret; active security additionally requires authenticity of the public shares.** Confidentiality of a channel is worthless if one established it with the wrong party. The remedy is to *authenticate the Diffie–Hellman messages*—to bind each public share to the identity of its sender—so that the recipient rejects a substituted share.

Remark 6.14 (How authentication is supplied)*.* Several mechanisms achieve this binding, all external to the bare key exchange:

- *Signed Diffie–Hellman.* Each party signs its ephemeral share with a long-term signing key whose public half a certificate authority or a trust-on-first-use record certifies. M cannot forge Alice's signature on g^m, so Bob rejects the substituted share. This is the structure of authenticated TLS.
- *Static keys with authenticated public keys.* If Bob's public key $B = g^b$ is itself authentic—fetched from a trusted directory or pinned in advance—then a static–ephemeral exchange to B already authenticates *Bob* to Alice: only the holder of b can complete it. This is the model relevant to public-key encryption and to shielded-note delivery, where the recipient's address *is* an authenticated public key. This authenticates the recipient, not the sender; sender authentication, when needed, layers on separately.
- *Out-of-band verification.* The parties compare a short fingerprint of the exchanged keys over an authentic side channel (reading digits aloud, scanning a QR code), detecting any substitution because M's injected g^m yields a different fingerprint.

In every case the security goal of the key agreement proper remains the passive guarantee of Theorem 6.10; authentication is a separable layer with which the key agreement composes, not a property of the group operation itself.

6.6 Elliptic-curve Diffie–Hellman

The development above is stated for an abstract cyclic group, by design: the protocol, the abstraction, and the security reductions are group-agnostic. We now instantiate the group $\mathbb{G}$ with the group of points of an elliptic curve, the instantiation used in modern systems and in Zcash specifically.

Recall from the companion volume that an elliptic curve E over a field $\mathbb{F}$ (of characteristic neither 2 nor 3, say given in short Weierstrass form $y^2 = x^3 + ax + b$ with nonzero discriminant) has a set of $\mathbb{F}$-rational points $E(\mathbb{F})$ which, together with the point at infinity $\mathcal{O}$ as identity and the chord-and-tangent addition law, forms a finite abelian group. We write this group *additively*, so the analogue of exponentiation $g \mapsto g^x$ is *scalar multiplication* $P \mapsto [x]P = \underbrace{P + \cdots + P}_{x}$, computable in $O(\log x)$ point additions by double-and-add.

Definition 6.15 (Elliptic-curve Diffie–Hellman)**.** Fix an elliptic curve $E/\mathbb{F}$, a base point $P \in E(\mathbb{F})$ of large prime order n generating a subgroup $\mathbb{G} = \langle P \rangle$, and cofactor $h = |E(\mathbb{F})|/n$. The scheme ECDH, in the abstraction of Definition 6.6, is

$$\mathsf{KeyGen} = a \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}, \qquad \mathsf{DerivePublic}(a) = [a]P, \qquad \mathsf{Agree}(a, Q) = [a]Q.$$

The shared secret of two parties with private keys a, b is

$$[a]\big([b]P\big) = [ab]P = [b]\big([a]P\big),$$

the point $[ab]P$. The associated hard problems—ECDLOG, ECDH, EDDH—are Definitions 2.2, 2.4, 2.5 read with $[x]P$ for g^x.

Proposition 6.16 (Correctness of ECDH)**.** *ECDH satisfies the correctness condition* (1)*.*

Proof. Scalar multiplication is the additive form of the cyclic-group exponentiation map: $[x]([y]P) = [xy]P$ because the group is abelian and $\mathbb{Z}$ acts on it through $\mathbb{Z}/n\mathbb{Z}$ (as $[n]P = \mathcal{O}$). Hence $\mathsf{Agree}(a, [b]P) = [a]([b]P) = [ab]P = [ba]P = [b]([a]P) = \mathsf{Agree}(b, [a]P)$, which is (1). □

Remark 6.17 (Why elliptic curves)*.* Section 2.5 quantifies the choice: elliptic-curve groups admit only the generic $O(\sqrt{n})$ attacks, whereas index calculus solves DLOG in $\mathbb{F}_p^\times$ in subexponential time (Remark 2.18), so a 256-bit curve delivers the security that a multi-thousand-bit prime field requires (Example 2.17). This efficiency, together with the availability of curves engineered for cheap in-circuit arithmetic, is why the Pallas curve underlies Orchard.

Remark 6.18 (Subgroup and curve validation)*.* The elliptic-curve setting reintroduces the small-subgroup concern of Remark 6.3 in a sharpened form. A recipient computing $\mathsf{Agree}(a, Q) = [a]Q$ on an attacker-supplied Q must guard against two pitfalls. First, Q must actually lie on E: an *invalid-curve attack* feeds a point on a different, weaker curve sharing the same addition formulas, so the implementation must check the curve equation. Second, if the full group $E(\mathbb{F})$ has cofactor $h > 1$, an adversary can submit a Q of small order h to confine $[a]Q$ to a tiny subgroup and learn a mod (small factor); the defence is either to multiply incoming points by the cofactor (*cofactor clearing*) or to use a prime-order curve where $h = 1$. Prime-order curves such as Pallas sidestep the cofactor issue entirely, leaving only the on-curve check.

6.7 Hybrid public-key encryption

We can now assemble the final construction of the section. Diffie–Hellman yields the two parties a shared group element, but three obstacles stand between that element and a usable encryption

scheme: (i) the shared secret is a structured group element, not uniform key bits; (ii) Diffie–Hellman by itself transports no message; and (iii) we want the static–ephemeral, non-interactive flow of Subsection 6.2, so that the recipient need not be online. The resolution is to compose key agreement with a key-derivation function and an authenticated cipher. The construction requires two ingredients from the symmetric world.

Definition 6.19 (Key-derivation function, recalled)**.** We use the specialisation of Definition 5.23 (§5.6) obtained by omitting the optional salt and fixing the output length to ℓ: a *key-derivation function* (KDF) is a deterministic function $\mathsf{KDF} : \mathcal{K} \times \{0,1\}^* \to \{0,1\}^\ell$ mapping a (possibly non-uniform, high-entropy) *input keying material* $k \in \mathcal{K}$ together with an optional context string to an ℓ-bit output. Security is as there: whenever k is drawn from a distribution of sufficiently high min-entropy that is hard to predict, the output $\mathsf{KDF}(k, c)$ is computationally indistinguishable from a uniform ℓ-bit string, even given the context c. When KDF is built from a cryptographic hash, the cleanest model is the *random oracle*: $\mathsf{KDF}(\cdot)$ behaves like a function whose every output is an independent uniform string.

Definition 6.20 (Authenticated encryption with associated data, recalled)**.** We recall the *authenticated encryption with associated data* (AEAD) schemes of Definition 5.13, with key space specialised to $\{0,1\}^\ell$ to match the KDF output: a pair (Enc, Dec), where

$$\mathsf{Enc}_K(N, \mathrm{ad}, m) = c, \qquad \mathsf{Dec}_K(N, \mathrm{ad}, c) \in \{m\} \cup \{\bot\},$$

N is a nonce, ad is associated data authenticated but not encrypted, and $\bot$ denotes rejection. Correctness: $\mathsf{Dec}_K(N, \mathrm{ad}, \mathsf{Enc}_K(N, \mathrm{ad}, m)) = m$. Security is *authenticated-encryption security* (Definitions 5.14 and 5.15): encryptions of any two equal-length messages are indistinguishable under chosen-plaintext attack *and* no adversary can produce a fresh ciphertext that decrypts to anything but $\bot$ (ciphertext integrity), so confidentiality and integrity hold simultaneously.

With these in hand we define the composed scheme. It is a *public-key* encryption scheme: anyone holding the recipient's static public key can encrypt to them, and only the holder of the matching private key can decrypt.

Definition 6.21 (DH-based hybrid public-key encryption)**.** Fix a key-agreement scheme (Definition 6.6)—concretely ECDH over $\mathbb{G} = \langle P \rangle$ of prime order n—a secure KDF, and a secure AEAD. The recipient holds a static key pair $(b, B = [b]P)$, with B published. To encrypt a message m to B:

1. **Ephemeral key.** Sample $e \xleftarrow{\$} \mathbb{Z}/n\mathbb{Z}$ and set the *ephemeral public key* $E_{\mathrm{pk}} = [e]P = \mathsf{DerivePublic}(e)$.
2. **Agree.** Compute the shared secret $S = \mathsf{Agree}(e, B) = [e]B = [eb]P$.
3. **Derive.** Set the symmetric key $K = \mathsf{KDF}(S, \text{context})$, where context includes E_{pk} (and protocol labels).
4. **Encrypt.** Output the ciphertext $\big(E_{\mathrm{pk}}, c\big)$, where $c = \mathsf{Enc}_K(N, \mathrm{ad}, m)$ for a fixed or derived nonce N and any associated data ad.

To decrypt (E_{pk}, c) with private key b:

1. **Agree.** Recompute $S' = \mathsf{Agree}(b, E_{\mathrm{pk}}) = [b]E_{\mathrm{pk}} = [be]P$.

2. **Derive.** Set $K' = \mathsf{KDF}(S', \text{context})$ with the same context (the recipient reads E_{pk} from the ciphertext).

3. **Decrypt.** Output $\mathsf{Dec}_{K'}(N, \text{ad}, c)$, which is m on success and $\perp$ on any tampering.

Theorem 6.22 (Correctness of hybrid encryption)**.** *For every message m and every honestly generated ciphertext (E_{pk}, c) produced as in Definition 6.21, decryption returns m.*

Proof. By correctness of the key agreement (Proposition 6.16), $S' = [b]E_{\text{pk}} = [b]([e]P) = [be]P = [eb]P = [e]([b]P) = [e]B = S$. Since KDF is deterministic and both sides supply the same context (the recipient recovers E_{pk}, and hence the context, from the ciphertext), $K' = \mathsf{KDF}(S', \text{context}) = \mathsf{KDF}(S, \text{context}) = K$. By AEAD correctness, $\mathsf{Dec}_K(N, \text{ad}, \mathsf{Enc}_K(N, \text{ad}, m)) = m$. □

Theorem 6.23 (Passive security of hybrid encryption)**.** *Model* KDF *as a random oracle and let* AEAD *be AEAD-secure. Then the scheme of Definition 6.21 is secure against a passive adversary who sees the recipient's public key B and a challenge ciphertext: under the CDH assumption in $\mathbb{G}$, no probabilistic polynomial-time adversary can distinguish encryptions of two equal-length messages of its choosing. Concretely, an adversary making at most q_H random-oracle queries has distinguishing advantage bounded by*

$$\mathrm{Adv}^{\text{cpa}}(\mathcal{A}) \;\le\; q_H \cdot \mathrm{Adv}^{\text{cdh}}_{\mathbb{G},P}(\mathcal{B}) \;+\; \mathrm{Adv}^{\text{ae}}(\mathcal{C}),$$

for explicit reductions $\mathcal{B}$ (against CDH) and $\mathcal{C}$ (against AEAD) running in essentially the same time as $\mathcal{A}$.

Proof. Sketch. The argument proceeds by a sequence of games. *Game 0* is the real chosen-plaintext game: the adversary submits m_0, m_1, receives the encryption of m_β for a hidden bit β, and attempts to guess β. The challenge ciphertext is (E_{pk}, c) with $E_{\text{pk}} = [e]P$, $S = [eb]P$, $K = \mathsf{KDF}(S, \text{context})$, and $c = \mathsf{Enc}_K(\cdots, m_\beta)$.

Game 1 replaces K by an *independent uniform* ℓ-bit string $\widetilde{K}$. In the random-oracle model, Games 0 and 1 are identical unless the adversary queries the oracle at the exact point $(S, \text{context}) = ([eb]P, \text{context})$; only then can it observe that K is the oracle's value rather than fresh randomness. But querying that point requires the adversary to produce $S = [eb]P$ from its view $(B, E_{\text{pk}}) = ([b]P, [e]P)$—which is exactly solving a CDH instance. Formally, the reduction $\mathcal{B}$ embeds its CDH challenge $([b]P, [e]P)$ as (B, E_{pk}) and runs $\mathcal{A}$, answering all random-oracle queries with fresh uniform values. In a pairing-free group $\mathcal{B}$ cannot *recognise* which query, if any, carries the first component $[eb]P$, so it cannot pick the solution out on sight; instead it selects one of $\mathcal{A}$'s at most q_H oracle queries uniformly at random and outputs that query's first component as its CDH answer. Whenever $\mathcal{A}$ queries the critical point—the only event distinguishing the two games—$\mathcal{B}$'s guess is correct with probability at least $1/q_H$. Hence $|\Pr[\mathcal{A} \text{ wins Game 0}] - \Pr[\mathcal{A} \text{ wins Game 1}]| \le \Pr[\text{critical query}] \le q_H \cdot \mathrm{Adv}^{\text{cdh}}_{\mathbb{G},P}(\mathcal{B})$. The factor q_H is a genuine security loss of the guessing kind catalogued in Definition 1.25; assuming the stronger *gap*-CDH (CDH given a DDH-decision oracle, with which $\mathcal{B}$ could test each query) would restore tightness, but we state the loose bound honestly.

In *Game 1* the AEAD key $\widetilde{K}$ is uniform and independent of everything in the adversary's view, so the only route to advantage is to break the AEAD scheme directly. A standard reduction $\mathcal{C}$ turns any remaining advantage into an advantage against AEAD confidentiality: $\mathcal{C}$ forwards m_0, m_1 to its own AEAD challenger, embeds the returned ciphertext as c, and relays $\mathcal{A}$'s guess. Thus $\Pr[\mathcal{A} \text{ wins Game 1}] - \frac{1}{2} \le \mathrm{Adv}^{\text{ae}}(\mathcal{C})$. Summing the two bounds gives the claim. (If we instead assume KDF only to be a secure key-derivation function in the sense of Definition 6.19, the same

argument goes through under DDH in place of CDH, using Theorem 6.10 to replace S by a uniform group element before deriving K.) □

Remark 6.24 (Why each ingredient is necessary). The composition is tight; dropping any piece breaks it.

- *Without the KDF*, one would key the cipher with the raw group element S, whose bit-representation is non-uniform and, in non-DDH groups, partially predictable (Remark 6.9); the KDF both standardises the format and extracts uniform randomness, and by binding E_{pk} into its context it ties the key to this particular ephemeral.
- *Without the AEAD's integrity*, a passively confidential but malleable cipher would let an active adversary tamper with c undetectably; authenticated encryption ensures any modification yields $\perp$, giving chosen-ciphertext robustness for the symmetric payload.
- *Without the ephemeral key* (a static–static secret), every message to B would reuse the same K, violating the nonce-uniqueness that AEAD security demands and destroying forward secrecy. The fresh ephemeral per message both supplies forward secrecy with respect to the ephemeral and guarantees a fresh key for each ciphertext.

Remark 6.25 (Delivering shielded notes). Definition 6.21 is precisely the pattern by which a shielded transaction delivers a note to its recipient *in band*—on the public ledger, readable by no one but the recipient. The recipient's shielded address embeds a static public key $B = [b]G_d$ on the protocol's curve (Pallas, in Orchard); the base G_d is not a protocol-wide generator but a per-address *diversified base*, the hash-to-curve image of the address's diversifier (*Ironwood Guide*). The sender, constructing the note, samples an ephemeral e, publishes $E_{\text{pk}} = [e]G_d$ over that same base as part of the transaction, derives $K = \mathsf{KDF}([eb]G_d, \dots)$, and AEAD-encrypts the note plaintext (value, recipient, randomness, memo) under K, attaching the ciphertext to the transaction. Anyone scanning the chain sees (E_{pk}, c) but, lacking b, cannot derive K and so learns nothing; the recipient, trial-deriving $K' = \mathsf{KDF}([b]E_{\text{pk}}, \dots)$ for incoming transactions, decrypts exactly those notes addressed to them. The non-interactivity of Subsection 6.2 is essential: the recipient need never have been online when the sender created the note. Authenticity of B (Remark 6.14) comes from the address being part of the recipient's verified payment credential, which is what defeats the man-in-the-middle concern of Subsection 6.5 in this setting; the zero-knowledge proof accompanying the transaction supplies the complementary integrity guarantees on the note's contents.

7 Commitment schemes

A *commitment scheme* is the cryptographic analogue of placing a message inside a sealed, tamper-evident envelope and handing it to another party. Once the sender delivers the envelope, its sender can no longer alter the message it contains (the scheme is *binding*), yet the recipient cannot read the message until the sender chooses to unseal it (the scheme is *hiding*). This deceptively simple primitive is among the most reusable building blocks in cryptography: it underlies coin-flipping over a telephone, zero-knowledge proofs, verifiable secret sharing, and—of central importance here—the polynomial commitments that form the backbone of modern succinct argument systems such as those used in Zcash's Orchard protocol.

This section develops commitments from first principles. It begins with the abstract syntax and the two security properties, hiding and binding, in each of their three quantitative flavours (perfect, statistical, computational), and establishes the fundamental impossibility result that no scheme can be simultaneously perfectly hiding and perfectly binding. It then studies concrete constructions of increasing algebraic richness: hash-based commitments, the Pedersen commitment and its vector generalisation, the Sinsemilla-style algebraic hash whose collision resistance reduces to discrete logarithm, and finally polynomial commitment schemes, contrasting the pairing-based and inner-product-argument families and making precise what it means to "commit to a polynomial and later prove an evaluation."

Throughout, $\lambda \in \mathbb{N}$ denotes the security parameter, and we use the vocabulary of Section 1: a function $\epsilon\colon \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible*, written $\epsilon(\lambda) = \mathrm{negl}(\lambda)$, if it shrinks faster than the reciprocal of every polynomial (Definition 1.11); $\mathrm{poly}(\lambda)$ denotes the class of functions bounded above by some fixed polynomial in λ; and a randomised algorithm is *probabilistic polynomial time* (PPT) if its running time is bounded by a polynomial in λ on every input (Definition 1.3). The notation $x \leftarrow D$ denotes sampling x according to a distribution D, and $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from a finite set S.

7.1 Syntax

We first fix the interface of a commitment scheme as a pair of algorithms, deliberately separating the public parameters (generated once and shared by all parties) from the per-commitment data.

Definition 7.1 (Commitment scheme)**.** A *commitment scheme* for a message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a triple of algorithms $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open})$:

- $\mathsf{Setup}(1^\lambda)$ is a PPT algorithm that on input the security parameter (in unary) outputs public parameters pp. These parameters implicitly determine $\mathcal{M} = \mathcal{M}_{\mathsf{pp}}$ and $\mathcal{R} = \mathcal{R}_{\mathsf{pp}}$, and serve as an implicit input to the remaining algorithms.
- $\mathsf{Commit}(\mathsf{pp}, m; r)$ is a deterministic algorithm that takes a message $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$ and outputs a *commitment* c. The notation $\mathsf{Commit}(\mathsf{pp}, m)$ without an explicit r signifies that the algorithm samples $r \xleftarrow{\$} \mathcal{R}$ uniformly and returns it alongside c.
- $\mathsf{Open}(\mathsf{pp}, c, m, r)$ is a deterministic algorithm that outputs 1 ("accept") or 0 ("reject").

The scheme must satisfy *correctness*: for all $\mathsf{pp} \in \mathsf{Setup}(1^\lambda)$, all $m \in \mathcal{M}$ and all $r \in \mathcal{R}$,

$$\mathsf{Open}\big(\mathsf{pp},\ \mathsf{Commit}(\mathsf{pp}, m; r),\ m,\ r\big) = 1.$$

We call the pair (m, r) used to produce a commitment the *opening* (or *decommitment*); revealing it is the means by which the committer later unseals the envelope. In the simplest and most common setting, $\mathsf{Open}(\mathsf{pp}, c, m, r)$ merely recomputes $\mathsf{Commit}(\mathsf{pp}, m; r)$ and checks whether the result equals

c; we term such a scheme *canonically opening*. We adopt this convention unless stated otherwise. For a canonically opening scheme, correctness holds automatically, and a valid opening of c is precisely a pair (m, r) with $\mathsf{Commit}(\mathsf{pp}, m; r) = c$.

Remark 7.2 (The two phases). A commitment scheme operates in two temporally separated phases. In the *commit phase*, the sender computes $c \leftarrow \mathsf{Commit}(\mathsf{pp}, m; r)$ and transmits c (only). In the later *reveal phase*, the sender transmits (m, r), and the receiver runs Open to verify. Hiding constrains what the receiver learns after the commit phase; binding constrains what the sender can do in the reveal phase. The setup phase, which produces pp, is a third concern: who runs it, and whether one can trust it, proves decisive for some constructions, as Remark 7.3 discusses.

Remark 7.3 (Setup models). Three setup models recur. In the *common reference string* (CRS) model, a trusted party runs Setup and we assume it erases the trapdoor randomness it used; of the schemes below, only KZG genuinely resides here. In the *uniform random string* (URS) or transparent model, pp consists only of public coins (e.g. uniformly random group elements with no known relations), so there is no trapdoor to erase and no party to trust; this is the regime of hash-based, Pedersen, and inner-product commitments—a Pedersen generator derived by hashing to the curve (Remark 3.28) is transparent, even though the textbook presentation below phrases Setup as sampling and discarding a secret. In the *plain* model there are no parameters at all, or only a fixed function such as a standardised hash. Transparent setup is highly desirable in practice because a flawed or subverted trusted setup can silently destroy binding, as we demonstrate below for KZG.

7.2 Hiding and binding

We now formalise the two security properties. Each is naturally cast as a game between a challenger and an adversary $\mathcal{A}$, and each comes in three flavours according to how strong a quantitative guarantee we demand against how powerful an adversary.

Hiding. Intuitively, a commitment should leak no information about the committed message. We capture this by demanding that the distributions of commitments to any two messages be indistinguishable.

Definition 7.4 (Hiding)**.** For a commitment scheme $\Pi = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open})$ and an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, consider the following experiment $\mathrm{Hide}^{\mathcal{A}}_{\Pi}(\lambda, b)$ parameterised by a bit $b \in \{0, 1\}$:

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$;
2. $\mathcal{A}_1(\mathsf{pp})$ outputs two messages $m_0, m_1 \in \mathcal{M}$ and a state st;
3. the challenger samples $r \xleftarrow{\$} \mathcal{R}$ and computes $c \leftarrow \mathsf{Commit}(\mathsf{pp}, m_b; r)$;
4. $\mathcal{A}_2(\mathsf{st}, c)$ outputs a bit b', which is the output of the experiment.

The *hiding advantage* of $\mathcal{A}$ is

$$\mathrm{Adv}^{\mathrm{hide}}_{\Pi}(\mathcal{A}, \lambda) \;=\; \Big|\Pr\big[\mathrm{Hide}^{\mathcal{A}}_{\Pi}(\lambda, 0) = 1\big] - \Pr\big[\mathrm{Hide}^{\mathcal{A}}_{\Pi}(\lambda, 1) = 1\big]\Big|.$$

The scheme is:

- *computationally hiding* if for every PPT $\mathcal{A}$, $\mathrm{Adv}^{\mathrm{hide}}_{\Pi}(\mathcal{A}, \lambda) = \mathrm{negl}(\lambda)$;
- *statistically hiding* if the bound holds even for computationally unbounded $\mathcal{A}$, i.e. $\mathrm{Adv}^{\mathrm{hide}}_{\Pi}(\mathcal{A}, \lambda) = \mathrm{negl}(\lambda)$ for all (not necessarily efficient) $\mathcal{A}$;

- *perfectly hiding* if $\mathrm{Adv}^{\mathrm{hide}}_{\Pi}(\mathcal{A},\lambda) = 0$ for all $\mathcal{A}$ and all λ.

Equivalently, the scheme is perfectly hiding iff for every fixed pp and every pair $m_0, m_1 \in \mathcal{M}$, the random variables $\mathsf{Commit}(\mathsf{pp}, m_0; r)$ and $\mathsf{Commit}(\mathsf{pp}, m_1; r)$, induced by $r \xleftarrow{\$} \mathcal{R}$, are identically distributed; it is statistically hiding iff their statistical distance is negligible. Recall that the *statistical distance* between two distributions P, Q over a finite set Ω is $\Delta(P,Q) = \frac{1}{2}\sum_{\omega\in\Omega} |P(\omega) - Q(\omega)|$, and that $\Delta(P,Q)$ equals exactly the maximum, over all (even unbounded) distinguishers, of the advantage in telling P from Q; this accounts for the coincidence of statistical hiding and the unbounded-adversary definition.

Binding. Intuitively, a committer must not be able to open one commitment in two different ways. The binding requirement forbids producing a single commitment together with two valid openings to distinct messages.

Definition 7.5 (Binding)**.** For Π and an adversary $\mathcal{A}$, consider the experiment $\mathrm{Bind}^{\mathcal{A}}_{\Pi}(\lambda)$:

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$;
2. $\mathcal{A}(\mathsf{pp})$ outputs a commitment c and two openings $(m_0, r_0), (m_1, r_1)$;
3. the experiment outputs 1 iff $m_0 \neq m_1$ and $\mathsf{Open}(\mathsf{pp}, c, m_0, r_0) = \mathsf{Open}(\mathsf{pp}, c, m_1, r_1) = 1$.

The *binding advantage* is $\mathrm{Adv}^{\mathrm{bind}}_{\Pi}(\mathcal{A},\lambda) = \Pr\big[\mathrm{Bind}^{\mathcal{A}}_{\Pi}(\lambda) = 1\big]$. The scheme is:

- *computationally binding* if for every PPT $\mathcal{A}$, $\mathrm{Adv}^{\mathrm{bind}}_{\Pi}(\mathcal{A},\lambda) = \mathrm{negl}(\lambda)$;
- *statistically binding* if the same holds for all unbounded $\mathcal{A}$;
- *perfectly binding* if $\mathrm{Adv}^{\mathrm{bind}}_{\Pi}(\mathcal{A},\lambda) = 0$ for all $\mathcal{A}$ and all λ.

A clean reformulation is useful. For fixed pp, two openings *collide* if $\mathsf{Commit}(\mathsf{pp}, m_0; r_0) = \mathsf{Commit}(\mathsf{pp}, m_1; r_1)$ with $m_0 \neq m_1$. For a canonically opening scheme, perfect binding means $\mathsf{Commit}(\mathsf{pp}, \cdot; \cdot)$ has the property that distinct messages never yield a common commitment value, i.e. the message m is a deterministic function of the commitment c. Thus perfect binding is equivalent to the map $c \mapsto m$ being well defined on the set of all attainable commitments.

Remark 7.6 (Note the asymmetry of the quantifiers). Hiding quantifies over an adversary's ability to extract information from a single honestly produced commitment; binding quantifies over an adversary's ability to manufacture an ambiguous commitment of its own choosing. A scheme can be strong in one and weak in the other, and the two properties pull in opposite directions, as the impossibility result of the next subsection makes precise.

7.3 Incompatibility of perfect hiding and perfect binding

The single most important structural fact about commitments is that perfect hiding and perfect binding cannot hold simultaneously. The intuition is short: perfect hiding implies that a commitment to m_0 could equally well be a commitment to m_1; but then *some* opening to m_1 must exist for that very commitment, which an unbounded committer can find, breaking binding.

Theorem 7.7 (No perfectly hiding and perfectly binding scheme)**.** *Let* $\Pi = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open})$ *be canonically opening with message space* $\mathcal{M}$ *of size at least* 2*. If* Π *is perfectly hiding, then it is not perfectly binding (and indeed an unbounded adversary breaks binding with probability* 1*). Conse-*

quently no nontrivial commitment scheme is simultaneously perfectly hiding and perfectly binding.

Proof. Fix any pp in the support of $\mathsf{Setup}(1^\lambda)$ and pick two distinct messages $m_0, m_1 \in \mathcal{M}$. For a message m, let

$$C_m \;=\; \{\, \mathsf{Commit}(\mathsf{pp}, m; r) \;:\; r \in \mathcal{R} \,\}$$

be the set of commitment values attainable from m, and let D_m be the distribution on C_m that $r \xleftarrow{\$} \mathcal{R}$ induces.

Perfect hiding implies that D_{m_0} and D_{m_1} are identical distributions; in particular they have the same support, so $C_{m_0} = C_{m_1}$. Pick any c in this common support. By definition of C_{m_0} there is $r_0 \in \mathcal{R}$ with $\mathsf{Commit}(\mathsf{pp}, m_0; r_0) = c$, and by definition of C_{m_1} there is $r_1 \in \mathcal{R}$ with $\mathsf{Commit}(\mathsf{pp}, m_1; r_1) = c$. Then $(c, (m_0, r_0), (m_1, r_1))$ is a binding break: both openings are valid (by correctness of canonical opening) and $m_0 \neq m_1$.

An unbounded adversary can carry out exactly this search—enumerating $\mathcal{R}$ to find r_0 and r_1—for any pp, and therefore wins $\mathrm{Bind}^{\mathcal{A}}_{\Pi}(\lambda)$ with probability 1. Hence $\mathrm{Adv}^{\mathrm{bind}}_{\Pi}(\mathcal{A}, \lambda) = 1 \neq 0$, so Π is not perfectly binding. □

Remark 7.8 (The information-theoretic content). The proof exposes the underlying tension as a counting fact. Perfect binding forces the map $c \mapsto m$ to be well defined, so the commitment c determines m and hence carries $\geq \log_2 |\mathcal{M}|$ bits of information about m. Perfect hiding forces c to carry *zero* bits of information about m. These are compatible only when $|\mathcal{M}| = 1$, i.e. when there is nothing to commit to. One therefore always chooses which property to make perfect (or statistical) and settles for the other being merely computational. Pedersen commitments (Subsection 7.5) are perfectly hiding and computationally binding; hash-based commitments in the random-oracle reading (Subsection 7.4) are computationally binding (from collision resistance) and computationally hiding.

Remark 7.9 (Two regimes, two failure modes). The choice has operational meaning. If binding is only computational, an adversary with sufficient computing power (e.g. one able to solve discrete logs) could retroactively change a committed value, so the soundness of any protocol built on top degrades to a computational assumption. If hiding is only computational, an adversary could retroactively learn a committed value, so any privacy guarantee degrades similarly, and *everlasting* secrecy is lost once the assumption is broken in the future. Which regime to prefer is a judgement about whether integrity or long-term confidentiality is the greater concern.

7.4 Hash-based commitments

The most elementary construction uses a cryptographic hash function. Recall that a hash function $H : \{0,1\}^* \to \{0,1\}^n$ is *collision resistant* if no PPT adversary can find $x \neq x'$ with $H(x) = H(x')$ except with negligible probability, and *preimage/one-way* if, given $H(x)$ for a random x of appropriate length, no PPT adversary can find any x' with $H(x') = H(x)$ except with negligible probability.

Definition 7.10 (Hash commitment)**.** Let $H : \{0,1\}^* \to \{0,1\}^n$ be a hash function, $\mathcal{M} = \{0,1\}^*$ and $\mathcal{R} = \{0,1\}^\ell$ with $\ell = \ell(\lambda)$ (e.g. $\ell = 2n$). Define

$$\mathsf{Commit}(m; r) \;=\; H(m \,\|\, r),$$

where $\|$ denotes concatenation, and open canonically. (There are no nontrivial public parameters beyond the choice of H.)

Proposition 7.11 (Binding of the hash commitment)**.** *If H is collision resistant, then the hash commitment is computationally binding.*

Proof. Suppose a PPT adversary $\mathcal{A}$ outputs a commitment c and two valid openings (m_0, r_0), (m_1, r_1) with $m_0 \neq m_1$ such that $H(m_0\|r_0) = c = H(m_1\|r_1)$. Since $m_0 \neq m_1$, the strings $m_0\|r_0$ and $m_1\|r_1$ are distinct (a difference in the message part survives concatenation when the message length is encoded, e.g. by prefixing $|m|$; we assume such an unambiguous encoding). Thus $\mathcal{A}$ has produced a collision for H. The reduction $\mathcal{B}$ that runs $\mathcal{A}$ and outputs $(m_0\|r_0, m_1\|r_1)$ therefore wins the collision game with exactly the binding advantage of $\mathcal{A}$, which must then be negligible. □

Proposition 7.12 (Hiding of the hash commitment)**.** *Model H as a random oracle. Then for messages of length bounded by* $\mathrm{poly}(\lambda)$ *and randomness length $\ell = \omega(\log \lambda)$, the hash commitment is computationally hiding; concretely, an adversary making q oracle queries has hiding advantage at most $q \cdot 2^{-\ell}$.*

Proof. Sketch. In the random-oracle model, $c = H(m_b\|r)$ is a uniformly random n-bit string *unless* the adversary ever queries H at the point $m_b\|r$. Since r is uniform in $\{0,1\}^\ell$ and information-theoretically hidden from $\mathcal{A}$ before it sees c, each of the adversary's q queries hits the point $m_b\|r$ with probability at most $2^{-\ell}$; a union bound bounds the probability that any query does by $q \cdot 2^{-\ell}$. Conditioned on no such query, the value c is independent of b, so the two worlds $b = 0$ and $b = 1$ are identically distributed. Hence the advantage is at most $q \cdot 2^{-\ell} = \mathrm{negl}(\lambda)$. □

Remark 7.13 (Why the randomness r is essential). Without r, the deterministic commitment $c = H(m)$ fails to be hiding whenever the message has low entropy: an adversary simply hashes its two candidate messages and compares. The randomness r (a "salt" or "nonce") injects min-entropy so that the input to H is unpredictable. This is the same phenomenon that renders *deterministic* encryption insecure for low-entropy plaintexts.

Remark 7.14 (Strength profile). Hash commitments are transparent (no trusted setup, no algebraic structure to subvert) and post-quantum plausible (collision resistance of good hash functions is not known to fall to quantum algorithms beyond a quadratic speedup, which a modest increase in n absorbs). Their weakness is that the commitment is an opaque bit string with *no algebraic structure*: one cannot, for instance, add two commitments to obtain a commitment to the sum of the messages. The next construction repairs exactly this.

7.5 The Pedersen commitment

We now turn to a commitment with rich algebraic structure. Let $\mathbb{G}$ be a cyclic group of prime order p, written additively, in which the *discrete logarithm problem* is hard. Additive notation $[a]P$ denotes the scalar multiple of $P \in \mathbb{G}$ by $a \in \mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$; thus $[a]P = P + P + \cdots + P$ (a summands), $[0]P = \mathcal{O}$ is the identity, and the map $a \mapsto [a]P$ is a group homomorphism $\mathbb{F}_p \to \mathbb{G}$. For present purposes $\mathbb{G}$ is the group of $\mathbb{F}_q$-points of an elliptic curve of prime order p; the only structural facts we use are that $\mathbb{G} \cong \mathbb{F}_p$ as a group and that discrete logs are hard.

Recall the DLP assumption of Definition 2.2. Throughout this section we fix a group generator $\mathcal{G}$ that on input 1^λ outputs $(\mathbb{G}, p, G)$ with $\mathbb{G}$ cyclic of prime order p (a $\Theta(\lambda)$-bit prime) and generator G; we say the *discrete logarithm* (DLog) *assumption* holds for $\mathcal{G}$ if no PPT adversary, given $(\mathbb{G}, p, G, [a]G)$ for uniform $a \xleftarrow{\$} \mathbb{F}_p$, outputs a except with negligible probability.

Definition 7.15 (Pedersen commitment)**.** On input 1^λ, let Setup run $(\mathbb{G}, p, G) \leftarrow \mathcal{G}(1^\lambda)$, sample $s \xleftarrow{\$} \mathbb{F}_p^\times$, and set $H = [s]G$; it outputs $\mathsf{pp} = (\mathbb{G}, p, G, H)$ and *discards* s. The message space is $\mathcal{M} = \mathbb{F}_p$

and the randomness space is $\mathcal{R} = \mathbb{F}_p$. Define

$$\mathsf{Commit}(\mathsf{pp}, m; r) \;=\; [m]G + [r]H,$$

and open canonically.

The group elements G and H are two generators whose relative discrete logarithm $s = \log_G H$ is unknown to all parties (the committer included). Everything below hinges on this: knowledge of s would break binding, while perfect hiding does not even require ignorance of s.

Theorem 7.16 (Perfect hiding of Pedersen)**.** *The Pedersen commitment is perfectly hiding.*

Proof. Fix $\mathsf{pp} = (\mathbb{G}, p, G, H)$ with $H \neq \mathcal{O}$ (which holds since $s \neq 0$), and fix any message $m \in \mathbb{F}_p$. Because H generates $\mathbb{G}$ (it is nonzero and $\mathbb{G}$ has prime order, so every nonzero element is a generator), the map $r \mapsto [r]H$ is a bijection $\mathbb{F}_p \to \mathbb{G}$. Hence, as r ranges uniformly over $\mathbb{F}_p$, the element $[r]H$ is uniformly distributed over $\mathbb{G}$, and so is its translate

$$\mathsf{Commit}(\mathsf{pp}, m; r) = [m]G + [r]H.$$

The distribution of the commitment is therefore the uniform distribution on $\mathbb{G}$, *independent of* m. Consequently for any two messages m_0, m_1 the commitment distributions coincide exactly, so the hiding advantage of *every* adversary—however powerful—is 0. □

Theorem 7.17 (Computational binding of Pedersen under DLog)**.** *If the discrete logarithm assumption holds for $\mathcal{G}$, then the Pedersen commitment is computationally binding. Concretely, from any adversary $\mathcal{A}$ that breaks binding with probability ε one constructs an adversary $\mathcal{B}$ that computes discrete logarithms with probability $(1 - 1/p)\varepsilon \geq \varepsilon - 1/p$ and essentially the same running time.*

Proof. Suppose $\mathcal{A}$, given $\mathsf{pp} = (\mathbb{G}, p, G, H)$, outputs a commitment c and two valid openings (m_0, r_0), (m_1, r_1) with $m_0 \neq m_1$ and

$$[m_0]G + [r_0]H \;=\; c \;=\; [m_1]G + [r_1]H.$$

Rearranging,

$$[m_0 - m_1]G \;=\; [r_1 - r_0]H.$$

We claim that $r_1 \neq r_0$. Indeed, if $r_0 = r_1$ then the right-hand side is $\mathcal{O}$, forcing $[m_0 - m_1]G = \mathcal{O}$; since G has order p this gives $m_0 \equiv m_1 \pmod{p}$, contradicting $m_0 \neq m_1$. With $r_1 - r_0 \neq 0$ invertible in the field $\mathbb{F}_p$, one may write

$$H \;=\; \left[(m_0 - m_1)(r_1 - r_0)^{-1}\right] G,$$

so the discrete logarithm of H to base G is $s = (m_0 - m_1)(r_1 - r_0)^{-1} \bmod p$.

Now consider the reduction $\mathcal{B}$: on a DLog challenge $(\mathbb{G}, p, G, Q)$ where the goal is to find $\log_G Q$, set $H := Q$, form $\mathsf{pp} = (\mathbb{G}, p, G, H)$, and run $\mathcal{A}(\mathsf{pp})$. The parameters pp are distributed almost exactly as in the real scheme: there $H = [s]G$ for uniform $s \in \mathbb{F}_p^\times$, so the real H is uniform over $\mathbb{G} \setminus \{\mathcal{O}\}$, whereas the challenge $Q = [a]G$ with uniform $a \in \mathbb{F}_p$ is uniform over all of $\mathbb{G}$; the two distributions lie within statistical distance $1/p = \mathrm{negl}(\lambda)$ (they differ only on the event $Q = \mathcal{O}$, which occurs with probability $1/p$). Moreover, when $Q = \mathcal{O}$ the relation $[m_0 - m_1]G = [r_1 - r_0]Q = \mathcal{O}$ forces $m_0 = m_1$, so $\mathcal{A}$ cannot win in that case anyway. When $\mathcal{A}$ succeeds, $\mathcal{B}$ outputs $(m_0 - m_1)(r_1 - r_0)^{-1} \bmod p$, which equals $\log_G Q$. Thus $\mathcal{B}$ succeeds exactly when $\mathcal{A}$ succeeds in the simulated game, giving $\mathrm{Adv}^{\mathrm{dlog}}(\mathcal{B}) = (1 - 1/p)\,\mathrm{Adv}^{\mathrm{bind}}(\mathcal{A}) \geq \varepsilon - 1/p$, the factor $1 - 1/p$ accounting for the event $Q = \mathcal{O}$, on which $\mathcal{A}$ wins with probability 0; this matches the convention of Theorem 7.22. Under DLog, $\varepsilon = \mathrm{negl}(\lambda)$. □

Remark 7.18 (Equivocation with the trapdoor). The proof shows more: any party that *knows* $s = \log_G H$ can open a Pedersen commitment to any message of its choosing. Given $c = [m]G + [r]H$ and a target m', set $r' = r + (m - m')s^{-1} \bmod p$; then $[m']G + [r']H = [m']G + [r]H + [(m - m')]G = [m]G + [r]H = c$. This *equivocation* (or *trapdoor*) property is not a defect: it is precisely why erasing s in Setup is mandatory, and it is the engine behind the zero-knowledge simulators of many protocols, which use the trapdoor to open commitments consistently with a simulated transcript.

Theorem 7.19 (Additive homomorphism)**.** *The Pedersen commitment is additively homomorphic: for all $m_0, m_1, r_0, r_1 \in \mathbb{F}_p$,*

$$\mathsf{Commit}(\mathsf{pp}, m_0; r_0) + \mathsf{Commit}(\mathsf{pp}, m_1; r_1) \;=\; \mathsf{Commit}(\mathsf{pp}, m_0 + m_1; r_0 + r_1),$$

and more generally, for scalars $\alpha, \beta \in \mathbb{F}_p$,

$$[\alpha]\,\mathsf{Commit}(\mathsf{pp}, m_0; r_0) + [\beta]\,\mathsf{Commit}(\mathsf{pp}, m_1; r_1) \;=\; \mathsf{Commit}(\mathsf{pp},\ \alpha m_0 + \beta m_1;\ \alpha r_0 + \beta r_1).$$

Proof. Both sides expand by the bilinearity of scalar multiplication over the additive group $\mathbb{G}$. For the first identity,

$$\big([m_0]G + [r_0]H\big) + \big([m_1]G + [r_1]H\big) = [m_0 + m_1]G + [r_0 + r_1]H,$$

using commutativity of $+$ in $\mathbb{G}$ and $[a]P + [b]P = [a+b]P$. The general identity follows from $[\alpha]([m]G + [r]H) = [\alpha m]G + [\alpha r]H$ and the same regrouping. □

Remark 7.20 (Consequences of homomorphism). Additive homomorphism is what renders Pedersen commitments indispensable in privacy-preserving value systems. In a confidential-transaction setting (as in Zcash's value-balancing), each input and output value v_i is committed as $[v_i]G + [r_i]H$; a transaction balances iff $\sum_i v_i^{\text{in}} = \sum_j v_j^{\text{out}}$, which a verifier checks *without learning any* v_i by testing whether $\sum_i C_i^{\text{in}} - \sum_j C_j^{\text{out}}$ is a commitment to 0, i.e. equals $[r]H$ for the *net* randomness r. The prover does not reveal r; it proves knowledge of r by *signing* the transaction under $[r]H$ as a verification key—the binding signature of §8.8. Homomorphism turns an arithmetic relation on hidden values into a group equation on public commitments.

7.6 Pedersen vector commitments

The scalar Pedersen scheme commits to a single field element. To commit to a whole vector $\mathrm{m} = (m_1, \ldots, m_n) \in \mathbb{F}_p^n$ in a single group element, the scheme uses n independent generators.

Definition 7.21 (Pedersen vector commitment)**.** On input 1^λ and a length $n = \mathrm{poly}(\lambda)$, let Setup output $(\mathbb{G}, p)$ together with generators $G_1, \ldots, G_n, H \xleftarrow{\$} \mathbb{G}$ sampled so that no nontrivial discrete-log relation among them is known (in practice one derives them from a public seed by hashing into the curve—a "nothing-up-my-sleeve" procedure that yields a transparent setup). The message space is $\mathbb{F}_p^n$ and the randomness space is $\mathbb{F}_p$. Define

$$\mathsf{Commit}(\mathsf{pp}, \mathrm{m}; r) \;=\; \sum_{i=1}^{n} [m_i]\,G_i \;+\; [r]\,H \;=\; \langle \mathrm{m},\, \mathrm{G} \rangle + [r]H,$$

where $\langle \mathrm{m}, \mathrm{G} \rangle = \sum_{i=1}^{n} [m_i]G_i$ denotes the "multiscalar" inner product of the scalar vector m with the generator vector $\mathrm{G} = (G_1, \ldots, G_n)$. Open canonically.

Theorem 7.22 (Properties of the vector commitment)**.** *The Pedersen vector commitment is perfectly hiding, and it is computationally binding under the discrete logarithm assumption in* $\mathbb{G}$*.*

Proof. Perfect hiding. Exactly as in Theorem 7.16: since $H \neq \mathcal{O}$ generates the prime-order group $\mathbb{G}$, the term $[r]H$ is uniform over $\mathbb{G}$ for uniform r, so $\langle \mathrm{m}, \mathrm{G} \rangle + [r]H$ is uniform over $\mathbb{G}$ regardless of m. Hence the commitment distribution is independent of the message.

Computational binding. The reduction is to the discrete logarithm in the stronger guise that finding *any* nontrivial relation among uniformly random generators is hard (this follows from DLog by embedding the challenge in every generator, as we now show). Concretely, suppose $\mathcal{A}$ outputs a commitment c with two valid openings $(\mathrm{m}, r) \neq (\mathrm{m}', r')$ and $\mathrm{m} \neq \mathrm{m}'$. Then

$$\sum_{i=1}^{n} [m_i]G_i + [r]H \;=\; \sum_{i=1}^{n} [m_i']G_i + [r']H,$$

hence

$$\sum_{i=1}^{n} [m_i - m_i']\,G_i \;+\; [r - r']\,H \;=\; \mathcal{O},$$

a nontrivial linear relation among $G_1, \ldots, G_n, H$ (nontrivial because some $m_i \neq m_i'$).

To convert this into a discrete log, the reduction $\mathcal{B}$ takes a DLog challenge $(\mathbb{G}, p, G, Q)$ with $Q = [s]G$ for unknown s, and embeds the challenge in *every* generator: it samples $u_i, w_i \xleftarrow{\$} \mathbb{F}_p$ and sets $G_i = [u_i]G + [w_i]Q$ for $i = 1, \ldots, n$, and likewise $H = [u_H]G + [w_H]Q$. Each generator is uniform in $\mathbb{G}$ (the term $[u_i]G$ alone is uniform), matching the real distribution, and the generators reveal nothing about the w-coordinates: for every candidate value of w_i there is exactly one u_i consistent with the observed G_i, so conditioned on the adversary's entire view the w_i, w_H remain independent and uniform. Writing the recovered relation as $\sum_i [a_i]G_i + [a_H]H = \mathcal{O}$ with the a_i, a_H not all zero, substitution yields the scalar equation $c_0 + c_1 s \equiv 0 \pmod{p}$ with $c_0 = \sum_i a_i u_i + a_H u_H$ and $c_1 = \sum_i a_i w_i + a_H w_H$. Since the coefficient vector is nonzero and the w-coordinates are uniform and independent of it, c_1 is uniform on $\mathbb{F}_p$, hence nonzero with probability $1 - 1/p$; then $s = -c_0\, c_1^{-1} \bmod p$ solves the challenge. Thus $\mathrm{Adv}^{\mathrm{dlog}}(\mathcal{B}) \geq (1 - 1/p)\,\mathrm{Adv}^{\mathrm{bind}}(\mathcal{A})$, which forces the binding advantage to be negligible. □

Remark 7.23 (A compressing commitment)*.* The vector commitment maps n field elements (plus randomness) to a *single* group element. It is therefore *compressing*: the commitment is exponentially shorter than the message for large n. Compression is incompatible with statistical binding (there are far more messages than commitments, so collisions exist in abundance), which is precisely why binding here is only *computational*—an unbounded adversary could find a colliding opening by solving the discrete-log relation. This is the same trade-off that Theorem 7.7 dictates, now visible at the level of cardinalities.

Remark 7.24 (Homomorphism in the vector setting)*.* The vector commitment remains additively homomorphic in both message and randomness, componentwise: $\mathsf{Commit}(\mathrm{m}; r) + \mathsf{Commit}(\mathrm{m}'; r') = \mathsf{Commit}(\mathrm{m} + \mathrm{m}'; r + r')$. This linearity is the foundation of the inner-product arguments discussed in Subsection 7.8, which prove statements about $\langle \mathrm{m}, \mathrm{b} \rangle$ for public vectors b while revealing only a single committed group element.

7.7 Sinsemilla: an algebraic hash-based commitment

The hash commitment of Subsection 7.4 is opaque; the Pedersen commitment is algebraic, but its binding rests on Commit being a simple linear map. A family of *algebraic hash functions* occupies an intermediate position: they are collision-resistant hashes whose internal operations are group operations on an elliptic curve, so that an arithmetic circuit verifies the entire computation cheaply, yet

whose collision resistance still reduces to discrete logarithm. The Sinsemilla hash used in Zcash's Orchard is the archetype. We present its core idea below, namely a Pedersen-style hash over a sequence of message chunks.

The starting point is the *Pedersen hash*. We split a message M into k chunks $m_1, \dots, m_k$, each interpreted as a scalar in some bounded range, and we fix k independent generators $P_1, \dots, P_k \in \mathbb{G}$ with unknown mutual discrete logs. Define

$$\text{PedersenHash}(M) \;=\; \sum_{j=1}^{k} [m_j]\, P_j \;\in\; \mathbb{G}.$$

This is precisely the (unblinded) Pedersen vector commitment map, now employed as a hash–its output is a group element rather than a fixed-length bit string, but composing with a fixed encoding of group elements yields an ordinary hash.

Proposition 7.25 (Collision resistance of the Pedersen hash reduces to DLog)**.** *If no efficient algorithm can find a nontrivial relation $\sum_j [a_j]P_j = \mathcal{O}$ (with not all $a_j = 0$ and each $|a_j|$ within the allowed chunk range) among the generators $P_1, \dots, P_k$–a hardness that follows from the discrete logarithm assumption in $\mathbb{G}$ when the P_j are sampled as independent uniform generators–then* PedersenHash *is collision resistant.*

Proof. A collision is a pair $M \neq M'$ with the same hash. Writing the chunk decompositions (m_j) and (m'_j), equality of hashes gives

$$\sum_{j=1}^{k} [m_j]P_j \;=\; \sum_{j=1}^{k} [m'_j]P_j \quad\Longrightarrow\quad \sum_{j=1}^{k} [\, m_j - m'_j\,]P_j \;=\; \mathcal{O}.$$

Because $M \neq M'$, the coefficient vector $(m_j - m'_j)$ is nonzero, so this is a nontrivial relation among the P_j. The reduction to discrete log is the every-generator embedding of Theorem 7.22: express each generator as $[u_i]G + [w_i]Q$ for the challenge Q and read off the unknown logarithm from the recovered relation. Hence an efficient collision-finder yields an efficient discrete-log solver, contradicting the assumption. □

The plain Pedersen hash requires one independent generator per chunk, which is costly when hashing long inputs. *Sinsemilla* reorganises the computation into an iterated, two-operand form that reuses a small, fixed table of generators while preserving the discrete-log reduction. It splits the message into pieces $m_1, m_2, \dots, m_k$ of a fixed width of w bits and maintains a running accumulator $\text{Acc} \in \mathbb{G}$. It uses a fixed base point Q and a table $S : \{0,1\}^w \to \mathbb{G}$ that assigns to each possible *value* of a w-bit piece its own generator, derived by hashing-to-curve a domain string together with that value—so the table has 2^w entries, shared by all positions. The iteration is, schematically,

$$\text{Acc}_0 = Q, \qquad \text{Acc}_j = \text{Acc}_{j-1} \;+\; \big(\text{Acc}_{j-1} + S(m_j)\big) \;=\; [2]\,\text{Acc}_{j-1} + S(m_j),$$

with output $\text{SinsemillaHash}(M) = \text{Acc}_k$, where the double-and-incomplete-add step is the elliptic-curve operation that the arithmetic circuit verifies in very few constraints. Unrolling the recurrence exhibits the hash as a fixed positional combination

$$\text{SinsemillaHash}(M) \;=\; \sum_{j=1}^{k} \big[2^{\,k-j}\big]\, S(m_j) \;+\; \big[2^k\big]\, Q,$$

whose integer coefficients are powers of two determined by position alone; the message enters only through *which* generator $S(m_j)$ each piece selects.

Proposition 7.26 (Collision resistance of Sinsemilla reduces to DLog)**.** *Modelling the generator-derivation map $m \mapsto S(m)$ and the base point Q as independent uniform generators (a "nothing-up-my-sleeve" hash-to-curve, which derives Q as well as the table), any collision in* SinsemillaHash *yields a nontrivial discrete-log relation among the generators $\{S(m)\} \cup \{Q\}$, and hence–via the same every-generator embedding as Proposition 7.25, extended to include Q–a discrete logarithm. Thus Sinsemilla is collision resistant under the discrete logarithm assumption in $\mathbb{G}$.*

Proof. Sketch. Consider first colliding messages $M \neq M'$ of equal piece-count k: the collision equates two combinations of the unrolled form; subtracting, the constant $[2^k]Q$ cancels and one obtains

$$\sum_{j=1}^{k} \big[2^{k-j}\big]\big(S(m_j) - S(m'_j)\big) \;=\; \mathcal{O},$$

a relation whose coefficient on each table generator $S(v)$ aggregates the positions where M, respectively M', carries chunk value v: writing $c_v = \sum_{j\,:\,m_j=v} 2^{k-j} - \sum_{j\,:\,m'_j=v} 2^{k-j}$, the relation reads $\sum_v [c_v]\,S(v) = \mathcal{O}$ over the *distinct* generators $S(v)$. Each c_v is a difference of two subset-sums of the distinct powers $2^{k-1}, \ldots, 2^0$, hence an integer of absolute value below 2^k. For admissible message lengths (2^k below the group order, a finite design-time check) c_v vanishes modulo the group order only if it vanishes as an integer, and by uniqueness of binary expansions $c_v = 0$ forces the two position sets to coincide; coincidence for every v forces $M = M'$ (the messages have equal piece-count k in this case). Since $M \neq M'$, some $S(v)$ carries a nonzero net coefficient and the relation is nontrivial. For piece-counts $k > k'$ the constants do not cancel: the relation acquires the coefficient $2^k - 2^{k'}$ on Q, a nonzero integer below the group order for admissible lengths, so the collision yields a nontrivial relation among $\{S(v)\} \cup \{Q\}$. Embedding a DLog challenge in every table generator and in Q, exactly as in Theorem 7.22, applied to the relation over the distinct generators with their net coefficients, recovers the challenge logarithm. □

In deployment the final piece is zero-padded to the piece width, so the deployed SinsemillaHash is collision resistant only between inputs of a fixed length for a given personalisation — exactly the property the protocol specification requires; the variable-length argument above applies to the piece-aligned presentation given here.

To turn the hash into a *hiding* commitment one appends a Pedersen blinding term: fix one further independent generator H' (hashed to the curve, with no known discrete-log relation to Q or the table) and define

$$\text{SinsemillaCommit}(m;\, r) \;=\; \text{SinsemillaHash}(m) \;+\; [r]\,H', \qquad r \xleftarrow{\$} \mathbb{F}_p.$$

Hiding is inherited verbatim from Pedersen: $[r]H'$ is uniform on $\mathbb{G}$ for uniform r, so the commitment's distribution is independent of m (Theorem 7.16). Binding rests on discrete log: two valid openings to distinct messages yield either a collision in SinsemillaHash (Proposition 7.26) or a nontrivial discrete-log relation between H' and the hash generators. Orchard's note commitment NoteCommit instantiates exactly this map; its key commitment CommitIvk composes it with the x-coordinate extraction (the specification's SinsemillaShortCommit), returning a base-field element rather than a point.

Remark 7.27 (Significance for circuits)*.* Sinsemilla's advantage is *verification cost inside a proof*. In a SNARK, the prover must re-express every step it takes as polynomial constraints; a bit-oriented hash such as SHA-256 costs tens of thousands of constraints, whereas Sinsemilla's per-piece cost is a handful of curve operations chosen to lie in the same field the proof system already works over. One obtains a collision-resistant hash and commitment (used for note commitments and Merkle trees in Orchard) that is simultaneously cheap to prove and grounded in the same discrete-log hardness as the surrounding protocol. The presentation above is the conceptual core; the *Ironwood Guide* instantiates it concretely (piece width, the incomplete-addition caveats, and the exact domain strings).

7.8 Polynomial commitment schemes

This section concludes with the family of commitments that underlies modern succinct arguments. A *polynomial commitment scheme* (PCS) lets a prover commit to a polynomial f with a short string, and later convince a verifier that $f(z) = y$ for a queried point z–again with a short proof–without revealing f. Concisely: commit to a polynomial, and later prove an evaluation.

Definition 7.28 (Polynomial commitment scheme)**.** Fix a field $\mathbb{F}$ and a degree bound d. A *polynomial commitment scheme* is a tuple (Setup, Commit, Open, Eval, Verify):

- $\mathsf{Setup}(1^\lambda, d) \to \mathsf{pp}$, public parameters supporting degree $\le d$.
- $\mathsf{Commit}(\mathsf{pp}, f; r) \to C$, committing to $f \in \mathbb{F}[X]_{\le d}$ (optionally with hiding randomness r).
- $\mathsf{Open}(\mathsf{pp}, C, f, r) \to \{0,1\}$, verifying that C is a commitment to the whole polynomial f.
- $\mathsf{Eval}(\mathsf{pp}, f, r, z) \to (y, \pi)$, producing the claimed value $y = f(z)$ and an *evaluation proof* π for a point $z \in \mathbb{F}$.
- $\mathsf{Verify}(\mathsf{pp}, C, z, y, \pi) \to \{0,1\}$, checking the evaluation proof against the commitment.

Beyond the hiding and binding of the underlying commitment, a PCS must satisfy *evaluation binding*: no PPT prover can produce a commitment C, a point z, and two accepting proofs for distinct claimed values $y \neq y'$ at z, except with negligible probability. Strong PCSs additionally satisfy an *extractability* (proof-of-knowledge) property: from a successful prover one can extract an actual polynomial f of degree $\le d$ consistent with all accepted evaluations.

A single committed value C admits "probing" at arbitrary points z because of the rigidity of polynomials: a nonzero polynomial of degree $\le d$ has at most d roots, so two distinct degree-$\le d$ polynomials agree on at most d points. Evaluation binding leverages this rigidity: answering a different value at even one point is the behaviour of a prover holding a different polynomial. Making this intuition stand is a separate cryptographic requirement–an accepting evaluation proof is not an opening of C, so evaluation binding does not follow from the binding of Commit; each construction proves it from its own assumption (d-strong Diffie–Hellman for KZG, discrete log via the folding extractor for the IPA). The central design problem is to make the *evaluation proof* π short and fast to verify. Two families dominate.

The pairing-based family: KZG

The Kate–Zaverucha–Goldberg (KZG) scheme commits to $f = \sum_{i=0}^{d} a_i X^i$ by evaluating it "in the exponent" at a secret point τ.

Definition 7.29 (KZG commitment)**.** Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order p with a nondegenerate bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, and generators $G \in \mathbb{G}_1$, $\widehat{G} \in \mathbb{G}_2$. Setup samples a secret $\tau \xleftarrow{\$} \mathbb{F}_p$ and publishes the *structured reference string*

$$\mathsf{pp} = \Big([\tau^0]G,\ [\tau^1]G,\ \ldots,\ [\tau^d]G,\ \ \widehat{G},\ [\tau]\widehat{G} \Big),$$

discarding τ. The commitment to f is the evaluation in the exponent,

$$\mathsf{Commit}(\mathsf{pp}, f) = \sum_{i=0}^{d} [a_i]\big([\tau^i]G\big) = [f(\tau)]\,G,$$

computable from pp without knowing τ.

To prove $f(z) = y$, the prover uses the polynomial identity that z is a root of $f(X) - y$: there is a quotient $q(X) = \dfrac{f(X) - y}{X - z} \in \mathbb{F}[X]_{\leq d-1}$ (a genuine polynomial precisely because $f(z) - y = 0$). The proof is the commitment to the quotient, $\pi = [\, q(\tau)\,]G$, and the verifier checks the pairing equation

$$e\big(\, C - [y]G,\ \widehat{G}\,\big) \stackrel{?}{=} e\big(\, \pi,\ [\tau]\widehat{G} - [z]\widehat{G}\,\big),$$

which, by bilinearity, is exactly the "in the exponent" form of the identity $f(\tau) - y = q(\tau)\,(\tau - z)$.

Evaluation binding rests on the d-strong Diffie–Hellman assumption in the bilinear group (given the powers $[\tau^i]G$, it is infeasible to produce $[(\tau - z)^{-1}]G$ for any z of one's choosing). The reduction is one line: two accepting proofs π, π' for distinct values $y \neq y'$ at the same point z satisfy, after subtracting the two pairing equations, $\pi - \pi' = [(y' - y)(\tau - z)^{-1}]G$, so the forger's output yields $[(\tau - z)^{-1}]G = [(y' - y)^{-1}](\pi - \pi')$, exactly the forbidden element.

Remark 7.30 (Trade-offs of KZG). KZG is notable for *constant-size* commitments and proofs (one and one group element, respectively) and *constant-time* verification (two pairings), *independent of the degree d*. Its cost is twofold. First, it requires a *trusted setup*: the secret τ is a global trapdoor; anyone who learns it can forge evaluation proofs for false statements, so one must run Setup so that τ is provably destroyed (in practice by a multi-party "powers-of-tau" ceremony in which the trapdoor stays secret as long as one participant is honest). Second, it requires pairing-friendly curves, a more delicate and less efficient setting than ordinary elliptic curves. In the spirit of Theorem 7.7, binding here is computational (it rests on the bilinear assumption).

The inner-product-argument family

The second family dispenses with pairings and with trusted setup, at the cost of larger proofs and slower verification. Its commitment is exactly the Pedersen vector commitment of Subsection 7.6, applied to the coefficient vector.

Definition 7.31 (IPA-style polynomial commitment)**.** With transparent generators $\mathrm{G} = (G_0, \ldots, G_d)$ and blinding generator H (all with unknown mutual discrete logs, derived from a public seed), commit to $f = \sum_{i=0}^{d} a_i X^i$ by the Pedersen vector commitment of its coefficients,

$$\mathsf{Commit}(\mathsf{pp}, f; r) \;=\; \sum_{i=0}^{d} [a_i]\, G_i \;+\; [r]\, H \;=\; \langle \mathrm{a},\, \mathrm{G} \rangle + [r]H.$$

The evaluation $f(z) = \sum_i a_i z^i = \langle \mathrm{a},\, \mathrm{z} \rangle$ is an *inner product* of the coefficient vector a with the public vector $\mathrm{z} = (1, z, z^2, \ldots, z^d)$ of powers of the query point. Proving an evaluation thus reduces to proving an inner-product relation about a committed vector. The *inner-product argument* accomplishes this with a recursive "folding" protocol: at each round it halves the dimension of the vectors and of the generator set by taking a random linear combination, sending two group elements per round, until a single scalar remains. After $\log_2(d+1)$ rounds the verifier checks one final equation.

Proposition 7.32 (Properties of the IPA-based PCS)**.** *The IPA-based polynomial commitment has* transparent setup *(public-coin parameters, no trapdoor),* logarithmic-size *evaluation proofs ($O(\log d)$ group elements), and* linear-time *verification ($O(d)$ group operations, though this can be batched/amortised). Its binding and evaluation binding reduce to the discrete logarithm assumption, inherited from the Pedersen vector commitment.*

Proof. Sketch. Binding of the commitment is Theorem 7.22. For evaluation binding and extractability, one shows the folding protocol is *tree special-sound* (Definition 9.15): from a suitable *tree of accepting transcripts*, branching at each round's challenge, an extractor computes a coefficient vector a consistent with the commitment and the claimed inner product; two distinct extracted openings of the same commitment would yield a nontrivial discrete-log relation among G, H, contradicting DLog. The $O(\log d)$ proof size follows immediately from the halving recursion; the round messages are the two "cross-term" commitments L_j, R_j produced at each fold. □

Remark 7.33 (Contrast and the Halo lineage). The two families embody a clean trade-off. KZG: constant proof and verification, but a structured trusted setup and pairings. IPA: transparent setup over a plain curve and only discrete-log hardness, but logarithmic proofs and linear verification. The Halo 2 system used in Zcash Orchard builds on the inner-product family precisely to avoid trusted setup, and recovers practical verification cost through *batch verification*: the deployed verifier folds the final multiscalar multiplications of many proofs into one shared multiexponentiation, amortising the per-proof group-operation cost, though the per-proof verifier work stays linear. The Halo lineage's "accumulation"/recursion technique extends the same deferral across recursively composed proofs—one proof attesting to another's deferred check—but deployed Zcash does not use recursion. For this reason the inner-product commitment, despite its asymptotically heavier verification, is the appropriate foundation there.

Remark 7.34 (Hiding variants and the role of blinding). In both families one obtains a hiding variant by adding a Pedersen-style blinding term: one can augment KZG with a $[r]\,[\gamma]G$ summand using an extra setup element $[\gamma]G$, and the IPA commitment already carries its $[r]H$. Without blinding, the schemes are binding but not hiding–the commitment is a deterministic function of f–which is acceptable when the polynomial encodes only public data, but must be repaired with randomness whenever the polynomial encodes secrets, as in the witness polynomials of a zero-knowledge proof. The interplay is once more the perfect-hiding/computational-binding regime of Pedersen, now lifted to polynomials.

Remark 7.35 (From commitments to arguments). Polynomial commitments are the linchpin that turns an *information-theoretic* proof object (a polynomial identity that holds over a large field) into a *succinct cryptographic argument*: the prover commits to its witness polynomials, the verifier challenges at random points, and the polynomial commitment's evaluation proofs convince the verifier that the committed polynomials satisfy the required identities–without revealing them. The Schwartz–Zippel lemma supplies the soundness (a nonzero low-degree polynomial is almost surely nonzero at a random point), and the commitment supplies the binding and the succinctness. This is the bridge from the elementary commitments of this section to the full argument systems developed in the sequel.

8 Digital signatures

A digital signature is the public-key analogue of a handwritten signature: it lets the holder of a secret key produce, for any message, a short string that anyone holding the corresponding public key can check, and that nobody lacking the secret key can produce. Unlike a message authentication code, which requires the verifier to share the signer's secret, a signature is *publicly verifiable* and *non-repudiable*. Signatures are the workhorse of authenticated communication and, in the setting that motivates this monograph, the mechanism by which the spender of a note authorises a transaction. The Zcash Orchard protocol uses two closely related Schnorr-type schemes built over an elliptic curve: a *spend authorisation* signature whose key can be re-randomised so that successive uses are unlinkable, and a *binding* signature that simultaneously certifies the consistency of a transaction's value commitments and proves knowledge of a discrete logarithm. Both constructions derive from one classical object, the Schnorr identification protocol, so we develop the entire account from there.

Throughout this section $\mathbb{G}$ denotes a cyclic group of prime order q, written additively, with a fixed generator G; the reader should keep in mind the example where $\mathbb{G}$ is the group of points of an elliptic curve over a finite field. We write scalar multiplication as $[x]P$ for $x \in \mathbb{Z}$ (equivalently $x \in \mathbb{F}_q$, since $\mathbb{G}$ has order q) and $P \in \mathbb{G}$. The *discrete logarithm* of P to base G is the unique $x \in \mathbb{F}_q$ with $P = [x]G$, when it exists; in a group of prime order generated by G it always exists and is unique. We assume the reader is comfortable with the asymptotic and probabilistic language (poly, negl, security parameter λ) developed for the other primitives.

8.1 Syntax and security goal

The exposition begins abstractly, fixing what a signature scheme *is* before asking what it means for one to be secure.

Definition 8.1 (Signature scheme)**.** A *digital signature scheme* is a triple of probabilistic polynomial-time algorithms $\Pi = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ together with, for each security parameter $\lambda \in \mathbb{N}$, a message space $\mathcal{M}_\lambda$, such that:

- $\mathsf{KeyGen}(1^\lambda)$ outputs a pair (pk, sk) consisting of a *public* (or *verification*) key and a *secret* (or *signing*) key;
- $\mathsf{Sign}(sk, m)$, for $m \in \mathcal{M}_\lambda$, outputs a *signature* σ;
- $\mathsf{Verify}(pk, m, \sigma)$ is deterministic and outputs a bit, 1 for *accept* and 0 for *reject*.

We require *correctness*: for every λ, every (pk, sk) in the support of $\mathsf{KeyGen}(1^\lambda)$, and every $m \in \mathcal{M}_\lambda$,

$$\Pr\big[\, \mathsf{Verify}(pk, m, \mathsf{Sign}(sk, m)) = 1 \,\big] = 1,$$

the probability taken over the coins of Sign (and, if applicable, Verify, though we take the latter to be deterministic). When the equality holds with probability 1 one speaks of *perfect* correctness; some schemes only achieve correctness with overwhelming probability, in which case we tolerate a $\mathrm{negl}(\lambda)$ failure rate.

Correctness states that honestly generated signatures verify. It says nothing about an adversary, and the entire interest of the subject lies in pinning down what an adversary should be unable to do. The strongest standard notion, and the one to which we hold every construction, is *existential unforgeability under an adaptive chosen-message attack*. We hand the adversary the public key and a *signing oracle*: a black box that will sign any message of the adversary's choosing, as often as required, adaptively. The adversary wins if it can output a valid signature on *any* message it never submitted to the oracle.

Definition 8.2 (EUF-CMA)**.** Let $\Pi = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ be a signature scheme and $\mathcal{A}$ an algorithm. Define the experiment $\mathsf{Forge}_{\mathcal{A},\Pi}(\lambda)$:

1. Run $(pk, sk) \leftarrow \mathsf{KeyGen}(1^\lambda)$.
2. Run $\mathcal{A}$ on input pk with oracle access to $\mathsf{Sign}(sk, \cdot)$. Let Q be the set of messages $\mathcal{A}$ submits to the oracle. Let (m^*, σ^*) be $\mathcal{A}$'s output.
3. The experiment outputs 1 iff $\mathsf{Verify}(pk, m^*, \sigma^*) = 1$ and $m^* \notin Q$.

We say Π is *existentially unforgeable under chosen-message attack (EUF-CMA)* if for every probabilistic polynomial-time $\mathcal{A}$ there is a negligible function negl with

$$\Pr\big[\, \mathsf{Forge}_{\mathcal{A},\Pi}(\lambda) = 1 \,\big] \le \mathrm{negl}(\lambda).$$

Remark 8.3 (Strong unforgeability)*.* A subtle but important strengthening is *strong* unforgeability (sUF-CMA), in which "(m^*, σ^*) was not among the pairs returned by the oracle" replaces the winning condition $m^* \notin Q$. Strong unforgeability forbids the adversary even from producing a *new* signature on an *already-signed* message. This matters whenever a signature itself serves as a unique token; for instance, certain anti-replay mechanisms break if signatures are malleable. Plain EUF-CMA permits a scheme in which, given one valid signature, one can cheaply compute a second, different valid signature on the same message. We note below which of these schemes are strongly unforgeable.

The remainder of the section constructs, from scratch, a concrete scheme meeting Definition 8.2 (and, with care, Remark 8.3) in an idealised model, and then enriches it with the re-randomisation and binding features the application requires.

8.2 The hash-and-sign paradigm

Before specialising to Schnorr, we record the single most important structuring idea in practical signature design, since it explains why every scheme below begins by hashing the message. Most "textbook" signature mechanisms (RSA, Schnorr's identification-derived signature, ECDSA) are naturally defined only on inputs of a fixed, bounded size: an element of $\mathbb{Z}_N^*$ for RSA, a scalar in $\mathbb{F}_q$ for Schnorr and ECDSA alike. Real messages are arbitrarily long. The *hash-and-sign* paradigm bridges the gap with a cryptographic hash function.

Let $\Pi' = (\mathsf{KeyGen}, \mathsf{Sign}', \mathsf{Verify}')$ be a scheme with message space $\mathcal{Y}_\lambda$ and let $H : \{0,1\}^* \to \mathcal{Y}_\lambda$ be drawn from a collision-resistant family (Definition 3.6). *Hash-and-sign* defines $\mathsf{Sign}(sk, m) = \mathsf{Sign}'(sk, H(m))$ and $\mathsf{Verify}(pk, m, \sigma) = \mathsf{Verify}'(pk, H(m), \sigma)$, extending the message space to all of $\{0,1\}^*$ at the cost of one hash. The composition preserves EUF-CMA security, by a two-case argument: a forgery on a fresh message m^* either reuses the digest of some queried message—exhibiting a collision in H—or presents a fresh digest, which is itself a forgery against Π'; hence $\Pr[\mathcal{A}\text{ wins}] \le \mathsf{Adv}^{\mathrm{cr}}_H + \mathsf{Adv}^{\mathrm{euf}}_{\Pi'}$.

Remark 8.4. In all the Schnorr-type schemes below, the hash that absorbs the message also absorbs the public key and the per-signature "commitment" value. Hashing the public key prevents *key-substitution* (duplicate-signature) attacks, and hashing the commitment is exactly what the Fiat–Shamir transform requires; Section 8.4 develops this carefully. Consequently, "hash-and-sign" in practice means hashing more than the message, but the composition above captures the message-compression part of the story.

8.3 The Schnorr identification protocol

An *identification protocol* lets a prover P convince a verifier V that it knows a secret associated with a public key, *without revealing the secret*. Schnorr's protocol is the canonical example of a Σ-*protocol*: a

three-move, public-coin, honest-verifier-zero-knowledge proof of knowledge. We extract a signature from it in the next subsection, but we first study it in its own right, because the protocol confers every security property of the signature, almost mechanically, on the signature in turn.

The setup fixes $\mathbb{G} = \langle G \rangle$ of prime order q. The prover's secret key is a scalar $x \in \mathbb{F}_q$ and the public key is $X = [x]G \in \mathbb{G}$. The relation being proved is

$$\mathcal{R}_{\mathrm{dl}} = \{\, (X, x) \in \mathbb{G} \times \mathbb{F}_q \;:\; X = [x]G \,\},$$

the discrete-logarithm relation. The protocol runs as follows.

Definition 8.5 (Schnorr identification)**.** On common input $X \in \mathbb{G}$, with the prover additionally holding x such that $X = [x]G$:

1. **Commitment.** The prover samples $r \leftarrow \mathbb{F}_q$ uniformly, computes $R = [r]G$, and sends R to the verifier. (R is the *commitment* or *nonce point*; r is the *nonce*.)
2. **Challenge.** The verifier samples $c \leftarrow \mathbb{F}_q$ uniformly and sends c.
3. **Response.** The prover computes $s = r + c\,x \in \mathbb{F}_q$ and sends s.

The verifier *accepts* iff

$$[s]G = R + [c]X. \tag{$*$}$$

We call a triple (R, c, s) satisfying $(*)$ an *accepting transcript* for X.

The three messages (R, c, s) follow the *commit–challenge–response* pattern; the first and last come from the prover, the middle is the verifier's "coin." Because the verifier's only message is a uniformly random value independent of R, the protocol is *public-coin*. We now verify the three defining properties of a Σ-protocol.

8.3.1 Perfect completeness

Proposition 8.6 (Perfect completeness)**.** *If the prover and verifier follow the protocol of Definition 8.5 honestly, the verifier accepts with probability* 1.

Proof. With $R = [r]G$, $X = [x]G$, and $s = r + cx$, linearity of scalar multiplication gives

$$[s]G = [r + cx]G = [r]G + [cx]G = R + [c]\,[x]G = R + [c]X,$$

which is exactly the verification equation $(*)$. The argument uses no randomness beyond what the honest parties already chose and holds identically for every choice of r and c; hence acceptance is certain. □

8.3.2 Special soundness

Completeness protects the honest prover. *Soundness* protects the verifier: a cheating prover who does *not* know x should fail. For Σ-protocols the appropriate notion is the very strong *special soundness*, which states that any two accepting transcripts that share the first message but differ in the challenge *reveal the witness*. This is the central mechanism: it makes the protocol a genuine proof *of knowledge*, and after Fiat–Shamir it becomes the source of unforgeability.

Proposition 8.7 (2-special-soundness)**.** *There is an efficient* extractor *that, given two accepting tran-*

> *scripts (R, c, s) and (R, c', s') for the same statement X and the same commitment R but with $c \neq c'$, outputs the discrete logarithm x of X.*

Proof. Both transcripts accept, so by $(*)$

$$[s]G = R + [c]X \qquad \text{and} \qquad [s']G = R + [c']X.$$

Subtracting (in $\mathbb{G}$) cancels R:

$$[s - s']G = [c - c']X = [(c - c')x]G.$$

Since G generates the group of prime order q, equality of $[s-s']G$ and $[(c-c')x]G$ forces $s-s' \equiv (c-c')x \pmod{q}$. As $c \neq c'$ in the field $\mathbb{F}_q$, the element $c - c'$ is invertible, and the extractor outputs

$$x = \frac{s - s'}{c - c'} \in \mathbb{F}_q.$$

One verifies $X = [x]G$ directly: $[s - s']G = [c - c']X$, and with $x = (s - s')(c - c')^{-1}$,

$$[x]G = \left[(c - c')^{-1}\right][s - s']G = \left[(c - c')^{-1}\right][c - c']X = X.$$

The computation is a single field inversion and is plainly efficient. □

Remark 8.8 (From special soundness to a proof of knowledge). Special soundness implies the standard *knowledge soundness* guarantee. Suppose a (possibly cheating) prover convinces the verifier with probability ε noticeably larger than $1/q$ (the chance of guessing a single challenge). Rewind it: run it to obtain R, feed challenge c and record whether it answers; rewind to just after it output R and feed a fresh independent c'. A standard averaging argument (the "heavy-row" lemma, an instance of Markov's inequality — *Math Guide*, §"Random variables and expectation") shows that with probability roughly ε^2 one obtains two accepting transcripts sharing R with $c \neq c'$, whereupon the extractor of Proposition 8.7 recovers x. Thus knowing how to make the verifier accept with non-trivial probability is, up to rewinding, the same as knowing x: the protocol is a *proof of knowledge* of the discrete logarithm. This rewinding argument recurs, sharpened into the forking lemma, in the analysis of the signature.

8.3.3 Honest-verifier zero knowledge

Finally, the prover must leak nothing about x. The clean formalisation for public-coin protocols is *honest-verifier zero knowledge*: there is a simulator that, *without the witness*, produces transcripts identically distributed to real ones, provided the challenge is the honest (uniform) one.

> **Proposition 8.9** (Special honest-verifier zero knowledge)**.** *There is an efficient simulator* Sim *that, on input a statement $X \in \mathbb{G}$ and a challenge $c \in \mathbb{F}_q$, outputs a transcript (R, c, s) whose distribution is identical to that of an honest execution conditioned on the challenge being c. In particular, for a uniformly random c the simulated and real transcript distributions coincide exactly.*

Proof. The simulator samples $s \leftarrow \mathbb{F}_q$ uniformly, sets

$$R := [s]G - [c]X,$$

and outputs (R, c, s). By construction $[s]G = R + [c]X$, so $(*)$ holds and the transcript accepts. It remains to match distributions.

In a *real* honest transcript with challenge fixed to c: the prover picks $r \leftarrow \mathbb{F}_q$ uniformly, sets $R = [r]G$, and $s = r + cx$. The map $r \mapsto s = r + cx$ is a bijection of $\mathbb{F}_q$ (a translation), so s is uniform on $\mathbb{F}_q$, and $R = [s - cx]G = [s]G - [c]X$ is then a deterministic function of s. Therefore the real transcript is exactly: sample s uniform, set $R = [s]G - [c]X$. That is precisely the simulator's output distribution. Hence the two distributions are identical, not merely statistically close. Averaging over a uniform c preserves identity of distributions, giving the final claim. □

Remark 8.10 (On the restriction to *honest* verifiers). The simulator only matches the real distribution when the challenge is sampled *independently of* R (as an honest verifier does). A cheating verifier could try to choose c as a function of R; against such a verifier the simulator, which must commit to R only after seeing c, no longer obviously works. For the present purposes honest-verifier zero knowledge is exactly sufficient, because after Fiat–Shamir a hash function applied to R produces the challenge, and in the random oracle model that hash behaves like an honest, R-independent coin. The deeper point is that the simulator's existence already proves the transcript carries *no information* about x beyond what X itself reveals: anyone could have manufactured it.

In summary, the Schnorr identification protocol is a Σ-protocol for $\mathcal{R}_{\mathrm{dl}}$: perfectly complete (Proposition 8.6), 2-special-sound (Proposition 8.7), and special honest-verifier zero knowledge (Proposition 8.9). We now convert these three properties into a signature.

8.4 From identification to signature via the Fiat–Shamir transform

The identification protocol is *interactive*: the verifier must contribute a fresh random challenge. A signature must be *non-interactive* and must bind to a message. The *Fiat–Shamir transform* achieves both simultaneously by replacing the verifier's random challenge with the output of a public hash function applied to the commitment (together with the message and the public key). Heuristically, a hash function "looks random" and resists influence in advance, so it plays the role of the verifier's coin; binding the message into the hash ties the proof to that message.

Definition 8.11 (Schnorr signature). Fix $\mathbb{G} = \langle G \rangle$ of prime order q and a hash function $H : \{0,1\}^* \to \mathbb{F}_q$. The Schnorr signature scheme is:

- $\mathsf{KeyGen}(1^\lambda)$: sample $x \leftarrow \mathbb{F}_q$, set $X = [x]G$; output $(pk, sk) = (X, x)$.
- $\mathsf{Sign}(x, m)$: sample $r \leftarrow \mathbb{F}_q$, compute $R = [r]G$, set $c = H(R \,\|\, X \,\|\, m)$, set $s = r + c\,x \in \mathbb{F}_q$; output $\sigma = (R, s)$.
- $\mathsf{Verify}(X, m, (R, s))$: recompute $c = H(R \,\|\, X \,\|\, m)$; accept iff $[s]G = R + [c]X$.

Two conventions deserve comment. First, the signature transmits (R, s); it omits the challenge c because the verifier recomputes it. A common space-saving variant transmits (c, s) instead, and the verifier recovers $R = [s]G - [c]X$ before checking that $H(R\|X\|m) = c$; the two are interchangeable. Second, the hash takes the public key X alongside R and m. This is the key-prefixing of Remark 8.4; the bare unforgeability proof of a single fixed key does not require it, but it forecloses attacks that relate signatures under different keys and is standard in modern instantiations such as EdDSA.

Proposition 8.12 (Correctness). *The Schnorr signature scheme is perfectly correct.*

Proof. For an honestly produced $\sigma = (R, s)$ with $R = [r]G$, $c = H(R\|X\|m)$, and $s = r + cx$, verification recomputes the same c (it is a deterministic function of R, X, m) and checks $[s]G = R + [c]X$, which holds by the computation in Proposition 8.6. □

The transform is generic: it converts *any* Σ-protocol into a non-interactive proof, and, when a message is folded into the hash, into a signature. We analyse the security of the result in the *random oracle model*, which we set up next.

8.5 Security in the random oracle model and the forking lemma

8.5.1 The random oracle model

The Fiat–Shamir heuristic replaces an interactive challenge with a hash. To *prove* anything we idealise the hash as a *random oracle*: a function H that every party (including the adversary and the reduction)

accesses only as a black box, and which returns, for each distinct input, an independent uniformly random output in its range, consistent across repeated queries.

Definition 8.13 (Random oracle model, recalled)**.** We work in the random oracle model of Definition 3.12, with one generalisation: the oracle is sampled uniformly from the functions $\{0,1\}^* \to \mathcal{Y}$ for an arbitrary finite range $\mathcal{Y}$ (the challenge space), rather than $\{0,1\}^n$ as in Definition 3.11; the lazy-sampling realisation carries over verbatim.

The ROM remains a heuristic, with the uninstantiability caveat of Theorem 3.16 and the reading of Remark 3.17: a ROM proof rules out all attacks that treat the hash as a black box, which constitute the overwhelming majority of attacks, and it is the standard yardstick for Fiat–Shamir signatures. A reduction exploits the two powers of Proposition 3.13 (cf. Remark 1.30): *programmability*, whereby the reduction may choose the oracle's answers on the fly, provided they appear uniform; and *observability*, whereby the reduction sees every query the adversary makes.

8.5.2 Simulation of signatures by programming the oracle

The first benefit of the ROM is that the reduction can simulate the EUF-CMA *signing oracle* without the secret key, using precisely the HVZK simulator of Proposition 8.9. This reduces forgery to a pure discrete-logarithm problem.

Lemma 8.14 (Signature simulation)**.** *In the ROM, there is an efficient simulator that, given only the public key X and the ability to program H, answers signing queries with signatures distributed identically to genuine ones, except that it aborts on a programming collision that occurs with probability at most $(q_s)(q_s + q_h)/q$ over an attack making q_s signing and q_h hash queries.*

Proof. To sign m without x: sample $c \leftarrow \mathbb{F}_q$ and $s \leftarrow \mathbb{F}_q$ uniformly, set $R := [s]G - [c]X$ (the HVZK simulator), and *program* $H(R\|X\|m) := c$. Output (R, s). The triple satisfies $[s]G = R + [c]X$, so it verifies, and by Proposition 8.9 its distribution matches that of a real signature. The simulation fails only if a previous hash or signing query *already defined* the point $H(R\|X\|m)$ to a different value, so that programming would be inconsistent. Since R is uniform in $\mathbb{G}$ of size q (it is $[s]G$ shifted by a fixed point, with s uniform), the adversary queried this particular input previously with probability at most $(q_h + q_s)/q$; union-bounding over the q_s signing queries gives the stated bound. Conditioned on no abort, the view is perfect. □

8.5.3 The forking lemma

The second benefit is *extraction*. A successful forger, by definition, produces a fresh accepting transcript (R, c, s) where the random oracle tied c to $R\|X\|m^*$. If the reduction rewinds the forger and *re-programs* the oracle to answer that one critical query with a fresh challenge c', then by special soundness two successful runs yield x. Making this precise, and quantifying the probability that the second run also succeeds and is "compatible" with the first, is the content of the *forking lemma*. We state the general (game-based) version due to Bellare and Neven and then apply it.

Lemma 8.15 (General forking lemma)**.** *Fix an integer $Q \geq 1$, a set $\mathcal{C}$ of size $|\mathcal{C}| = h \geq 2$, and a randomised input generator* Gen*. Let $\mathcal{B}$ be a randomised algorithm that, on input y and elements $c_1, \ldots, c_Q \in \mathcal{C}$, returns a pair (I, out) where $I \in \{0, 1, \ldots, Q\}$ is an* index *(with $I = 0$ meaning "failure"). Let*

$$acc \;=\; \Pr\big[\, I \geq 1 \;:\; y \leftarrow \mathsf{Gen},\; c_1, \ldots, c_Q \leftarrow \mathcal{C},\; (I, out) \leftarrow \mathcal{B}(y, c_1, \ldots, c_Q) \,\big]$$

be $\mathcal{B}$'s success probability. Consider the forking algorithm $\mathsf{F}_{\mathcal{B}}(y)$*: pick random coins ρ for $\mathcal{B}$ and*

$c_1, \dots, c_Q \leftarrow \mathcal{C}$; run $(I, out) \leftarrow \mathcal{B}(y, c_1, \dots, c_Q; \rho)$; if $I = 0$ return $\perp$; otherwise resample $c'_I, \dots, c'_Q \leftarrow \mathcal{C}$ afresh and run $(I', out') \leftarrow \mathcal{B}(y, c_1, \dots, c_{I-1}, c'_I, \dots, c'_Q; \rho)$ with the same *coins ρ; if $I' = I$ and $c_I \neq c'_I$, return (I, out, out'), else return $\perp$. Define $frk = \Pr\left[\, \mathsf{F}_{\mathcal{B}}(y) \neq \perp \;:\; y \leftarrow \mathsf{Gen} \,\right]$, the probability taken also over $\mathsf{F}_{\mathcal{B}}$'s coins. Then*

$$frk \;\geq\; acc\left(\frac{acc}{Q} - \frac{1}{h}\right).$$

Proof. Sketch. Condition on the random coins ρ and on the prefix $c_1, \dots, c_{I-1}$ up to the forking point. For each fixed value of these, the single "pivotal" challenge c_I (the value returned at the queried index) together with the suffix determines $\mathcal{B}$'s success; abstract this as a function of c_I only by also averaging over the suffix, giving for each prefix a set S of "good" values of $c_I \in \mathcal{C}$ on which $\mathcal{B}$ succeeds with index I. The forking algorithm succeeds at this prefix iff both the first draw c_I and the independent redraw c'_I land in S and differ. Two independent uniform draws from $\mathcal{C}$ both land in S with probability $(|S|/h)^2$, and subtracting the diagonal (equal draws) costs at most $|S|/h \cdot 1/h$; so the per-prefix fork probability is at least $(|S|/h)^2 - |S|/h^2 = (|S|/h)(|S|/h - 1/h)$. Writing $p = |S|/h$ for the prefix's success probability and taking expectations over prefixes, Jensen's inequality applied to $p \mapsto p^2$ (convexity) gives $\mathbb{E}[p^2] \geq (\mathbb{E}[p])^2 = acc^2$; accounting for the index I ranging over Q possibilities introduces the factor $1/Q$, turning this into acc^2/Q. Collecting terms yields $frk \geq acc^2/Q - acc/h$, which is the stated bound. The full argument carefully books these conditional expectations; the convexity step is where the squared probability—and hence the tightness loss—appears. □

Theorem 8.16 (EUF-CMA security of Schnorr signatures)**.** *Model H as a random oracle. If there is an adversary $\mathcal{A}$ running in time t, making at most q_h random-oracle queries and q_s signing queries, that forges a Schnorr signature with probability ε, then there is an algorithm $\mathcal{D}$ running in time $\approx 2t$ that solves the discrete-logarithm problem in $\mathbb{G}$ with probability at least*

$$\varepsilon' \;\geq\; \frac{\varepsilon^2}{q_h + q_s + 1} \;-\; \frac{(q_h + q_s + 1)}{q} \;-\; \frac{q_s(q_h + q_s)}{q}.$$

Consequently, if the discrete-logarithm problem in $\mathbb{G}$ is hard, the Schnorr signature scheme is EUF-CMA in the random oracle model.

Proof. Transform $\mathcal{A}$ into an algorithm $\mathcal{B}$ of the shape required by Lemma 8.15, with input $y = X$ (the discrete-log challenge $X = [x]G$, x unknown; here Gen is the discrete-log instance generator outputting $X = [x]G$ for uniform $x \leftarrow \mathbb{F}_q$) and challenge list $c_1, \dots, c_Q$ where $Q = q_h + q_s + 1$ bounds the number of distinct hash inputs that can become the "relevant" challenge.

Running $\mathcal{B}$. On input X and the list (c_i), $\mathcal{B}$ runs $\mathcal{A}(X)$. It maintains the random-oracle table and answers $\mathcal{A}$'s j-th *distinct* hash query by handing out c_j from the list (so the oracle's answers are the externally supplied, uniformly random c_j). The simulator of Lemma 8.14 answers signing queries, programming the oracle as needed; if a programming collision occurs $\mathcal{B}$ aborts (this is the $q_s(q_h+q_s)/q$ loss). When $\mathcal{A}$ halts with a valid forgery $(m^*, (R^*, s^*))$, let $c^* = H(R^* \| X \| m^*)$. Because the forgery is fresh and valid, with overwhelming probability $\mathcal{A}$ *actually queried* the value c^* to the oracle during the run rather than guessing blindly: if $\mathcal{A}$ never queried $R^* \| X \| m^*$, the value c^* is uniform and unseen by $\mathcal{A}$, so $[s^*]G = R^* + [c^*]X$ holds for the fresh forgery only with probability $1/q$ (the $(q_h + q_s + 1)/q$ term collects this and similar guessing events). Let I be the index in the list at which the oracle answered this critical query; $\mathcal{B}$ outputs (I, out) with $out = (R^*, c^*, s^*, m^*)$. Thus $acc \geq \varepsilon -$ (negligible loss terms).

Forking. Apply $\mathsf{F}_{\mathcal{B}}$. The two runs use the *same coins* ρ and identical challenges $c_1, \dots, c_{I-1}$ up to the I-th query; because ρ and that shared prefix determine the prover's behaviour up to and including producing the commitment R^*, both runs query the *same* critical input $R^* \| X \| m^*$ at index I, but receive *different* answers $c_I = c^* \neq c'^* = c'_I$. When forking succeeds, it yields two accepting transcripts (R^*, c^*, s^*) and (R^*, c'^*, s'^*), with the same commitment and different challenges.

Extraction. Feed these to the special-soundness extractor of Proposition 8.7:

$$x = \frac{s^* - s'^*}{c^* - c'^*} \in \mathbb{F}_q,$$

which is the discrete logarithm of X. Hence $\mathcal{D}$ solves the instance whenever forking succeeds, so $\varepsilon' \geq frk$. Substituting $acc \geq \varepsilon - O((q_h + q_s)/q) - q_s(q_h + q_s)/q$ and $h = q$, $Q = q_h + q_s + 1$ into Lemma 8.15's bound $frk \geq acc^2/Q - acc/h$ and simplifying yields the displayed inequality. □

Remark 8.17 (The tightness loss). The dominant term of the bound is $\varepsilon' \gtrsim \varepsilon^2/q_h$. The reduction is therefore *not tight*: extracting the discrete logarithm requires running the forger *twice* and incurs a *squaring* of the success probability, divided by the number of hash queries. Concretely, if a forger succeeds with probability $\varepsilon = 2^{-40}$ after $q_h = 2^{60}$ hash queries, the reduction only guarantees a discrete-log solver of advantage about $2^{-80}/2^{60} = 2^{-140}$, far weaker than ε. To compensate, one must choose group parameters offering more bits of discrete-log security than the target signature-security level, roughly doubling the security margin demanded of $\mathbb{G}$. This quadratic loss is intrinsic to rewinding-based Fiat–Shamir proofs; tight reductions are known only under stronger assumptions (e.g. the one-more discrete log or DDH with key-prefixing) or in idealised algebraic models. The loss is the cost of converting an *interactive* proof of knowledge into a *non-interactive* signature by forking.

Remark 8.18 (Nonce reuse is fatal). Special soundness has a dark mirror. If a signer ever produces two signatures (R, s) and (R, s') on *different* messages $m \neq m'$ with the *same* commitment R — for instance by reusing the nonce r, or by drawing r from a biased source — then the challenges $c = H(R\|X\|m)$ and $c' = H(R\|X\|m')$ differ, and *anyone* can apply the extractor of Proposition 8.7 to recover the secret key $x = (s - s')/(c - c')$. Security therefore rests entirely on the nonce being fresh and unpredictable for every signature. This motivates *deterministic* nonce generation, the defining idea of EdDSA below.

8.6 RedDSA: re-randomisable Schnorr for Orchard

We reach the variant used in Zcash by way of its immediate ancestor. *EdDSA* (the Edwards-curve Digital Signature Algorithm) is the engineering refinement of Schnorr that modern systems standardise. It retains the algebra of Definition 8.11 and hardens its two implementation surfaces: the *nonce*, which EdDSA derives deterministically as a hash of a secret prefix and the message—so that no faulty random-number generator can ever repeat a nonce and leak the key (Remark 8.18)—and the *challenge hash*, which it key-prefixes by absorbing the public key alongside R and m (Remark 8.4). In the random oracle model the deterministic nonce is, to anyone ignorant of the secret prefix, a fresh uniform value per message, so the unforgeability proof of Theorem 8.16 carries over essentially unchanged.

RedDSA is EdDSA generalised in two directions: the base point is a *parameter* of the scheme (so that different uses can sign with respect to different generators), and the key may be *re-randomised*. The first generalisation is cosmetic but essential for the binding-signature application; the second is the crux of Orchard's privacy.

Definition 8.19 (RedDSA, schematic)**.** RedDSA is parameterised by a generator $P \in \mathbb{G}$ of the prime-order group. KeyGen samples $x \leftarrow \mathbb{F}_q$ and sets the public key $X = [x]P$. To sign m under x:

1. sample a nonce r (in Zcash, $r := H(T \,\|\, X \,\|\, m)$ for a fresh random byte string T, i.e. *hedged*-deterministic);
2. $R := [r]P$;
3. $c := H(R \,\|\, X \,\|\, m) \in \mathbb{F}_q$;
4. $s := r + c\,x$; output $\sigma = (R, s)$.

Verification accepts iff $[s]P = R + [c]X$. Orchard instantiates RedDSA twice, both times as `RedPallas`—RedDSA over the Pallas curve—and the two instantiations differ *only in the base point*: the *spend-authorisation* base point for the re-randomisable signature, and the *value-commitment randomness* base point for the binding signature below. (The curve varies only across protocol generations: different curves is what distinguishes Sapling's `RedJubjub`, over Jubjub, from Orchard's `RedPallas`.)

Functionally RedDSA is, for a fixed base point, the Schnorr/EdDSA scheme of the previous subsections, and its EUF-CMA security in the ROM is again Theorem 8.16 with G replaced by P. The novelty is the re-randomisation operation, which we treat next.

8.7 Key re-randomisation and unlinkability

In a shielded transaction the spender must prove authority to spend a note. If the same long-term public key appeared on chain each time, an observer could link all spends by a given key, destroying privacy. Orchard's solution is to publish, for each spend, a *fresh re-randomised* public key rk derived from the long-term pk by a random additive shift, to sign with the correspondingly shifted secret key, and to prove in zero knowledge that rk is a valid re-randomisation of an authorised pk. The signature scheme must therefore support re-randomisation *algebraically*, and the re-randomised keys must be *unlinkable*.

Definition 8.20 (Key re-randomisation)**.** Fix the base point P. Given a secret key x with public key $X = [x]P$ and a *randomiser* $\alpha \in \mathbb{F}_q$, define

$$rsk \;=\; x + \alpha \in \mathbb{F}_q, \qquad rk \;=\; X + [\alpha]P \in \mathbb{G}.$$

These are the re-randomised signing and verification keys.

Proposition 8.21 (Re-randomisation is consistent)**.** *The pair (rsk, rk) of Definition 8.20 is a valid RedDSA key pair, i.e. $rk = [rsk]P$. Consequently a RedDSA signature produced under rsk verifies under rk.*

Proof. Directly,

$$[rsk]P = [x + \alpha]P = [x]P + [\alpha]P = X + [\alpha]P = rk,$$

using linearity of scalar multiplication. Thus (rsk, rk) is exactly a key pair of the form KeyGen produces, and signing/verification correctness (Proposition 8.12) applies verbatim. □

The additive structure is what makes this work: because the public key is a *homomorphic* image $[\cdot]P$ of the scalar, shifting the secret by α shifts the public key by $[\alpha]P$, a quantity computable from α and P alone, *without* the secret key. This is precisely why a verifier (or a circuit) can take rk and α and check the relationship, while only the spender, knowing x, can sign.

We now formalise the privacy guarantee. The randomiser α is sampled *uniformly and freshly* for each use and kept secret (in Orchard it is derived inside the spend and revealed only to the circuit). The claim is that rk then carries *no information whatsoever* about which long-term key it came from.

Proposition 8.22 (Perfect unlinkability)**.** *Fix the base point P. For any public key $X \in \mathbb{G}$, if $\alpha \leftarrow \mathbb{F}_q$ is uniform, then $rk = X + [\alpha]P$ is uniformly distributed over $\mathbb{G}$. Consequently, for* any two *public keys $X_0, X_1 \in \mathbb{G}$, the distributions of rk obtained by re-randomising X_0 and by re-randomising X_1 with a fresh uniform randomiser are* identical*. An adversary, even one of unbounded computational power and even one who chose X_0, X_1, cannot distinguish which key a given rk re-randomises with*

probability better than 1/2.

Proof. Since P generates $\mathbb{G}$ of order q, the map $\alpha \mapsto [\alpha]P$ is a bijection $\mathbb{F}_q \to \mathbb{G}$; hence $[\alpha]P$ is uniform over $\mathbb{G}$ when α is uniform over $\mathbb{F}_q$. Translation by the fixed element X is a bijection of the group $\mathbb{G}$, and a bijection carries the uniform distribution to the uniform distribution; therefore $rk = X + [\alpha]P$ is uniform over $\mathbb{G}$, *independently of* X. The distribution of rk is thus the uniform distribution on $\mathbb{G}$ regardless of which X one used, so re-randomisations of X_0 and of X_1 are identically distributed. No test can tell two identical distributions apart; the best any distinguisher can do on the resulting indistinguishability game is to guess, succeeding with probability exactly 1/2. □

Remark 8.23 (What unlinkability does and does not give). Proposition 8.22 is an *information-theoretic* (perfect) statement: it does not rest on any computational assumption. It states, however, only that the re-randomised *key* leaks nothing; the full unlinkability of a spend further requires that the accompanying signature and zero-knowledge proof leak nothing beyond their statements, that the randomiser α stay secret, and that no other transaction field correlate two spends. In Orchard, the protocol publishes rk while proving inside the Halo 2 circuit the witness that it equals $X + [\alpha]P$ for an *authorised* X; the verifier checks the spend authorisation signature against the public rk. The signer must of course know $rsk = x + \alpha$, which requires knowing both x and α; thus only the legitimate key-holder, in possession of α for this spend, can sign under rk. One must also verify that signing under re-randomised keys does not itself harm unforgeability. A forgery (R, s) under rk satisfies $[s]P = R + [c]rk$ with $c = H(R\|rk\|m)$, and subtracting $[\alpha]P$ from the verification equation gives $[s - c\alpha]P = R + [c]X$ — but this identity holds only with $c = H(R\|rk\|m)$, whereas the base verifier recomputes $H(R\|X\|m)$, so the plain subtraction converts forgeries only for the non-key-prefixed variant of Schnorr. For the key-prefixed scheme of Definition 8.19 one instead proves the security of signing under re-randomised keys in the ROM by either (i) observing that $(rsk, rk) = (x + \alpha,\ X + [\alpha]P)$ is itself a base-scheme key pair whose public key is uniform (Proposition 8.22), so a forger against rk is literally a forger against the base scheme at the key rk, and the reduction, which knows α, recovers x from the extracted rsk as $x = rsk - \alpha$; or (ii) when the adversary sees signatures under several randomisers α_i, by re-running the forking argument of Theorem 8.16 on the critical query $H(R^*\|rk\|m^*)$ — simulating signing under the remaining keys by the oracle-programming of Lemma 8.14 — to extract rsk and hence x. (A naive ROM translation that programs $H(R\|rk\|m) := H_{\text{base}}(R\|X\|m)$ must be a careful injective domain swap and fails outright with multiple randomisers, since distinct rk_i-prefixed inputs would collide on one X-prefixed point, a detectable deviation from a random oracle.)

8.8 Binding signatures as a proof of knowledge of a discrete logarithm

The final piece is the *binding signature*, an unusually elegant use of a Schnorr signature in which the "message-authenticating" role is incidental and the substantive task is to prove an algebraic balance: that the values flowing into a transaction equal the values flowing out. The construction arranges so that the binding-signature *verification key is itself a commitment* that the verifier can recompute, and that key equals $[\cdot]P$ of a secret the signer can know only if the transaction balances. A valid signature is then exactly a proof of knowledge of that discrete logarithm.

We sketch the value-commitment setup that motivates the construction. The shielded protocols commit to value with a homomorphic Pedersen commitment

$$\mathrm{cm}(v, \rho) = [v]\,V + [\rho]\,P,$$

where V and P are two independent generators of $\mathbb{G}$ (independent meaning that no party knows the discrete log of one to base the other) and ρ is a random blinding scalar. Homomorphism is the essential property: the product (here, group sum) of commitments commits to the sum of values with the sum of blindings. Sapling commits to each note's value separately: a transaction carries input value commitments $\mathrm{cm}(v_i^{\text{in}}, \rho_i^{\text{in}})$ and output value commitments $\mathrm{cm}(v_j^{\text{out}}, \rho_j^{\text{out}})$, and the *net commitment* is

$$B \;=\; \sum_i \mathrm{cm}(v_i^{\text{in}}, \rho_i^{\text{in}}) \;-\; \sum_j \mathrm{cm}(v_j^{\text{out}}, \rho_j^{\text{out}}).$$

Orchard has *no* per-note value commitments: each Action commits once to its *net* value change, publishing

$$\mathrm{cv}^{\mathrm{net}} \;=\; \mathrm{cm}(v^{\mathrm{old}} - v^{\mathrm{new}},\, \rho) \;=\; [v^{\mathrm{old}} - v^{\mathrm{new}}]\,V + [\rho]\,P$$

for the spent value v^{old} and created value v^{new} of that Action (*Ironwood Guide*), and the net commitment is the plain sum $B = \sum_i \mathrm{cv}^{\mathrm{net},i}$ over the transaction's Actions. In either case, by homomorphism,

$$B \;=\; [\Delta v]\,V \;+\; [\bar{\rho}]\,P,$$

where Δv is the net value ($\sum_i v_i^{\mathrm{in}} - \sum_j v_j^{\mathrm{out}}$ in Sapling, $\sum_i (v_i^{\mathrm{old}} - v_i^{\mathrm{new}})$ in Orchard) and $\bar{\rho}$ is the corresponding net blinding. The transaction *balances* (no value created or destroyed, modulo a public fee) precisely when the net value Δv equals the public balance v^{bal}. Suppose, for clarity of the core idea, that the transaction is internally balanced so that $\Delta v = 0$. Then the V-component vanishes and

$$B \;=\; [\bar{\rho}]\,P.$$

That is: the net commitment B is a known multiple of the single generator P if and only if the values balance, and the multiplier $\bar{\rho}$ is the net blinding factor — a quantity the signer can compute (it knows all the ρ's) but that resists production otherwise, since producing any P-multiple representation of an *unbalanced* B would require knowing the discrete log of V to base P, which we assume hard.

Definition 8.24 (Binding signature)**.** With B and $\bar{\rho}$ as above (assuming balance, so $B = [\bar{\rho}]P$), set the *binding verification key bvk* $:= B$ and the *binding signing key bsk* $:= \bar{\rho}$. The binding signature is a RedDSA signature (Definition 8.19) over base point P, signing key $\bar{\rho}$, verification key B, on a message that is the hash of the transaction (the *sighash*). The verifier recomputes B from the publicly listed value commitments and the public balance, then checks the RedDSA equation $[s]P = R + [c]B$ with $c = H(R\|B\|\text{sighash})$.

Proposition 8.25 (The binding signature is a proof of knowledge of $\log_P B$)**.** *A valid binding signature on a given transaction is, in the random oracle model, a proof of knowledge of the discrete logarithm of the net commitment B to base P. Hence a valid binding signature certifies that the producer knew a scalar $\bar{\rho}$ with $B = [\bar{\rho}]P$; combined with the soundness of the accompanying zero-knowledge proofs (per Action in Orchard, per note in Sapling), this forces the transaction to balance.*

Proof. The binding signature is exactly a Fiat–Shamir-transformed Schnorr proof for the statement "$B \in \langle P\rangle$ with known discrete log," i.e. for the relation $\mathcal{R}_{\mathrm{dl}}$ with base P and statement B. By the special soundness of Proposition 8.7 together with the forking argument of Theorem 8.16 (specialised to a *single* statement and no signing oracle), the forking lemma can rewind any algorithm that produces a valid signature on B with non-negligible probability to yield two accepting transcripts $(R, c, s), (R, c', s')$ with $c \neq c'$, from which the extractor recovers $\bar{\rho} = (s - s')/(c - c')$ with $B = [\bar{\rho}]P$. Thus producing a valid binding signature is equivalent, up to the standard rewinding loss, to knowing $\log_P B$.

It remains to argue that knowing $\log_P B$ forces balance. Write $B = [\Delta v]V + [\bar{\rho}']P$ for the *true* net value Δv and net blinding $\bar{\rho}'$ the commitments imply (well-defined once the accompanying proofs fix the committed openings: in Orchard the per-Action circuit fixes v^{old}, v^{new} and the opening of $\mathrm{cv}^{\mathrm{net}}$; in Sapling the per-note proofs fix each v_i, ρ_i). If the signer knows some $\bar{\rho}$ with $B = [\bar{\rho}]P$, then subtracting,

$$[\Delta v]V = [\bar{\rho} - \bar{\rho}']P,$$

so $V = [(\bar{\rho} - \bar{\rho}')\,(\Delta v)^{-1}]P$ whenever $\Delta v \neq 0$ (using that Δv is invertible in $\mathbb{F}_q$ for a nonzero field element). This exhibits the discrete logarithm of V to base P, contradicting the assumed independence of the generators V, P. Therefore $\Delta v = 0$: the transaction balances. (In the deployed scheme, signing against $B - [v^{\mathrm{bal}}]V$ folds in the public balance v^{bal}; the same argument then forces $\Delta v = v^{\mathrm{bal}}$.) □

Remark 8.26 (Why a signature and not "just" a proof). A *single* group element pair (R, s) accomplishes three things. First, it proves knowledge of $\bar{\rho}$, which (Proposition 8.25) certifies balance. Second, because the Fiat–Shamir challenge hashes the transaction *sighash*, the very same signature *authenticates the transaction*: altering any signed field changes c and invalidates (R, s), so the binding signature doubles as an integrity check binding all the shielded components together (this is the origin of the name "binding"). Third, the construction requires *no trusted setup and no separate circuit* for the balance check — it is pure elliptic-curve arithmetic that anyone can verify. The economy is striking: the homomorphism of the commitment turns the arithmetic statement "values balance" into the algebraic statement "B is a known multiple of P," and the Schnorr signature is the off-the-shelf proof of knowledge for that statement. The binding signature is, in this sense, the purest illustration in the entire protocol of the principle that a digital signature *is* a proof of knowledge of a discrete logarithm.

Remark 8.27 (Relation to the spend authorisation signature). The two RedDSA uses merit contrast. The *spend authorisation* signature (Section 8.7) uses a *re-randomised* key $rk = X + [\alpha]P$ to authorise a spend while hiding which long-term key authorised it; here the signing key $rsk = x + \alpha$ encodes the spender's authority and α provides unlinkability. The *binding* signature uses a key $B = [\bar{\rho}]P$ that the spender does *not* choose but the transaction's commitments *force*; here the "secret key" $\bar{\rho}$ is the net blinding factor and signing it proves balance. Both consist of the same three lines of Schnorr arithmetic; what differs is the *meaning assigned to the discrete logarithm* — authority in one case, value-conservation in the other. This is the unifying lesson of the section: once a public key is the image of a secret under $x \mapsto [x]P$, a Schnorr signature is a compact, non-interactive proof that one knows that secret, and the designer is free to choose what knowing the secret *means*.

9 Interactive proofs, zero knowledge, and SNARKs

The primitives of the preceding sections—commitments, hash functions, signatures—let one party convince another of the integrity or authenticity of a *fixed* message. We now pose a more ambitious question. Suppose a party, the *prover*, knows that a certain statement is true: that a number is composite, that a graph is three-colourable, that she holds a discrete logarithm, that a long computation halts in an accepting state. Can she convince a sceptical *verifier* of this fact? Can she do so without revealing *why* it is true—without leaking the factorisation, the colouring, the discrete logarithm, the computation transcript? And can she do so with a proof so short and so cheaply checked that it is dwarfed by the statement it certifies?

These three questions—convincing, hiding, and succinctness—are the subject of this section. The theory of interactive proofs and arguments of knowledge, zero knowledge, and succinct non-interactive arguments of knowledge (SNARKs) answer them respectively. The development culminates in the architecture that underlies the Halo 2 proving system: a polynomial interactive oracle proof compiled by a polynomial commitment scheme and made non-interactive by the Fiat–Shamir transform. We build the entire edifice from the ground up.

Throughout, n denotes a security parameter; "probabilistic polynomial time" (PPT) means a randomised algorithm whose running time is bounded by a fixed polynomial in the length of its input (and in n when n is supplied in unary as 1^n); and $\mathrm{negl}(n)$ denotes a *negligible* function, one that eventually drops below $1/n^c$ for every constant c. We use freely the asymptotic and probabilistic language of the companion volume.

9.1 Languages, relations, and witnesses

The objects a prover proves things about are membership claims for a language, or—more usefully for cryptography—claims that one knows a witness for an instance.

Definition 9.1 (Language and relation)**.** Fix a finite alphabet, say $\{0,1\}$. A *language* is a set $L \subseteq \{0,1\}^*$ of bit strings. A *binary relation* is a set $R \subseteq \{0,1\}^* \times \{0,1\}^*$ of pairs (x, w); we call x the *instance* (or *statement*) and w a *witness* for x. The relation *induces* the language

$$L_R = \{\, x \in \{0,1\}^* : \exists\, w \text{ with } (x,w) \in R \,\}.$$

We write $R(x) = \{\, w : (x,w) \in R \,\}$ for the (possibly empty) *witness set* of x. The relation R is an *NP-relation* if it is *polynomially balanced*—there is a polynomial p with $|w| \le p(|x|)$ for all $(x,w) \in R$—and *polynomial-time decidable*: a deterministic algorithm, given (x,w), decides whether $(x,w) \in R$ in time polynomial in $|x|$. By a foundational theorem of complexity theory, $L \in \mathsf{NP}$ if and only if $L = L_R$ for some NP-relation R.

Example 9.2 (Three running examples)**.** The following relations recur throughout.

1. *Discrete logarithm.* Fix a cyclic group $\mathbb{G}$ of prime order q with generator g. Set $R_{\mathrm{dl}} = \{\, (h, w) : h = g^w,\ w \in \mathbb{Z}_q \,\}$. The instance $h \in \mathbb{G}$ always has a witness (the exponent exists), so the *language* is trivial; what is hard is to *produce* w. This is the prototype for which the distinction between proving membership and proving knowledge becomes essential (Subsection 9.3).

2. *Graph three-colouring.* $R_{\mathrm{3col}} = \{\, (G, c) : c \text{ is a proper 3-colouring of the graph } G \,\}$. The induced language L_{3col} is NP-complete, so a (zero-knowledge) protocol for it yields one for every NP language by reduction.

3. *Circuit satisfiability.* $R_{\mathrm{sat}} = \{\, (C, w) : C \text{ is a Boolean (or arithmetic) circuit and } C(w) = 1 \,\}$. This is the universal target of practical proof systems: one first "arithmetises" arbitrary computations into satisfiability of an arithmetic circuit over a finite field $\mathbb{F}$, and the prover proves she knows a satisfying assignment.

The discrete-logarithm example already shows why "$x \in L$" is too weak a target. Every group element is *some* power of g, so the membership language is all of $\mathbb{G}$ and proving membership is vacuous. The content lies in proving that the prover *knows* the exponent. Formalising "knows" is the central conceptual move of the section.

9.2 Interactive protocols, proofs, and arguments

Definition 9.3 (Interactive protocol)**.** An *interactive protocol* is a pair (P, V) of algorithms, the *prover* P and the *verifier* V, that exchange messages. Both receive a common input x; P additionally receives a private input (its witness or auxiliary string) w, and each holds its own private random tape. They alternate sending messages $a_1, a_2, \ldots$ over a number of *rounds* that is polynomial in $|x|$. After the last message, V outputs a single bit: 1 (*accept*) or 0 (*reject*). We write

$$\langle P(w), V\rangle(x) \in \{0, 1\}$$

for the verifier's output, a random variable over the parties' coins, and call the full sequence of exchanged messages (together with x and V's coins, when relevant) the *transcript*. The protocol is *public-coin* if every message V sends is a fresh uniformly random string that V also reveals—equivalently, V's messages are its random coins—and V's final decision is a deterministic function of the transcript. We always require V to run in probabilistic polynomial time.

The verifier is the party we must protect; its resources are therefore fixed at PPT once and for all. The prover is the party whose power we vary, and that variation is exactly the difference between a *proof* and an *argument*.

Definition 9.4 (Interactive proof; interactive argument)**.** Let R be a relation with language $L = L_R$, and let (P, V) be an interactive protocol. We say (P, V) is an *interactive proof system* for L with completeness error $c(\cdot)$ and soundness error $s(\cdot)$ if:

Completeness. For every $x \in L$ (with some witness w),

$$\Pr\big[\, \langle P(w), V\rangle(x) = 1 \,\big] \;\geq\; 1 - c(|x|).$$

Soundness. For every $x \notin L$ and *every* prover strategy P^*—of *unbounded* computational power, with arbitrary auxiliary input z—

$$\Pr\big[\, \langle P^*(z), V\rangle(x) = 1 \,\big] \;\leq\; s(|x|).$$

We require $c(|x|) + s(|x|) \leq 1 - 1/\operatorname{poly}(|x|)$ (the two thresholds must be bounded apart); usually c and s are negligible. The collection of languages with such proofs is the complexity class IP.

(P, V) is an *interactive argument system* (also called a *computationally sound proof*) for L if completeness holds as above, and soundness holds only against cheating provers P^* that are PPT:

$$x \notin L, \quad P^* \text{ PPT} \implies \Pr\big[\, \langle P^*(z), V\rangle(x) = 1 \,\big] \;\leq\; s(|x|).$$

Here soundness typically rests on a computational hardness assumption, and s must be negligible for all PPT P^*.

Remark 9.5 (Proofs versus arguments: which is stronger?)*.* The two notions trade soundness strength against other resources, and neither dominates the other across all axes.

- A *proof* resists a computationally unbounded adversary: no amount of computation lets a cheating prover certify a false statement except with probability s. This is an information-theoretic, unconditional guarantee.

- An *argument* resists only *efficient* adversaries. A cheating prover with unlimited time can typically break it—e.g. by breaking the binding of a commitment or computing a discrete logarithm by brute force. In return, arguments can achieve properties *provably impossible* for proofs, most importantly *succinctness* (Subsection 9.8): a sound interactive proof for an NP-complete language with communication much smaller than the witness would collapse complexity classes that are believed distinct, whereas succinct *arguments* exist under standard assumptions.

For cryptographic applications, including everything in the SNARK literature, the prover is a real machine and the argument notion is exactly the right one. "Soundness against a bounded prover" is not a weakness to apologise for; it is the price of, and the gateway to, succinctness.

Remark 9.6 (Why interaction and randomness are essential). If the verifier were deterministic and the protocol non-interactive (a single prover message read and checked), the system would prove exactly the languages decidable in polynomial time with a certificate, i.e. NP, and would offer no hope of zero knowledge: the certificate *is* the witness in spirit, and V could simply replay P's analysis. Randomness in V's challenges is what forces a cheating prover to commit before it knows what it will be asked, and interaction is what lets V adaptively probe. A celebrated line of results shows that interaction buys real power: IP = PSPACE, the class of languages decidable with polynomial memory and unrestricted time, believed to reach far beyond NP. For *this* section the relevant point is humbler but sharper—interaction and randomness are what make zero knowledge and knowledge extraction possible at all.

Soundness amplification by repetition. Repetition upgrades a protocol with a constant soundness error such as $1/2$ to negligible error. Running k *independent* instances and accepting only if all accept drives soundness error from s to s^k for proofs (sequential repetition always works; parallel repetition works for proofs and, with care, for many arguments), while completeness error grows only to $\leq k \cdot c$ by a union bound. Taking $k = \omega(\log |x|)$, e.g. $k = |x|$, makes s^k negligible. We use this freely, and design base protocols aiming merely for a noticeable soundness gap.

9.3 Knowledge soundness and extractors

Return to R_{dl}. Soundness in the sense of Definition 9.4 asserts nothing: every $h \in \mathbb{G}$ lies in the language, so the verifier may always accept and the soundness condition (quantified over $x \notin L$) holds vacuously. We seek instead the assurance that a convincing prover *possesses* the discrete logarithm. "Possesses" admits an operational reading: if the prover holds the witness, then an efficient procedure granted full control over the prover can *compute* the witness from it. That procedure is the *extractor*, and the access it requires is the ability to *rewind*.

Definition 9.7 (Knowledge soundness; proof/argument of knowledge)**.** Let R be a relation and (P, V) a protocol with the completeness property for R. The protocol (P, V) is a *proof of knowledge* for R with *knowledge error* $\kappa(\cdot)$ if there exists a probabilistic algorithm E, the *(knowledge) extractor*, and a polynomial q, such that for every (possibly cheating, possibly unbounded) prover P^* and every instance x, the following holds. Let

$$\varepsilon(x) \;=\; \Pr\big[\, \langle P^*, V\rangle(x) = 1 \,\big]$$

be the probability that P^* makes V accept. Then E, given x and *oracle access to P^* with the ability to rewind it*—that is, to run P^* from any point with the same coins and feed it verifier messages of E's choosing—outputs a witness $w \in R(x)$ within an expected number of steps bounded by

$$\frac{q(|x|)}{\varepsilon(x) - \kappa(x)} \qquad \text{whenever } \varepsilon(x) > \kappa(x).$$

The protocol (P, V) is then a *knowledge-sound* protocol; if soundness need hold only against PPT P^*, it is an *argument of knowledge*.

The definition warrants careful reading. The extractor's running time may grow as the prover's success probability ε shrinks toward the knowledge error κ: a prover that barely succeeds is correspondingly hard to extract from, and a prover that succeeds with probability at most κ need not yield anything at all. Yet *any* prover that succeeds noticeably above κ surrenders the witness to an efficient extractor. This is precisely the formal content of "a convincing prover knows the witness": we define knowledge as efficient computability *relative to the prover's own code and coins*.

Remark 9.8 (Rewinding, and the justification for assuming it). The extractor is a thought experiment, not a runtime procedure: it never executes during an honest protocol run. It exists only to give meaning to the security claim. Its power—to clone the prover's state and replay it under different challenges—is legitimate precisely because in the security *reduction* the extractor *is* the reduction, and it holds the cheating prover's code in hand as a subroutine. The same rewinding ability that the simulator employs to fake transcripts (Subsection 9.4) is what the extractor uses to mine real witnesses; rewinding is the single most important technical device in this entire theory.

Proposition 9.9 (Knowledge soundness implies soundness)**.** *If (P, V) is a proof of knowledge for R with knowledge error κ, then it is an interactive proof for L_R with soundness error $s(x) \le \kappa(x)$ for x of each length.*

Proof. Let $x \notin L_R$, so $R(x) = \varnothing$, and suppose for contradiction that some prover P^* achieves $\varepsilon(x) > \kappa(x)$. By Definition 9.7 the extractor $E^{P^*}(x)$ halts in finite expected time with output $w \in R(x)$. But $R(x)$ is empty, so E cannot output any such w—a contradiction. Hence $\varepsilon(x) \le \kappa(x)$ for every P^*, which is precisely soundness with error κ. □

The converse fails: a protocol can be sound (it accepts no false statement) yet not a proof of knowledge (one may convince the verifier of a *true* statement without knowing why). Knowledge soundness is strictly stronger, and it is the notion on which every cryptographic application actually relies. The Σ-protocols introduced below make this concrete, for there extraction takes its cleanest form.

9.4 The simulation paradigm and zero knowledge

We now require the protocol to leak nothing. The difficulty lies in specifying what "nothing" means: the verifier already learns that $x \in L$, and it observes a transcript of messages, so it certainly learns *something*. The resolution—the *simulation paradigm*, the most influential idea in modern cryptography—declares that the verifier learned nothing beyond the truth of the statement if it could have produced, entirely on its own, a transcript indistinguishable from the real one. Anything the verifier can manufacture without the prover, it did not learn from the prover.

We recall the three grades of "indistinguishable" first. Recall the statistical distance and the three grades of indistinguishability — perfect ($\equiv$), statistical ($\approx_s$), and computational ($\approx_c$) — of Definitions 1.8 and 1.12, together with Propositions 1.9 and 1.13. One adaptation is needed here: the ensembles are indexed not by the security parameter but by instances x (and auxiliary inputs), with negligibility measured in $n = |x|$; the definitions and the hierarchy carry over verbatim under this re-indexing.

Definition 9.10 (Zero knowledge)**.** Let (P, V) be an interactive proof or argument for $L = L_R$. For a verifier strategy V^*, instance x, and auxiliary input z to V^*, let

$$\mathrm{View}_{V^*}\big(P(w), V^*(z)\big)(x)$$

denote the *view* of V^*: the random variable consisting of x, V^*'s random tape, and every message V^* receives during the interaction (with P running on witness w). The protocol is *zero knowledge* if there exists a PPT algorithm Sim, the *simulator*, such that for every $x \in L$ (with witness w) and every auxiliary z,

$$\{\mathrm{Sim}(x,z)\} \quad \text{is indistinguishable from} \quad \{\mathrm{View}_{V^*}(P(w),V^*(z))(x)\}.$$

According to the grade of indistinguishability (Definition 1.12) one speaks of *perfect*, *statistical*, or *computational* zero knowledge (PZK, SZK, CZK). We distinguish two regimes of V^*:

Honest-verifier zero knowledge (HVZK). The guarantee need hold only for the *honest* verifier, $V^* = V$ following the protocol (with z empty). The simulator must reproduce the distribution of an honest transcript.

(Malicious-verifier / auxiliary-input) zero knowledge. The guarantee holds for *every* PPT V^*, running an arbitrary strategy and holding an arbitrary auxiliary input z. The simulator Sim may typically use V^* as a subroutine *and rewind it*, while still running in expected polynomial time.

Remark 9.11 (The basis for the simulator's adequacy, and the role of z). The simulator receives x but *not* the witness w. If it can nonetheless output transcripts indistinguishable from real ones, then the real transcript carries no efficiently extractable information about w beyond $x \in L$—for were it otherwise, the distinguisher could detect the difference. The crucial subtlety is that Sim wields a power P lacks: it can *rewind* V^*, retrying until V^*'s challenges align with a transcript the simulator could prepare without w. This is legitimate because Sim is, again, the reduction's tool, not a protocol participant. The auxiliary input z models prior knowledge a malicious verifier may bring (e.g. from earlier protocol runs); requiring zero knowledge to hold for all z is what makes the property *compose*—a zero-knowledge protocol remains zero knowledge when run as a subroutine inside a larger system. Practice adopts this auxiliary-input formulation.

9.5 Σ-protocols

Almost every practical zero-knowledge building block is a Σ-*protocol*: a three-move, public-coin protocol whose shape—a Greek capital sigma traced by the prover's commitment going out, the verifier's challenge coming back, and the prover's response going out again—gives the family its name. The structure is rigid enough to admit clean, checkable security definitions, and rich enough to capture an enormous range of statements through composition.

Definition 9.12 (Σ-protocol)**.** Let R be a relation. A Σ-*protocol* for R is a three-move public-coin protocol (P,V) on common input x and prover input w with $(x,w) \in R$, of the following form, where $\mathcal{C}$ is the *challenge space*:

1. *Commitment.* P sends a first message a (the *commitment*, also called the *announcement*), computed from (x,w) and fresh randomness.
2. *Challenge.* V sends a uniformly random $e \in \mathcal{C}$.
3. *Response.* P sends z, computed from (x,w,a,e) and its randomness.

V accepts or rejects by a deterministic predicate $\mathrm{Verify}(x,a,e,z) \in \{0,1\}$. The protocol must satisfy:

Completeness. If $(x,w) \in R$ and P is honest, V accepts with probability 1.

Special soundness. There exists a PPT algorithm that, given x and any two accepting transcripts (a, e, z) and (a, e', z') with the *same commitment* a but *distinct challenges* $e \neq e'$, outputs a witness w with $(x, w) \in R$.

Special honest-verifier zero knowledge (sHVZK). There exists a PPT simulator that, on input x and *any* challenge $e \in \mathcal{C}$, outputs a pair (a, z) such that the simulated transcript (a, e, z) is distributed exactly (or statistically/computationally close) to a real honest transcript conditioned on the challenge being e. In particular the simulator chooses a *after* fixing e, and $\mathrm{Verify}(x, a, e, z) = 1$.

Two accepting transcripts sharing the commitment but differing in the challenge constitute a *collision*; special soundness states that a collision *is* a witness. This is the local, two-transcript reformulation of knowledge extraction, and it is what renders Σ-protocols so tractable.

Theorem 9.13 (Special soundness yields knowledge soundness)**.** *Let (P, V) be a Σ-protocol for R with challenge space $\mathcal{C}$, $|\mathcal{C}| = N$. Then (P, V) is a proof of knowledge for R with knowledge error $\kappa = 1/N$.*

Proof. Fix a cheating prover P^* on instance x, with success probability $\varepsilon = \varepsilon(x) > 1/N$. Because the protocol is public-coin with a single challenge, we may model P^* by its random tape ρ: once ρ is fixed, P^* deterministically produces the commitment $a = a(\rho)$ and, for each challenge e, a response $z = z(\rho, e)$. Consider the boolean matrix H indexed by rows ρ and columns $e \in \mathcal{C}$, with $H[\rho, e] = 1$ iff $\mathrm{Verify}(x, a(\rho), e, z(\rho, e)) = 1$. The overall success probability equals the fraction of 1-entries (weighting rows by the probability of ρ),

$$\varepsilon = \Pr_{\rho, e}\big[\, H[\rho, e] = 1 \,\big].$$

The extractor E runs the following *rewinding* procedure.

1. Run P^* with a fresh random ρ and a uniformly random challenge e; obtain (a, e, z). If V rejects, restart this step.

2. Once an accepting (a, e, z) appears, alternate two kinds of trial: (i) *rewind* P^* to just after it sent a (i.e. keep ρ fixed) and feed it one fresh uniform challenge $e' \neq e$; (ii) run one entirely fresh probe as in Step 1, with a new ρ and a new uniform e. If a trial of kind (i) accepts, the two transcripts collide; proceed to Step 3. If a trial of kind (ii) accepts, abandon the current row and continue Step 2 from the new accepting transcript.

3. Apply the special-soundness algorithm to (a, e, z) and (a, e', z') and output the witness w.

Correctness follows immediately from special soundness once we have two such transcripts; it remains to bound the expected running time. The expected cost of Step 1 is $1/\varepsilon$ protocol runs (the geometric waiting time of the *Math Guide*, §"Beyond finite spaces: countable additivity, limits, and densities"). For Step 2, partition the trials into *phases*, one per row visited; for the current row ρ, let $\delta_\rho = \Pr_e[H[\rho, e] = 1]$ be its acceptance fraction, and call the row *heavy* if $\delta_\rho \geq \varepsilon/2$. By a Markov-type averaging argument (the standard "heavy-row" lemma), an accepting transcript lies in a heavy row with probability at least $1/2$; this applies to the transcript opening each phase, since every phase opens with a fresh accepting transcript distributed as in Step 1. Each alternation ends the phase with probability at least ε (the kind-(ii) trial alone accepts that often), so a phase consumes $O(1/\varepsilon)$ invocations of P^* in expectation—this is what the interleaving buys: even a row whose only accepting challenge is the already-used e cannot trap the extractor. Within a heavy row, a kind-(i) trial accepts with probability at least $\delta_\rho - 1/N \geq \varepsilon/2 - 1/N$ (a fresh challenge accepts with probability δ_ρ and equals the used e with probability at most $1/N$), so the phase ends in a collision rather than a row switch with probability $\Omega\big((\varepsilon/2 - 1/N)/\varepsilon\big)$. Combining, each phase yields a collision with probability

$\Omega\big((\varepsilon/2-1/N)/\varepsilon\big)$, so the expected number of phases is $O\big(\varepsilon/(\varepsilon/2-1/N)\big)$, and by Wald's identity (*Math Guide*, same section) the total expected number of invocations of P^* is bounded by

$$O\Big(\frac{\varepsilon}{\varepsilon/2-1/N}\Big)\cdot O\Big(\frac{1}{\varepsilon}\Big) \;=\; O\Big(\frac{1}{\varepsilon-2/N}\Big),$$

which is the form Definition 9.7 requires, with $\kappa = 2/N$ and a polynomial q absorbing the per-run cost. (Rows with $\delta_\rho \le 1/N$ contribute no collisions; without the interleaved fresh probes they would trap the extractor for an unbounded expected time, which is why Step 2 alternates rather than insisting on the first row it finds.) Hence E extracts a witness in expected time $q(|x|)/(\varepsilon - 2/N)$; the crude factor of two in the threshold $\delta_\rho \ge \varepsilon/2$ is an artefact of this exposition, costing the extra $1/N$. The optimal knowledge error $\kappa = 1/N$ of the statement, matching the soundness error, is delivered by the tight extractors in the literature: Damgård's refined heavy-row analysis in his Σ-protocol lecture notes, and the k-special-soundness extractor of Attema, Cramer, and Kohl, which achieves knowledge error exactly $(k-1)/N$ — here $k = 2$, i.e. $1/N$. □

Remark 9.14 (Knowledge error and the challenge space). The knowledge error $1/N$ is also the soundness error: a prover who guesses the challenge in advance can answer that one challenge without a witness, succeeding with probability $1/N$. To render this negligible one takes $\mathcal{C}$ exponentially large—e.g. $\mathcal{C} = \mathbb{Z}_q$ for a large prime q, so $N = q$—or repeats a small-challenge protocol $\omega(\log n)$ times. Large challenge spaces are the reason a single round of Schnorr (below) already has negligible soundness error, whereas the three-colouring protocol requires many repetitions.

The two-transcript notion generalises to protocols with several challenge rounds, and the generalisation is the exact bridge to the multi-round folding arguments of the Halo 2 system.

Definition 9.15 (Tree special soundness). Let (P, V) be a $(2\mu+1)$-move public-coin protocol for R, in which the prover speaks first and the verifier issues μ uniformly random challenges from a space $\mathcal{C}$. For positive integers $k_1, \dots, k_\mu$, a *$(k_1, \dots, k_\mu)$-tree of accepting transcripts* for an instance x is a tree of depth μ in which every node at level i has k_i children labelled by *pairwise distinct* challenges for round i, every root-to-leaf path carries the prover messages of one accepting transcript, and transcripts through a common node share their prefix. The protocol is *$(k_1, \dots, k_\mu)$-special-sound* if a PPT algorithm computes a witness $w \in R(x)$ from any such tree.

A Σ-protocol is the case $\mu = 1$, $k_1 = 2$. The extraction argument of Theorem 9.13 generalises: rewinding the prover round by round, an extractor assembles the required tree of $\prod_i k_i$ accepting transcripts, so a $(k_1, \dots, k_\mu)$-special-sound protocol is a proof of knowledge whose knowledge error grows from $1/|\mathcal{C}|$ to the order of $\big(\sum_i (k_i - 1)\big)/|\mathcal{C}|$, and whose extraction cost is polynomial provided the tree size $\prod_i k_i$ is. Both degradations are benign when $\mathcal{C}$ is exponentially large, μ is logarithmically bounded, and the k_i are constant, for then the tree size $\prod_i k_i$ is polynomial. This is the notion under which the inner-product argument's extractor operates (Proposition 7.32): each of the $\log_2 d$ folding rounds contributes one level of branching. The *Halo 2 Guide* instantiates this notion for its IPA extractor with one refinement: because each round's verification equation involves only the monomials $u^{-2}, 1, u^2$, the three challenges at every node must have pairwise distinct *squares* ($u \ne \pm u'$), a negligible-fraction exclusion absorbed into the knowledge error.

Proposition 9.16 (sHVZK is HVZK). *A Σ-protocol satisfying special HVZK is honest-verifier zero knowledge in the sense of Definition 9.10.*

Proof. The honest verifier's view is (a, e, z) together with its coins, where e is uniform in $\mathcal{C}$. The HVZK simulator draws $e \leftarrow \mathcal{C}$ uniformly, then invokes the special-HVZK simulator on (x, e) to obtain (a, z), and outputs (a, e, z). By special HVZK, for each fixed e the pair (a, z) is distributed as in a real run conditioned on that e; averaging over the uniform e reproduces the honest view exactly (or up to the relevant negligible distance). Thus the simulated and real views coincide as distributions. □

Example 9.17 (Schnorr's protocol for discrete logarithm)**.** The canonical Σ-protocol is Schnorr's identification protocol for R_{dl}, developed in full in §8.3: the commitment is $R = [r]G$, the challenge $c \in \mathbb{F}_q$ is uniform, the response is $s = r + cx$, and the verifier checks $[s]G = R + [c]X$. Its three Σ-properties are exactly Propositions 8.6 (perfect completeness), 8.7 (2-special soundness: a collision yields the witness $x = (s - s')(c - c')^{-1}$), and 8.9 (special HVZK: sample s, solve for $R = [s]G - [c]X$). With challenge space $\mathbb{F}_q$ the knowledge error is $1/q$, already negligible in a single round (Remark 9.14); by Theorem 9.13 and Proposition 9.16, Schnorr is a perfect-HVZK proof of knowledge of a discrete logarithm.

Remark 9.18 (From HVZK to full zero knowledge)*.* Σ-protocols are only *honest-verifier* zero knowledge: a malicious V^* might choose e as a function of a to leak information, and the sHVZK simulator (which picks e itself) does not obviously cope. Three standard remedies upgrade HVZK to malicious-verifier ZK:

1. *Sequential repetition* of a protocol with a small challenge, where V commits to nothing across rounds—this gives full ZK by a rewinding simulator but costs rounds.
2. Have V *commit* to its challenge e before seeing a (a coin-tossing or commitment step), so a malicious V^* cannot make e depend on a; the simulator then rewinds past the opening.
3. Apply Fiat–Shamir (Subsection 9.6), which removes the verifier's choice of e altogether by deriving e from a hash—in the random-oracle model the resulting non-interactive protocol is zero knowledge against *all* verifiers.

For the SNARK pipeline it is route 3 that matters, and HVZK is exactly the property Fiat–Shamir requires.

9.6 The Fiat–Shamir transform: from interactive to non-interactive

The only verifier messages in a public-coin protocol are random challenges. The Fiat–Shamir heuristic removes the verifier from the loop by *computing* each challenge as a hash of everything sent so far. The interaction collapses into a single non-interactive proof string, and—because the prover no longer talks to a live, possibly-malicious verifier but to a fixed public function—honest-verifier zero knowledge upgrades to full non-interactive zero knowledge. The cost is that the hash function must be idealised: the security proof rests on the *random-oracle model* (ROM) of Definition 8.13, with the oracle $\mathcal{H} : \{0,1\}^* \to \mathcal{C}$ now mapping into the challenge space $\mathcal{C}$. We again exploit the two powers the model grants a reduction: *observability* (the reduction sees every query the adversary makes) and *programmability* (the reduction may fix $\mathcal{H}$'s output at a freshly queried point to any value it likes, provided the answers remain consistent and uniformly distributed).

Definition 9.19 (Fiat–Shamir transform)**.** Let (P, V) be a public-coin interactive protocol (we describe the Σ case; the multi-round case applies the rule per round). Let $\mathcal{H}$ be a random oracle with range the challenge space $\mathcal{C}$. The *Fiat–Shamir transform* $\mathrm{FS}[(P, V), \mathcal{H}]$ is the non-interactive protocol:

- *Prover.* Compute the commitment a as in (P, V); set $e = \mathcal{H}(x, a)$; compute the response z. Output the proof $\pi = (a, z)$ (equivalently (a, e, z)).
- *Verifier.* Given x and $\pi = (a, z)$, recompute $e = \mathcal{H}(x, a)$ and accept iff $\mathrm{Verify}(x, a, e, z) = 1$.

In multi-round public-coin protocols, the i-th challenge is $e_i = \mathcal{H}(x, a_1, e_1, \ldots, a_i)$—each challenge hashes the *entire* transcript prefix. To bind a message m (turning the proof into a signature), include m in the hash: $e = \mathcal{H}(x, a, m)$.

Remark 9.20 (Hashing the whole transcript: the Fiat–Shamir pitfall). The requirement that each challenge hash *all* prior messages—the instance included—is not pedantry. The notorious "weak Fiat–Shamir" bug omits the statement x (or part of the transcript) from the hash, letting an attacker choose x *after* the challenge is fixed and forge proofs of false statements. For multi-round protocols, hashing only the latest message rather than the running prefix similarly breaks soundness. The maxim is: the challenge must be a hash of everything the verifier would have seen up to the moment it speaks.

Theorem 9.21 (Fiat–Shamir: security in the ROM)**.** *Let (P, V) be a Σ-protocol for R with challenge space $\mathcal{C}$, $|\mathcal{C}| = N$, special soundness, and special HVZK. In the random-oracle model, $\Pi = \mathrm{FS}[(P, V), \mathcal{H}]$ is a non-interactive argument that is:*

1. complete *(perfectly, inheriting completeness of (P, V));*
2. knowledge sound*: from any PPT prover P^* making at most Q random-oracle queries and producing an accepting proof for x with probability ε, an extractor obtains a witness $w \in R(x)$ with probability at least $\varepsilon \cdot (\varepsilon/Q - 1/N)$ (up to constants), in particular non-negligibly whenever ε is non-negligible and N is super-polynomial; and*
3. (non-interactive) zero knowledge*: there is a PPT simulator that, controlling the random oracle by programming, produces proofs indistinguishable from real ones* without *the witness, for all (even malicious) verifiers.*

Proof. Completeness is immediate: the honest prover computes $e = \mathcal{H}(x, a)$ and an honest response, which verifies, and the verifier recomputes the same e.

Zero knowledge by programming. The simulator, given x, runs the special-HVZK simulator: it draws $e \leftarrow \mathcal{C}$, obtains an accepting (a, z), and then *programs* the random oracle by setting $\mathcal{H}(x, a) := e$. Provided (x, a) was not previously queried—which holds except with probability at most (number of prior queries) $\cdot\, 2^{-H_\infty(a)}$, negligible because the honest first message a has high min-entropy—the programmed oracle is consistent and uniformly distributed, and the output (a, z) is distributed exactly as a real proof. Programming remains invisible to the verifier, so this is zero knowledge against all verifiers (the verifier's malice is moot: it has no live challenge to skew).

Knowledge soundness by the forking lemma. Let P^* be a prover that outputs an accepting $\pi = (a, z)$ for x with probability ε, making Q oracle queries. With overwhelming probability P^* actually queried the value $e = \mathcal{H}(x, a)$ used in a successful proof (else P^* guessed an unqueried random output, probability $\le 1/N$ per proof); say it was the j-th query for some index $j \le Q$. The extractor runs P^* once to obtain a successful transcript with its query index j; it then *rewinds* P^* to just before the j-th query and re-runs it with *fresh, reprogrammed* oracle answers from that point on—so the first $j - 1$ answers, and hence the commitment a, remain unchanged, but the j-th answer (the challenge on (x, a)) is a new uniform $e' \in \mathcal{C}$. The *general forking lemma* guarantees that, if P^* succeeds with probability ε and there are Q query slots, then with probability at least $\varepsilon\,(\varepsilon/Q - 1/N)$ both runs succeed *and* $e \neq e'$. Conditioned on that event we hold two accepting transcripts (a, e, z) and (a, e', z') with the same a and $e \neq e'$; the special-soundness extractor (Definition 9.12) outputs $w \in R(x)$. This is exactly the knowledge-soundness conclusion, with the stated success probability. □

Remark 9.22 (What the random oracle is, and is not). The ROM is an idealisation: real hash functions are fixed, public, and not random, and there exist (contrived) protocols secure in the ROM yet insecure under *every* concrete instantiation. The transform is nonetheless trusted in practice and underlies essentially every deployed non-interactive proof and signature, including the Fiat–Shamir-compiled SNARKs below. Two refinements matter for those systems: *(i)* the multi-round forking analysis is more delicate—one must fork each of the protocol's rounds, and the success probability degrades with the round count and the soundness structure (special soundness must generalise to the tree special soundness of Definition 9.15); *(ii)* when the underlying interactive argument is computationally (not statistically) sound, the ROM extraction layers on top of the computational soundness reduction. A

subtle but practically important issue is the gap between *programmable* ROM proofs and the non-programmable setting; deployed systems adopt the programmable model.

Example 9.23 (Schnorr signatures)**.** Fiat–Shamir applied to Schnorr's Σ-protocol (Example 9.17), with a message m hashed into the challenge, is precisely the Schnorr signature scheme of Definition 8.11, whose EUF-CMA security Theorem 8.16 establishes by exactly the forking argument above. This is the canonical illustration that "proof of knowledge + Fiat–Shamir = signature."

9.7 From Σ-protocols toward general computation

Σ-protocols handle algebraic relations (discrete logs, openings of commitments, linear statements) with a single move of each kind. To prove arbitrary NP statements—"I know an input making this circuit output 1"—we need protocols for general computation.

Remark 9.24 (The feasibility theorem and its cost)*.* The classical route passes through an NP-complete problem. For graph three-colouring ($R_{3\mathrm{col}}$ of Example 9.2) a simple commit–challenge–reveal protocol—the prover commits to a randomly recoloured copy of her colouring, the verifier challenges with one uniformly random edge, the prover opens the two endpoint commitments, and the verifier checks that the revealed colours differ—is a computational-HVZK proof of knowledge with soundness error $1 - 1/|E|$, which repetition drives to negligible. This is the constructive core of the celebrated feasibility theorem: under the existence of one-way functions (which yield the commitments), every NP language—indeed every language in IP = PSPACE—has a computational zero-knowledge proof. The theorem settles *feasibility* but not *efficiency*: the protocol commits to a fresh copy of the entire colouring in each of polynomially many repetitions, so both communication and the prover's work scale with (circuit size) × (repetitions). Practical proof systems abandon the generic reduction and instead arithmetise computation and exploit polynomial structure—the subject of the final subsection.

9.8 SNARKs: succinct non-interactive arguments of knowledge

This is the destination. A SNARK certifies a possibly enormous computation with a proof that is tiny and that one checks almost instantly, hiding the witness if desired. We assemble its definition from the notions already built, then give the modern recipe—a polynomial interactive oracle proof compiled by a polynomial commitment scheme and made non-interactive by Fiat–Shamir—that the Halo 2 system instantiates.

Definition 9.25 (Non-interactive argument; preprocessing)**.** A *non-interactive argument* for an NP-relation R consists of three PPT algorithms:

- $\mathsf{Setup}(1^n, R) \to \mathsf{srs}$, producing a *structured reference string* (also "public parameters"; in the ROM this may be a single random oracle, giving a *transparent* setup with no secret to discard);
- $\mathsf{Prove}(\mathsf{srs}, x, w) \to \pi$, with $(x, w) \in R$, producing a proof π;
- $\mathsf{Verify}(\mathsf{srs}, x, \pi) \to \{0, 1\}$.

It is *complete* if honest proofs always verify. A scheme is a *preprocessing* argument if Setup additionally depends on the circuit/relation and outputs a short verification key, amortising per-statement verifier work.

Definition 9.26 (SNARK)**.** A *SNARK* (succinct non-interactive argument of knowledge) for an

NP-relation R is a non-interactive argument that is, in addition:

Knowledge sound. For every PPT prover P^* producing an accepting proof for some x, there is a PPT extractor (with the same setup, and—in the ROM—observing P^*'s oracle queries and rewinding/forking it) that outputs w with $(x, w) \in R$, except with negligible probability. Formally, $\Pr[\mathsf{Verify}(\mathsf{srs}, x, \pi) = 1 \wedge (x, E^{P^*}) \notin R] \leq \mathrm{negl}(n)$.

Succinct. The proof size $|\pi|$ and the verifier's running time are *sublinear* in the size of the computation being proved—typically $|\pi| = O(\mathrm{poly}(n))$ or $O(\mathrm{poly}(n) \log |C|)$ and verification $O(\mathrm{poly}(n) \log |C|)$ for a circuit C, *independent of, or only polylogarithmic in, the witness and circuit size*. (Some literature reserves "succinct" for polylogarithmic proofs; "$O(1)$-size" pairing-based schemes are the strongest form.)

If it also satisfies zero knowledge (Definition 9.10, in its non-interactive ROM form), it is a *zk-SNARK*.

Remark 9.27 (Why succinctness forces *arguments*, not proofs). A succinct *proof*—information-theoretic soundness with sublinear communication—for an NP-complete language is essentially ruled out: an unbounded prover could otherwise compress NP-certificates below the bounds that complexity theory forbids, and indeed a transcript shorter than the witness cannot information-theoretically pin down the witness. Succinct *arguments* escape this because a computationally bounded prover *cannot find* a convincing-but-false short proof, even though one may exist. This is the deepest reason the field works with *arguments* (Remark 9.5): succinctness and computational soundness arise together, by necessity.

Remark 9.28 (What knowledge soundness provides an application). For a deployed system, plain soundness does not suffice; knowledge soundness is the property that renders the proof *useful*. Consider a private transaction asserting "there exists a note I am authorised to spend, and the input and output values balance." Soundness alone guarantees only that *some* satisfying witness exists—it does not prevent a prover who has no such note from proving the existential statement were she somehow to find a proof. Knowledge soundness guarantees that the proving party *actually holds* a concrete witness—a real spendable note, a real authorisation—that the extractor could recover. The application reasons: *whoever produced this proof possesses the secret it attests to*. This is what permits a verifier to safely act on the proof (release funds, grant access) as though the prover had presented the witness, while it remains hidden. In short: **soundness protects against false statements; knowledge soundness protects against provers who do not actually know what they claim**—and almost every cryptographic use case requires the latter.

The polynomial-IOP-and-commitment compilation recipe. Modern SNARKs, Halo 2 included, are not monolithic. They factor cleanly into an *information-theoretic* object and a *cryptographic* compiler, glued and de-interactivised by Fiat–Shamir. We describe the three layers in turn.

Definition 9.29 (Polynomial interactive oracle proof (Poly-IOP))**.** A *polynomial IOP* for a relation R over a field $\mathbb{F}$ is a public-coin interactive protocol between P and V in which, instead of sending field elements, the prover in each round sends an *oracle* for a polynomial $p_i \in \mathbb{F}[X]$ of bounded degree. The verifier sends uniformly random field-element challenges, and may *query* each polynomial oracle at points of its choice, receiving $p_i(\zeta)$. After the interaction V accepts or rejects based on the (few) evaluations it queried and its challenges. Completeness and (knowledge) soundness are as for interactive proofs, with the soundness analysis purely algebraic—typically resting on the *Schwartz–Zippel lemma*: a nonzero polynomial of degree d over $\mathbb{F}$ vanishes at a uniformly random point with probability at most $d/|\mathbb{F}|$. The verifier in a good Poly-IOP makes only $O(1)$ or $O(\log)$ queries, each a single evaluation.

Remark 9.30 (Arithmetisation: reaching a Poly-IOP). The first step in building any of these systems is *arithmetisation*: encoding the claim "I know w with $C(w) = 1$" as a small set of polynomial identities over $\mathbb{F}$ that hold iff the computation is correct. One expresses computations as constraint systems—rank-1 constraint systems (R1CS), or the customisable *PLONKish* gate-and-permutation form used by Halo 2—whose satisfaction is equivalent to a handful of polynomials (encoding the wire values, the gate constraints, and a permutation argument enforcing wiring/copy constraints) satisfying identities like $q_L\, a + q_R\, b + q_M\, ab + q_O\, c + q_C = 0$ on a chosen evaluation domain. Checking such identities at a single random point—legitimised by Schwartz–Zippel—is the Poly-IOP. Crucially, the Poly-IOP is unconditionally sound: it rests on no cryptographic assumptions *yet*, only the algebra of low-degree polynomials.

The cryptographic half of the recipe is the *polynomial commitment scheme* of Definition 7.28: a short, binding commitment $\mathsf{Commit}(p)$ to a polynomial $p \in \mathbb{F}[X]$ of bounded degree, together with short proofs, verifiable against the commitment alone, that the committed polynomial satisfies $p(\zeta) = v$—with evaluation binding and, for SNARK use, the extractability property that a prover who opens evaluations must "know" an underlying polynomial. Instantiations include KZG (pairing-based, $O(1)$-size, trusted setup), the inner-product-argument (IPA) commitment over a single elliptic curve used in Halo 2 (transparent, no trusted setup, logarithmic-size openings), and FRI (hash-based, transparent, used in STARKs).

Theorem 9.31 (The compilation recipe, informally)**.** *Let* PIOP *be a public-coin polynomial IOP for R that is complete and knowledge-sound, and let* PC *be a polynomial commitment scheme that is correct, evaluation-binding, and extractable. Compile them as follows: wherever the prover would send a polynomial oracle p_i, it instead sends* $\mathsf{Commit}(p_i)$*; wherever the verifier would query $p_i(\zeta)$, the prover sends the claimed value with a* PC *evaluation proof, which the verifier checks. The result is a public-coin* interactive argument of knowledge *for R. Applying the Fiat–Shamir transform (Definition 9.19) in the random-oracle model—deriving every verifier challenge as a hash of the transcript so far—yields a non-interactive argument of knowledge: a* SNARK*. If, in addition, the Poly-IOP is made zero knowledge (e.g. the prover blinds its polynomials with random masking terms and the commitment is hiding), the SNARK is a* zk-SNARK*.*

Sketch. Completeness transfers verbatim: honest commitments and honest evaluation proofs let the verifier reconstruct every value it would have queried in the Poly-IOP, and the Poly-IOP verifier accepts. *Knowledge soundness* is layered. The SNARK extractor first runs the PC extractability guarantee on each commitment the prover produced, recovering actual polynomials $\hat{p}_i$ consistent with all opened evaluations (here the binding/extractable property of PC is the cryptographic assumption that turns the proof into an *argument*). It then feeds these $\hat{p}_i$ to the Poly-IOP's (unconditional) knowledge extractor, which, because the Poly-IOP is sound against *any* polynomials—in particular the extracted ones—outputs a witness $w \in R(x)$; evaluation-binding ensures the prover could not have answered queries inconsistently with the committed $\hat{p}_i$, so the algebraic soundness applies. *Fiat–Shamir* converts the resulting public-coin interactive argument of knowledge into a non-interactive one in the ROM by the (multi-round generalisation of the) analysis of Theorem 9.21: challenges become transcript hashes, the forking/observability of the random oracle supplies the rewinding the extractor needs, and HVZK of the blinded protocol becomes full NIZK by programming. *Succinctness* of the proof comes entirely from PC: it consists of a constant or logarithmic number of short commitments and short evaluation proofs, so the proof *size* is sublinear in the circuit, independent of the witness. The verifier checks $O(1)$ or $O(\log)$ evaluations and a handful of field identities; its running time is sublinear only when the PC's evaluation checks are themselves succinct—as for KZG. With the inner-product commitment each such check costs $O(d)$ group operations (Proposition 7.32), so the compiled verifier is linear-time in the circuit while the proof stays succinct. Combining, the compiled object satisfies every clause of Definition 9.26 when PC openings verify succinctly, and otherwise yields succinct proofs with linear-time verification. □

Remark 9.32 (Where Halo 2 sits, and what comes next). The system this monograph builds toward, Halo 2, is exactly an instance of Theorem 9.31: a *PLONKish* arithmetisation (custom gates, lookup arguments, and a permutation argument for copy constraints) furnishes the polynomial IOP; the *inner-product-argument* polynomial commitment over a single elliptic curve furnishes a *transparent* PC with no trusted setup and logarithmic openings; and Fiat–Shamir over a concrete hash makes it non-interactive. Its signature innovation, *accumulation* (the "Halo" technique), defers and batches the expensive final IPA check across many proofs, enabling recursive composition without a trusted setup—one proof attesting to the verification of another. We have now built each ingredient named here—elliptic-curve groups, polynomial commitments, the Schwartz–Zippel lemma, the Fiat–Shamir transform, knowledge soundness via extraction—from first principles, and the chapters that follow assemble them into that system. The reader should carry forward three load-bearing ideas: *rewinding* (Remark 9.8) as the universal device behind both extraction and simulation; the clean split between an *unconditionally sound* information-theoretic core and a *cryptographically* enforced commitment; and the principle that *knowledge* soundness—not mere soundness—is what an application stakes its security on.

10 Merkle trees and commitments to sets

A recurring problem in cryptography is the following. A party — the *prover* — holds a large collection of data: a list of transactions, a set of public keys, a database of records, or, in the application of principal concern here, a growing set of *note commitments* in a shielded payment system. A second party — the *verifier* — wishes to be convinced of statements about this collection without storing it in its entirety. The two canonical statements are:

- *membership*: "the value v occurs at position i in the list", or "v belongs to the set";
- *integrity*: "the collection is exactly the one committed to earlier, unaltered."

We require a single short string — a *commitment* — that determines the entire collection, together with short *proofs* that individual elements are present. "Short" here means: of size independent of, or at worst logarithmic in, the number of elements. The *Merkle tree*, introduced by Ralph Merkle, achieves this using nothing more than a collision-resistant hash function, and it is the structural backbone of essentially every modern blockchain and of the membership argument inside Zcash's Orchard protocol.

This section develops Merkle trees from first principles. We first recall the precise security property of the underlying compression function; we then construct the tree, define and analyse authentication paths, prove the reduction of path forgery to collision finding, and treat the engineering subtleties (domain and layer separation) that render the reduction honest. The dynamic setting follows — append-only and incrementally updatable trees and the notion of a frontier — after which we abstract the construction into the language of *vector commitments*. Finally, we show that composing a Merkle root over note commitments with a zero-knowledge proof of an authentication path yields a privacy-preserving membership primitive, which is precisely the role Merkle trees play in Orchard.

Throughout, $\{0,1\}^*$ denotes the set of finite binary strings, $\{0,1\}^n$ the strings of length n, and $\|$ denotes concatenation. The symbol λ denotes the security parameter; all algorithms are probabilistic polynomial-time (PPT) in λ unless stated otherwise, and $\mathrm{negl}(\lambda)$ denotes a negligible function (one that decays faster than the inverse of every polynomial).

10.1 The compression function and collision resistance

The single cryptographic ingredient of a Merkle tree is a function that takes a fixed-size input and returns a (typically shorter) fixed-size output, in such a way that no one can exhibit two distinct inputs colliding on the same output. We isolate the required property as follows.

Definition 10.1 (Hash family)**.** A *hash family* is a pair of PPT algorithms (Gen, H). On input 1^λ, Gen outputs a key k (the *public parameters*), which determines the length $\ell = \ell(\lambda)$ of the digest. The (deterministic) evaluation algorithm computes a function

$$H_k : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

When the input length is restricted to a fixed multiple of the output length — the relevant case here — we term the function a *compression function*; a two-to-one compression function has the signature

$$h_k : \{0,1\}^{2\ell} \longrightarrow \{0,1\}^\ell.$$

The key k models the choice of a concrete function from a family, which is what renders collision resistance a meaningful asymptotic notion (Remark 3.2); in practice we use a fixed standardised function, or inside arithmetic circuits an algebraic hash, and we suppress k from the notation when it plays no role, writing H and h. The security property we need is *collision resistance* exactly as in Definition 3.6: no PPT adversary outputs $x \neq x'$ with $H_k(x) = H_k(x')$, except with probability $\mathrm{negl}(\lambda)$.

Recall from §3.2 that this is the strongest of the three classical hash notions (Proposition 3.7), and from Lemma 3.9 that the generic birthday attack caps an ℓ-bit digest at $\ell/2$ bits of collision security, whence the 256-bit outputs used here. Merkle trees require full collision resistance, for the reason Theorem 10.13 establishes.

10.2 Binary Merkle trees

Fix a two-to-one compression function $h : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$. We treat first the clean case in which the number of leaves is a power of two.

Definition 10.2 (Binary Merkle tree, full case)**.** Let $d \in \mathbb{N}$ and let $L = (L_0, L_1, \ldots, L_{2^d-1})$ be an ordered list of 2^d *leaf values*, each in $\{0,1\}^{\ell}$. The *Merkle tree of depth d* over L is the complete binary tree with 2^d leaves whose nodes carry labels in $\{0,1\}^{\ell}$, defined level by level from the leaves up. Index the levels 0 (leaves) through d (root). The labels of level 0 are

$$N_{0,j} \;=\; L_j, \qquad 0 \le j < 2^d,$$

and the label of each internal node is the compression of its two children's labels:

$$N_{t+1,\,j} \;=\; h\big(N_{t,2j} \,\|\, N_{t,2j+1}\big), \qquad 0 \le t < d, \quad 0 \le j < 2^{d-t-1}. \tag{$*$}$$

The unique node at level d, namely $N_{d,0}$, is the *root* of the tree; write $\mathsf{root}(L) = N_{d,0}$.

Here $N_{t,j}$ denotes the j-th node (counting from the left, starting at 0) on level t. Each level t has 2^{d-t} nodes, so the levels shrink by a factor of two as one ascends, the root being a single ℓ-bit string. The construction iterates the application of h arranged in a balanced binary fan-in.

Example 10.3 (A tree of depth 2)**.** With $d = 2$ and leaves L_0, L_1, L_2, L_3, the tree has seven nodes. The level-1 labels are

$$N_{1,0} = h(L_0 \| L_1), \qquad N_{1,1} = h(L_2 \| L_3),$$

and the root is

$$\mathsf{root}(L) = N_{2,0} = h\big(\, h(L_0 \| L_1) \,\|\, h(L_2 \| L_3) \,\big).$$

Pictorially: four leaves along the bottom, two internal nodes above them each hashing a pair, and the root above those two hashing the pair of internal nodes. The single ℓ-bit root summarises the whole list of four ℓ-bit strings.

Remark 10.4 (Padding to a power of two)*.* We handle a list whose length m is not a power of two by fixing a depth d with $2^d \ge m$ and *padding* the remaining $2^d - m$ leaf slots with a canonical value, e.g. the all-zero string or the hash of the empty string. Padding must not create ambiguity: two different lists must never yield the same padded leaf sequence. Including the true length m in the domain-separated computation of the root (Subsection 10.5) removes any such ambiguity. An alternative is the unbalanced tree, in which a node with a single child passes that child's label upward unchanged — a convenient but subtle choice we discuss together with second-preimage attacks in Remark 10.17.

The root is intended to serve as a commitment: a short string that determines the list. The next definition and proposition make this precise.

Definition 10.5 (Merkle commitment scheme)**.** The *Merkle commitment* to a list L of 2^d leaves is the value $\mathsf{root}(L) \in \{0,1\}^{\ell}$. The scheme is *binding* if no PPT adversary can produce two distinct lists $L \neq L'$ of equal length with $\mathsf{root}(L) = \mathsf{root}(L')$, except with negligible probability.

Proposition 10.6 (Binding of the root)**.** *If h is collision resistant, then the Merkle commitment is binding. Concretely, from any two distinct equal-length lists $L \neq L'$ with $\mathsf{root}(L) = \mathsf{root}(L')$ one can extract, in time linear in the tree size, a collision for h.*

Proof. Let L and L' be distinct lists of 2^d leaves with equal roots, and write $N_{t,j}$ and $N'_{t,j}$ for the node labels of the two trees. Because the lists differ, there is at least one leaf index j_0 with $N_{0,j_0} \neq N'_{0,j_0}$. Consider the leaf-to-root path

$$N_{0,j_0},\ N_{1,\lfloor j_0/2 \rfloor},\ N_{2,\lfloor j_0/4 \rfloor},\ \ldots,\ N_{d,0}$$

in the first tree and the correspondingly indexed path in the second. At level 0 the two paths carry different labels, by choice of j_0; at level d they carry the *same* label, namely the common root. Hence as one ascends there is a smallest level t at which the two labels first agree:

$$N_{t,j} = N'_{t,j} \quad \text{but} \quad N_{t-1,2j} \neq N'_{t-1,2j} \ \text{ or } \ N_{t-1,2j+1} \neq N'_{t-1,2j+1},$$

where $j = \lfloor j_0/2^t \rfloor$. By the recurrence $(*)$,

$$h\big(N_{t-1,2j} \| N_{t-1,2j+1}\big) = N_{t,j} = N'_{t,j} = h\big(N'_{t-1,2j} \| N'_{t-1,2j+1}\big),$$

while the two 2ℓ-bit inputs $N_{t-1,2j}\|N_{t-1,2j+1}$ and $N'_{t-1,2j}\|N'_{t-1,2j+1}$ differ, since they disagree in at least one of the two halves. This is a collision for h. The extraction walks one root-to-leaf path and so costs $O(d)$ hash comparisons.

Consequently, a PPT adversary breaking binding with non-negligible probability ε yields a PPT adversary finding a collision with the same probability ε, contradicting collision resistance. Hence $\varepsilon = \mathrm{negl}(\lambda)$. □

The argument in Proposition 10.6 is the prototype of every Merkle security proof: *a discrepancy at the leaves and agreement at the root must, by the pigeonhole-like "first level of agreement" argument, manifest a collision somewhere on the path.* We reuse it almost verbatim for authentication paths.

10.3 Authentication paths and membership proofs

The root binds the whole list, but a verifier who knows only the root should be able to check a claim about a single leaf without re-reading the list. The data that enables this is the sequence of *siblings* along the leaf-to-root path.

Definition 10.7 (Authentication path)**.** Let T be a Merkle tree of depth d over L, and fix a leaf index i with $0 \leq i < 2^d$. Write the binary expansion of i as $b_{d-1}b_{d-2}\cdots b_0$, so that $b_t \in \{0,1\}$ records, at level t, whether the relevant node is a left child ($b_t = 0$) or a right child ($b_t = 1$). The *authentication path* (or *Merkle proof*, or *membership witness*) for index i is the sequence

$$\pi_i \ = \ \big((s_0, b_0),\ (s_1, b_1),\ \ldots,\ (s_{d-1}, b_{d-1})\big),$$

where s_t is the label of the *sibling* of the level-t node lying on the path from leaf i to the root. Explicitly, with $j_t = \lfloor i/2^t \rfloor$ the index of the path node at level t,

$$s_t \ = \ \begin{cases} N_{t,\,j_t+1}, & b_t = 0 \quad \text{(path node is a left child; sibling on the right)},\\ N_{t,\,j_t-1}, & b_t = 1 \quad \text{(path node is a right child; sibling on the left)}. \end{cases}$$

The path has exactly d entries.

The verifier, given the claimed leaf value, the index i, the authentication path, and the trusted root, recomputes the chain of labels up the tree and checks that it arrives at the root.

Definition 10.8 (Path verification)**.** Define $\mathsf{Verify}(\mathsf{rt}, i, v, \pi_i)$, where rt is a root, i an index, $v \in \{0,1\}^{\ell}$ a claimed leaf value, and $\pi_i = ((s_0, b_0), \ldots, (s_{d-1}, b_{d-1}))$ a candidate path, as follows. First check the path against the index: output reject unless, for every t, the bit b_t carried in π_i equals bit t of the binary expansion of i. Then set $a_0 := v$, and for $t = 0, 1, \ldots, d-1$ compute

$$a_{t+1} := \begin{cases} h(a_t \,\|\, s_t), & b_t = 0, \\ h(s_t \,\|\, a_t), & b_t = 1. \end{cases} \tag{$\dagger$}$$

Output accept if $a_d = \mathsf{rt}$ and reject otherwise. We term the value a_d the *root recomputed from* (i, v, π_i).

The bit b_t indicates on which side the verifier places the running value a_t before compressing, so that the order of arguments to h matches the order used when building the tree. The recomputation ($\dagger$) folds the leaf upward through the supplied siblings, one compression per level. The following completeness statement is immediate.

Proposition 10.9 (Completeness)**.** *For every list L, every index i, and π_i the genuine authentication path for i in the tree over L,* $\mathsf{Verify}(\mathsf{root}(L), i, L_i, \pi_i) = \mathsf{accept}$.

Proof. By induction on t we show $a_t = N_{t,\lfloor i/2^t \rfloor}$, i.e. the running value equals the genuine label of the path node at level t. The base case $a_0 = L_i = N_{0,i}$ holds by definition. For the step, the path node at level t and its sibling s_t are the two children of the path node at level $t+1$; placing a_t on the side dictated by b_t and s_t on the other reproduces exactly the two arguments of h in the recurrence ($*$), so $a_{t+1} = N_{t+1,\lfloor i/2^{t+1} \rfloor}$. At $t = d$ we obtain $a_d = N_{d,0} = \mathsf{root}(L)$. □

Proposition 10.10 (Logarithmic proof size)**.** *An authentication path in a tree of depth d consists of d sibling labels and d direction bits, for a total of $d \cdot \ell + d$ bits. Verification performs exactly d evaluations of h. Choosing the minimal sufficient depth $d = \lceil \log_2 m \rceil$ for a list of $m \geq 2$ leaves yields proofs of $\lceil \log_2 m \rceil (\ell + 1)$ bits, logarithmic in the list length.*

Proof. Immediate from Definitions 10.7 and 10.8: there is one sibling and one compression per level, and there are d levels above the leaves. A depth-d tree accommodates m leaves precisely when $2^d \geq m$ (Remark 10.4), so $\lceil \log_2 m \rceil$ is the minimal sufficient depth. □

Thus a membership proof for a set of a billion elements (2^{30}) consists of 30 hash values and 30 bits — under a kilobyte with a 256-bit hash — regardless of how the billion elements are arranged. This logarithmic succinctness is the central objective.

Example 10.11 (A depth-2 membership proof)**.** Continue Example 10.3 and prove membership of L_2 at index $i = 2 = (10)_2$, so $b_0 = 0,\ b_1 = 1$. At level 0 the path node is $N_{0,2} = L_2$, a left child, so the sibling is $s_0 = N_{0,3} = L_3$. At level 1 the path node is $N_{1,1} = h(L_2 \| L_3)$, a right child, so the sibling is $s_1 = N_{1,0} = h(L_0 \| L_1)$. The path is $\pi_2 = ((L_3, 0), (h(L_0 \| L_1), 1))$. Verification computes

$$a_1 = h(L_2 \| L_3), \qquad a_2 = h\big(h(L_0 \| L_1) \,\|\, a_1\big),$$

and indeed $a_2 = \mathsf{root}(L)$. The verifier never saw L_0 or L_1 themselves — only the digest $h(L_0 \| L_1)$ — yet is convinced L_2 sits at index 2 of the committed list.

10.4 Soundness: forging a path implies a collision

We now establish the security guarantee that renders the membership proof meaningful: against a fixed, honestly computed root, no efficient adversary can produce an accepting path for a leaf value that is not the one actually committed at that position. "Cannot" is, as always, conditional on collision resistance.

Definition 10.12 (Path forgery)**.** Fix a list L of 2^d leaves with root $\mathsf{rt} = \mathsf{root}(L)$. A *path forgery* at index i is a pair $(v^\star, \pi^\star)$ with

$$\mathsf{Verify}(\mathsf{rt}, i, v^\star, \pi^\star) = \mathsf{accept} \qquad \text{and} \qquad v^\star \neq L_i.$$

That is, the adversary makes the verifier accept a leaf value different from the genuine one at position i, against the genuine root.

Theorem 10.13 (Soundness of Merkle membership proofs)**.** *Let h be collision resistant. Then no PPT adversary, given the list L (equivalently, given the whole tree), produces a path forgery except with negligible probability. More precisely, there is a deterministic linear-time extractor that, from any list L, index i, and accepting forgery $(v^\star, \pi^\star)$ with $v^\star \neq L_i$, outputs a collision for h.*

Proof. Let $\mathsf{rt} = \mathsf{root}(L)$ and let $\pi^\star = ((s_0^\star, b_0^\star), \dots, (s_{d-1}^\star, b_{d-1}^\star))$ be the forged path. Since the forgery is accepting, the index-consistency check of Definition 10.8 guarantees $b_t^\star = b_t$, bit t of i, for every t.

Run the verifier, obtaining the recomputed chain $a_0^\star = v^\star,\ a_1^\star, \dots, a_d^\star$ defined by (†); by hypothesis $a_d^\star = \mathsf{rt}$. In parallel, let $a_0, a_1, \dots, a_d$ be the genuine chain for the true leaf L_i, i.e. $a_t = N_{t,\lfloor i/2^t \rfloor}$ with genuine siblings; by Proposition 10.9, $a_d = \mathsf{rt}$ as well.

There are now two chains that *disagree at the bottom* — $a_0^\star = v^\star \neq L_i = a_0$ — yet *agree at the top*, $a_d^\star = a_d = \mathsf{rt}$. Let $t^\star$ be the smallest level at which they agree:

$$t^\star = \min\{\, t \,:\, a_t^\star = a_t \,\} \in \{1, \dots, d\}, \qquad \text{so} \qquad a_{t^\star}^\star = a_{t^\star} \ \text{ but } \ a_{t^\star-1}^\star \neq a_{t^\star-1}.$$

Such a $t^\star$ exists because the chains disagree at level 0 and agree at level d. Write $t = t^\star$ and $b = b_{t-1}$ $(= b_{t-1}^\star)$. The recurrence (†) computes the two equal values at level t as

$$a_t^\star = \begin{cases} h(a_{t-1}^\star \| s_{t-1}^\star), & b = 0 \\ h(s_{t-1}^\star \| a_{t-1}^\star), & b = 1 \end{cases} \qquad a_t = \begin{cases} h(a_{t-1} \| s_{t-1}), & b = 0 \\ h(s_{t-1} \| a_{t-1}), & b = 1, \end{cases}$$

where s_{t-1} is the genuine sibling and $s_{t-1}^\star$ the supplied one. Let

$$X^\star = \begin{cases} a_{t-1}^\star \| s_{t-1}^\star, & b = 0 \\ s_{t-1}^\star \| a_{t-1}^\star, & b = 1 \end{cases} \qquad X = \begin{cases} a_{t-1} \| s_{t-1}, & b = 0 \\ s_{t-1} \| a_{t-1}, & b = 1. \end{cases}$$

Then $h(X^\star) = a_t^\star = a_t = h(X)$. Moreover $X^\star \neq X$: in the half occupied by the running value, $a_{t-1}^\star \neq a_{t-1}$ by minimality of $t^\star$, so the two 2ℓ-bit strings differ in that half. Hence $(X^\star, X)$ is a collision for h. The extractor runs the verifier once and compares the two chains level by level, costing $O(d)$ work.

Finally, suppose a PPT adversary $\mathcal{A}$ produced forgeries with non-negligible probability $\varepsilon(\lambda)$. The collision finder $\mathcal{B}$ that samples L (or receives it), runs $\mathcal{A}$, and applies the extractor outputs a collision whenever $\mathcal{A}$ succeeds, hence with probability $\varepsilon(\lambda)$. Collision resistance forces $\varepsilon(\lambda) = \mathrm{negl}(\lambda)$. □

Remark 10.14 (What soundness does and does not promise)*.* Theorem 10.13 is a statement about a *fixed honest root*. It asserts: *relative to a root that genuinely is* $\mathsf{root}(L)$*, the only leaf value that admits an accepting path at position i is L_i (up to a collision in h).* It does *not* by itself prevent an adversary who

is free to choose the root from committing to a maliciously crafted tree; that threat is exactly what binding (Proposition 10.6) rules out, and the two properties together establish that the root is a sound, binding commitment with succinct openings. An untrusted party supplying the root makes both necessary: binding ensures the root determines a unique list, and path soundness ensures openings are faithful to it.

10.5 Layer and domain separation

The proofs above tacitly assume that the only way to produce a label is through the intended recurrence. Real attacks exploit the gaps in that assumption. Two distinct hazards demand attention: confusing a leaf with an internal node, and confusing nodes at different depths or trees.

Second-preimage attacks from leaf/node confusion. Suppose, naively, that leaves are arbitrary-length data hashed by the *same* function used internally, so that an internal label $h(A\|B)$ is itself a valid leaf value (it has the right length ℓ). Then an adversary can present the internal node $N_{1,0} = h(L_0\|L_1)$ as if it were a leaf at the top of a *shallower* tree, producing a shorter authentication path that the verifier — who does not learn the depth — accepts. More generally, absent a way to distinguish "I am a leaf" from "I am an internal node," a single string can play both roles, and the clean leaf-to-root structure on which Theorem 10.13 relies collapses. This is a genuine second-preimage attack: the tree's root has a second list explaining it.

Definition 10.15 (Domain separation)**.** A Merkle construction has *domain (and layer) separation* if it forces the function applied at distinct structural positions to differ, by prepending a position-dependent tag to its input. Concretely, fix distinct tags and define

$$\mathsf{LeafHash}(v) = H(\texttt{0x00} \,\|\, v), \qquad \mathsf{NodeHash}_t(A,B) = H(\texttt{0x01} \,\|\, \langle t\rangle \,\|\, A \,\|\, B),$$

where $\texttt{0x00}, \texttt{0x01}$ distinguish the *layer* (leaf vs. internal) and $\langle t\rangle$ is the encoded *level index*, distinguishing internal nodes by depth. Level-0 labels are $N_{0,j} = \mathsf{LeafHash}(L_j)$ and the recurrence becomes $N_{t+1,j} = \mathsf{NodeHash}_{t+1}(N_{t,2j}, N_{t,2j+1})$.

Proposition 10.16 (Domain separation forces honest collisions)**.** *With the tagging of Definition 10.15, any collision extracted by the proofs of Proposition 10.6 or Theorem 10.13 is a collision for H between two inputs sharing the same leading tag, hence a collision in which a leaf is matched against a leaf and a level-t node against a level-t node. In particular no accepting forgery can substitute an internal node for a leaf, or a node at one level for a node at another, without colliding H within a single domain.*

Proof. Sketch. The extractor of Theorem 10.13 finds the smallest level t at which the genuine and forged chains agree while disagreeing just below. The *same* structural rule at that position forms both colliding inputs — both via $\mathsf{NodeHash}_t$ if $t \geq 1$, or both via $\mathsf{LeafHash}$ if the disagreement is at the leaves — because the verifier's recomputation applies the position's prescribed tagged function, as does the honest tree. Hence the two preimages share the leading tag $\texttt{0x01}\|\langle t\rangle$ (or $\texttt{0x00}$). The cross-domain confusions are therefore impossible unless H collides *within* a fixed domain, which collision resistance of H forbids. □

Remark 10.17 (Unbalanced trees and padding). Deployed failures (notably Bitcoin's CVE-2012-2459, where a duplicate-last-node rule let two different transaction lists share a root) teach a single lesson: layer-separate leaves from internal nodes, make every padding or duplication rule injective on admissible lists (e.g. by binding the true leaf count into the root), and feed $\mathsf{NodeHash}$ only fixed-length inputs so that no length-extension structure of the underlying hash can intervene.

Remark 10.18 (Algebraic hashes and the arithmetic setting). When the Merkle path must be verified *inside a zero-knowledge circuit* (Subsection 10.8), arithmetic gates over a finite field $\mathbb{F}_p$ evaluate the compression function, and bit-oriented hashes like SHA-256 become enormously expensive (tens of thousands of constraints per call). One therefore uses an *algebraic* hash — Poseidon, Rescue, or in Orchard the Sinsemilla function — with compression $h : \mathbb{F}_p^2 \to \mathbb{F}_p$: Poseidon and Rescue are low-degree permutation-based maps computable in few constraints, whereas Sinsemilla is cheap in constraints via lookups and incomplete elliptic-curve additions, not via low degree. For Poseidon and Rescue, domain and layer separation arise not from byte tags but from distinct field constants (initial capacity values or round constants) fed into the permutation; Sinsemilla instead retains the tag mechanism of Definition 10.15 in encoded form: Orchard's MerkleCRH prepends the 10-bit encoded layer index to the two 255-bit child encodings, all layers sharing one instance, while separation between distinct Sinsemilla instances comes from the personalisation string fed to hash-to-curve to derive the instance-specific base point Q (the generator table S is shared across instances). The security argument is identical, with "collision" read as a collision of the field-valued compression function. Sinsemilla in particular is collision resistant under a discrete-log assumption, fusing the hashing and the commitment into one Pedersen-like map.

10.6 Append-only and incrementally updatable trees

In the applications of concern the leaf set is not fixed once and for all: note commitments are *appended* as new transactions arrive, and the tree grows monotonically. Recomputing the root from scratch at each insertion would cost $\Theta(m)$ hashes; $O(\log m)$ per append is the objective. The key observation is that appending a leaf changes only the labels on the single path from the new leaf to the root, and that these labels depend only on a small amount of retained state called the *frontier*.

Definition 10.19 (Append-only Merkle tree)**.** Fix a maximum depth d, giving capacity 2^d. An *append-only* (or *incremental*) Merkle tree maintains a position counter m (the number of leaves inserted so far, $0 \le m \le 2^d$) and supports a single mutating operation $\mathsf{Append}(v)$, which places v at leaf index m and increments m. Empty leaf slots $m, m{+}1, \dots, 2^d{-}1$ carry a fixed *empty-leaf* value $\perp_0$; the empty-subtree labels $\perp_t$ at level t follow by precomputation from $\perp_{t+1} = \mathsf{NodeHash}_{t+1}(\perp_t, \perp_t)$. The root after m appends is root_m, the root of the depth-d tree whose first m leaves are the inserted values and whose remaining leaves are $\perp_0$.

Definition 10.20 (Frontier)**.** The *frontier* of an append-only tree holding m leaves is the minimal state needed to (i) compute the current root and (ii) update incrementally on the next Append. It consists, for each level $t \in \{0, 1, \dots, d-1\}$, of at most one "carry" node: the label of the *left* child of the next node to be completed at level $t+1$, retained precisely when leaf index m has a 1 bit at position t, the completed left sibling awaiting its right neighbour. Equivalently, writing m in binary as $b_{d-1} \cdots b_0$, the frontier stores, for each t with $b_t = 1$, the already-finalised left-sibling label at level t along the rightmost filled path. The frontier thus comprises at most d labels — one per set bit of m — i.e. $O(\log m)$ state.

The frontier is exactly the set of subtree roots that are "complete and waiting for a right neighbour." Equivalently, the binary representation of m is a list of completed perfect subtrees of distinct heights, and the frontier holds their roots; this is the Merkle analogue of a binary counter, and Append is the analogue of incrementing it, where a "carry" at level t corresponds to combining two height-t subtrees into one of height $t+1$.

Proposition 10.21 (Incremental append in logarithmic time)**.** *From the frontier of an append-only tree holding $m < 2^d$ leaves, the operation* Append(v) *computes the new frontier and the new root* root_{m+1} *using at most d evaluations of* NodeHash *and $O(d)$ additional work, while storing only $O(d)$ labels.*

Proof. Place v at level 0 as the running label $c \leftarrow v$ and let $t \leftarrow 0$. The new leaf sits at index m; inspect the bits of m from least significant upward. While bit t of m equals 1, the node at level t on the new leaf's path is a *right* child whose left sibling is the frontier carry f_t stored at level t; set $c \leftarrow \mathsf{NodeHash}_{t+1}(f_t, c)$, clear that carry, and increment t (this is the carry-propagation of the binary counter). When bit t of m equals 0, the path node at level t is a *left* child with no completed right sibling yet; store c as the new carry f_t at level t and stop the propagation. Let z be the number of trailing 1-bits of m. Carry propagation costs exactly z hashes and deposits f_z at level z: the root of the just-completed height-z subtree containing the new leaf, hence the root-path node at that level. To obtain root_{m+1}, fold this value upward from level z to level d, combining at each level with the stored carry f_t (as left sibling) or the precomputed empty-subtree label $\perp_t$ (as right sibling) — one hash per level, i.e. $d - z$ hashes (zero if $z = d$). The total is exactly d evaluations of NodeHash, and the only retained state is the at-most-d carries, i.e. the frontier of $m + 1$ leaves. □

Example 10.22 (Frontier as a binary counter)**.** Take $d = 3$ (capacity 8) and insert leaves v_0, v_1, v_2 in turn. After v_0 ($m = 1 = (001)_2$): one completed height-0 subtree; frontier $= \{f_0 = v_0\}$. After v_1 ($m = 2 = (010)_2$): inserting v_1 at index 1 (a right child) triggers a carry: $c = \mathsf{NodeHash}_1(v_0, v_1)$, which is a completed height-1 subtree; frontier $= \{f_1 = c\}$, and f_0 is cleared. After v_2 ($m = 3 = (011)_2$): v_2 at index 2 is a left child, so it becomes a new height-0 carry; frontier $= \{f_1 = \mathsf{NodeHash}_1(v_0, v_1),\ f_0 = v_2\}$ — exactly the two set bits of 3. Each append touched only the path to the new leaf, never the whole tree.

Definition 10.23 (History binding)**.** An append-only tree is *history-binding* when the protocol treats every historical root as an acceptable commitment in its own right: each membership proof verifies against the root it was formed for, and appending further leaves never invalidates a previously valid (root, leaf, path) triple for that root. In Orchard and similar systems, the chain records each block's note-commitment-tree root, a spend may prove membership against *any* recorded historical root, and the soundness of each individual proof is exactly Theorem 10.13 applied to the corresponding fixed root.

Remark 10.24 (Why append-only, not arbitrary update)*.* Deletion or in-place update of leaves could also be supported; each still touches only one root-to-leaf path and costs $O(\log m)$ given the relevant siblings. But append-only trees enjoy a crucial monotonicity: once a leaf is inserted it is never removed, so a membership proof, once valid against the root at insertion time, remains a valid proof against *that* historical root forever. This is exactly the property a shielded pool requires — a note, once added, can always be spent by proving against some past root — and it obviates the need to track a moving target. The cost is that the verifier must accept a window of historical roots rather than a single current one.

10.7 Vector commitments

Having constructed and analysed the Merkle tree, we now name the abstraction it instantiates: the *vector commitment*, a short commitment to an ordered list with short, position-indexed openings. (The unordered sibling notion, the *cryptographic accumulator*, packages the same construction by set membership instead of position; Orchard never needs that abstraction, so we do not develop it.)

Definition 10.25 (Vector commitment)**.** A *vector commitment* (VC) is a tuple of PPT algorithms (Setup, Commit, Open, Vf):

- $\mathsf{Setup}(1^\lambda, n) \to \mathsf{pp}$ produces public parameters for vectors of length n;
- $\mathsf{Commit}(\mathsf{pp}, (v_0, \dots, v_{n-1})) \to (C, \mathsf{aux})$ outputs a short commitment C and auxiliary state;
- $\mathsf{Open}(\mathsf{pp}, i, \mathsf{aux}) \to \pi_i$ produces a proof that position i holds v_i;
- $\mathsf{Vf}(\mathsf{pp}, C, i, v, \pi) \to \{\mathsf{accept}, \mathsf{reject}\}$ verifies an opening.

It is *correct* if honest openings always verify, and *position-binding* if no PPT adversary can output C, i, v, v', π, π' with $v \neq v'$ and both $\mathsf{Vf}(\mathsf{pp}, C, i, v, \pi)$ and $\mathsf{Vf}(\mathsf{pp}, C, i, v', \pi')$ accepting, except with negligible probability. The VC is *succinct* if $|C|$ and $|\pi_i|$ are bounded by a fixed polynomial in λ and $\log n$ (i.e. independent of n up to logarithmic factors).

Theorem 10.26 (Merkle trees are succinct position-binding vector commitments)**.** *The Merkle construction, with* Setup *sampling h,* Commit = root *(and* aux *the full tree),* Open *returning the authentication path, and* Vf = Verify*, is a correct, position-binding vector commitment of commitment size ℓ and opening size $O(\ell \log n)$, assuming h is collision resistant.*

Proof. Correctness is Proposition 10.9. For position-binding, suppose an adversary outputs a root C, an index i, and two accepting openings (v, π) and (v', π') with $v \neq v'$. Running the verifier on each yields two recomputed chains from the leaf up to the common root C; they agree at the top (both equal C) and disagree at the bottom ($v \neq v'$). Both openings are accepted at the same index i, so by the index-consistency check of Definition 10.8 both carry i's direction bits, and at each level the two chains apply h with the running value on the same side. The "first level of agreement" extraction of Theorem 10.13, applied to these two chains, therefore yields two distinct equal-length inputs to h with the same image, that is, a collision. (This argument requires *no honest reference tree*: it pits the two forged openings against each other, so it binds even a maliciously chosen root, which is the strong form of position-binding.) Collision resistance bounds the success probability by $\mathrm{negl}(\lambda)$. The size bounds are Proposition 10.10. □

Remark 10.27 (Position-binding vs. path soundness)*.* Theorem 10.13 fixed an honest root and forbade opening a position to anything but its true value; Theorem 10.26 forbids opening a single position two *different* ways, even under an adversarial root. The latter is precisely the property a verifier requires who receives a root from an untrusted source, and it is what "commitment" should mean: C determines at most one value per position. Both reduce to the same collision extraction; the difference lies only in which pair of chains we collide.

10.8 Privacy-preserving membership via zero-knowledge paths

We can now assemble the primitive that Orchard actually uses. The membership proof of Subsection 10.3 is succinct but *not private*: revealing the authentication path discloses the leaf value (e.g. the note commitment) and the index i, and the siblings can leak structural information. For a shielded payment system this is fatal: spending a note must not reveal *which* note the spender spends. The remedy is to prove the existence of a valid path *in zero knowledge*, treating the path as a private witness.

Definition 10.28 (Membership relation for ZK)**.** Fix the domain-separated Merkle construction

of depth d. Define the *membership relation*

$$\mathcal{R}_{\mathsf{mem}} = \{(\underbrace{\mathsf{rt}}_{\text{statement}} ; \underbrace{(v, i, \pi)}_{\text{witness}}) : \mathsf{Verify}(\mathsf{rt}, i, v, \pi) = \mathsf{accept}\},$$

in which the *statement* is the public root rt and the *witness* comprises the leaf v, its index i, and the authentication path π. A zero-knowledge argument for $\mathcal{R}_{\mathsf{mem}}$ enables a prover to convince a verifier that "some leaf is in the tree with root rt" while revealing none of v, i, or π.

We assume a *zk-SNARK* as Section 9 develops it (Definition 9.26): a non-interactive argument with three properties — *completeness* (honest provers convince), *knowledge soundness* (an accepting prover "knows" a witness, formalised by an extractor), and *zero knowledge* (the proof reveals nothing beyond the truth of the statement, formalised by a simulator). The membership circuit encodes exactly the verifier's folding computation (†).

Definition 10.29 (In-circuit path verification)**.** The *Merkle-path circuit* C_{mem} of depth d takes as public input the root rt and as private (witness) inputs the leaf v, the direction bits $b_0, \dots, b_{d-1}$, and the siblings $s_0, \dots, s_{d-1}$. It implements d copies of the compression h and the conditional swap of (†) — gate-for-gate, the swap selects (a_t, s_t) or (s_t, a_t) according to b_t — and enforces the single output constraint $a_d = \mathsf{rt}$. Its size is $d \cdot (\text{cost of one } h) + O(d)$ constraints. The circuit witnesses the position only through the bits $b_0, \dots, b_{d-1}$, the binary decomposition of i, so the index-consistency check of Definition 10.8 is enforced structurally — exactly as Orchard's circuit decomposes the witnessed position into the swap bits.

Theorem 10.30 (Privacy-preserving membership)**.** *Let h be collision resistant and let* (P, V) *be a zk-SNARK for the relation $\mathcal{R}_{\mathsf{mem}}$ (equivalently, for the satisfiability of C_{mem}). Then the protocol "prover sends a SNARK proof for* rt*, verifier checks it" is a membership argument that is*

1. complete*: a prover holding a genuine leaf and its path produces an accepting proof;*
2. sound*: any prover producing an accepting proof for an honest root* $\mathsf{rt} = \mathsf{root}(L)$ *knows (via the SNARK extractor) a witness* (v, i, π) *with* $\mathsf{Verify}(\mathsf{rt}, i, v, \pi) = \mathsf{accept}$*; by Theorem 10.13, that v equals the genuine leaf L_i unless it thereby finds a collision in h — so the proven leaf is a real member;*
3. private (zero-knowledge)*: the proof reveals nothing about v, i, or π beyond the bare fact that* some *valid leaf exists under* rt.

Proof. Completeness is the completeness of the SNARK composed with Proposition 10.9: a genuine (v, i, π) satisfies C_{mem}, so the verifier accepts the honest prover's proof. For soundness, knowledge soundness of the SNARK yields an extractor that, from any accepting prover, outputs a witness (v, i, π) satisfying C_{mem}, i.e. with $\mathsf{Verify}(\mathsf{rt}, i, v, \pi) = \mathsf{accept}$. If rt is an honest root $\mathsf{root}(L)$ and the extracted $v \neq L_i$, then (v, π) is a path forgery and Theorem 10.13 converts it into a collision for h; under collision resistance this occurs only with negligible probability, so the extracted v is genuinely $L_i \in L$. Zero knowledge carries over verbatim from the SNARK: its simulator produces, from the statement rt alone, a proof indistinguishable from the real one, so the real proof leaks nothing about the witness (v, i, π) — in particular not the spent note's value or position. □

Remark 10.31 (Why the path is hidden but the root is public)*.* The asymmetry is essential. The root must be *public* so that the verifier can check the proof against a state on which everyone agrees (a previously published note-commitment-tree root); soundness is relative to that public root, exactly as in Remark 10.14. The path must be *private* so that observers cannot link the spend to a particular

note. Because the SNARK is succinct, the verifier checks a proof whose *size* is essentially independent of d—polylogarithmic in it—even though the underlying witness has d siblings (pairing-based schemes such as KZG achieve constant-size proofs with fast verification; Halo 2's IPA-based proofs grow logarithmically in size, but their verification is linear in the circuit size, amortised in deployment by batch-verifying many proofs in a single multiexponentiation—the accumulation technique that would instead make per-proof work sublinear is not deployed): the construction compresses the membership proof while simultaneously hiding it. This is the precise sense in which "a Merkle root over note commitments gives a privacy-preserving membership primitive when the path is proved in zero knowledge."

Example 10.32 (A shielded spend, end to end)**.** In Orchard, each shielded output appends a *note commitment* cm — itself a hiding, binding commitment to the note's value, recipient, and randomness — as a leaf of an append-only Merkle tree built with the Sinsemilla compression function (strictly, the leaf is the extracted x-coordinate $\mathsf{cm}_x = \mathsf{ExtractP}(\mathsf{cm})$, the 255-bit base-field element matching the child encodings of Remark 10.18). Periodically the protocol publishes the current root (the "anchor") on chain. To spend a note, the owner:

1. locates the leaf cm and reads off its authentication path π and position i from the public tree;
2. chooses a recent anchor $\mathsf{rt} = \mathsf{root}_m$ that contains cm;
3. proves, in zero knowledge, the statement rt with witness (cm, i, π): that cm verifies against rt (membership), that the note opening is well-formed, and that the proof computes a derived *nullifier* correctly to prevent double-spending;
4. publishes the proof and the nullifier (but neither cm, nor i, nor π).

By Theorem 10.30, the spend convinces the network that the spender owns *some* note genuinely added to the tree — soundness rests on collision resistance of Sinsemilla and knowledge soundness of the proof system — yet the network learns nothing about *which* note, since zero knowledge hides the entire path. The Merkle root thus serves as a succinct, binding, privacy-preserving commitment to the set of all notes ever created, and the authentication path, proved in zero knowledge, is the membership primitive on which shielded spends rest.

10.9 Summary

We constructed the binary Merkle tree from a single collision-resistant compression function (Definition 10.2); showed that the root is a binding commitment to the leaf list (Proposition 10.6); defined authentication paths and proved them complete, logarithmic in size, and sound — forging one yields a collision (Propositions 10.9, 10.10, Theorem 10.13); explained why layer and domain separation must hold to make those reductions honest (Subsection 10.5); developed append-only trees and the frontier that updates the root in $O(\log m)$ per insertion (Subsection 10.6); abstracted the construction as a succinct position-binding vector commitment (Subsection 10.7); and finally combined a Merkle root over note commitments with a zero-knowledge proof of an authentication path to obtain the privacy-preserving membership primitive at the heart of Orchard (Theorem 10.30, Example 10.32). The single recurring proof technique — "disagreement at the leaves plus agreement at the root forces a collision on the path" — underlies binding, path soundness, and position-binding alike, and is the essential idea of the entire construction.